PRISM: A LIDAR-DRIVEN, PCU-WEIGHTED PRESSURE-BASED ADAPTIVE TRAFFIC SIGNAL CONTROLLER FOR HETEROGENEOUS SRI LANKAN TRAFFIC - A DRONE-CALIBRATED SUMO STUDY OF THE MAVITHTHARA JUNCTION

A thesis submitted to NSBM Green University for the degree of Bachelor of Science (Honours) in Computer Science

By

P R S PERERA

(29243)

Department of Computer & Data Science

Faculty of Computing

NSBM Green University

Sri Lanka

October 2026

DECLARATION

I declare that the content of this undergraduate thesis titled "PRISM: A LiDAR-Driven, PCU-Weighted Pressure-Based Adaptive Traffic Signal Controller for Heterogeneous Sri Lankan Traffic — A Drone-Calibrated SUMO Study of the Mavithara Junction" is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for any other degree in any university or institution of higher learning.

Signature: _____

Name: P R S Perera (29243)

Date: _____

_____

Signature of the Supervisor

Mrs. Hirushi Dilpriya
Principal Supervisor
Lecturer
Department of Computer & Data Science
NSBM Green University

Date: _____

ACKNOWLEDGEMENT

The author wishes to express sincere gratitude to the supervisor of this research, Mrs. Hirushi Dilpriya, Lecturer in the Faculty of Computing, NSBM Green University, for her continuous guidance, patience and constructive feedback from the first proposal to the final draft. Her insistence on grounding an ambitious idea in careful measurement and honest evaluation shaped every chapter of this thesis.

The drone survey on which this work rests would not have been possible without Mr. A M Lakshitha Chulanga Bandara, the CAASL-registered drone operator who planned and flew both recording sessions at the Mavithhara junction with great care and professionalism. The author is equally grateful to Mr. Ravindu Sulakshana, a fellow undergraduate, whose persistence in preparing letters and following up with the authorities was instrumental in obtaining the permissions required for the survey.

The author gratefully acknowledges the Piliyandala Traffic Police and the Road Development Authority for discussing the operational problems of the junction, for supporting this research and for authorising the aerial survey, as well as the Civil Aviation Authority of Sri Lanka and the Ministry of Defence for granting the necessary approvals.

Thanks are also due to the lecturers and academic staff of the Faculty of Computing, NSBM Green University, whose teaching provided the foundation on which this work was built. Finally, the author owes the deepest thanks to family members, whose daily journeys through Piliyandala traffic first inspired this research and whose support and encouragement made its completion possible.

ABSTRACT

Signalised junctions in the Colombo District are still operated by fixed-time plans that cannot respond to fluctuating, highly heterogeneous demand in which motorcycles and three-wheelers filter between cars, buses and lorries; when queues grow excessive, traffic police switch the signals off and direct traffic manually. This research designed, implemented and evaluated PRISM (Pressure-based Real-time Intersection Signal Management), an adaptive signal controller for the Mavithhara junction on the Piliyandala bypass. A drone survey under police, Ministry of Defence and Civil Aviation Authority approval recorded traffic at the junction, and a Drone Trajectory eXtractor combining frame stabilisation, a fine-tuned YOLOv8s detector with sliced inference, ByteTrack tracking and homography-based georeferencing converted the video into vehicle trajectories, demand, composition, turning ratios, saturation flow and waiting times. These data calibrated a sublane SUMO model of the junction and parameterised ten demand scenarios representing its operating conditions: balanced flow at 300, 500 and 700 veh/h per approach, an empty side road, unbalanced roads, a peak, a platoon surge, night, heavy vehicles and sensor failure. A Virtual LiDAR Sensor reproduced the noise, dropout and occlusion of a pole-mounted LiDAR and supplied the only input to PRISM, whose control law is purely mathematical: a passenger-car-unit weighted max-pressure score with a quadratic starvation-urgency term, hysteresis, gap-out and spillback protection. Evaluation used ten held-out random seeds per scenario against fixed-time, actuated and Webster-optimised control. The first evaluation exposed weaknesses in the control law as originally specified, and a revised law, PRISM-R, was developed. PRISM-R reduced the average waiting time per vehicle against the existing 78-second plan in all ten scenarios, on every one of the ten seeds of each (Wilcoxon $p = 0.002$), by 40% on average: for example -25% at 500 veh/h, -33% at 700 veh/h and -85% with an empty side road. At 700 veh/h it lowered the share of 78-second windows that would trigger manual police control from 24% to 5%, and it cut the green time shown to empty approaches from 648 to 328 s per hour. A decision takes about 0.25 ms. Performance degraded gracefully under sensor imperfections. The results indicate that privacy-preserving LiDAR sensing combined with a transparent pressure-based control law is a viable path to adaptive signal control at Sri Lankan junctions.

Keywords: adaptive traffic signal control, max-pressure, LiDAR, heterogeneous traffic, passenger car unit, drone video analytics, SUMO, Sri Lanka

TABLE OF CONTENTS

Declaration ii

Acknowledgement iii

Abstract iv

Table of Contents v

List of Figures x

List of Tables xii

List of Abbreviations xv

1 Introduction 1

1.1 Chapter overview 1

1.2 Problem background 1

1.3 Problem statement 2

1.3.1 General problem 2

1.3.2 Specific problem 2

1.4 Research question 4

1.5 Research motivation 4

1.6 Research aim 5

1.7 Research objectives 5

1.8 Rich picture of the proposed solution 5

1.9 Resource requirements 7

1.9.1 Hardware requirements 7

1.9.2 Software requirements 8

1.10 Project scope 8

1.11 Chapter summary 9

2 Literature Review **10**

2.1 Chapter overview 10

2.2 Conceptual map of the literature 10

2.3 Domain overview 10

2.4 Existing systems, frameworks and designs 12

2.4.1 Fixed-time and vehicle-actuated control 12

2.4.2 Model-based adaptive systems 13

2.4.3 Max-pressure control and its deployments 13

2.4.4 Learning-based systems 14

2.4.5 Comparative assessment 14

2.5 Technological analysis 16

2.5.1 Algorithmic analysis 16

2.5.2 Design analysis 21

2.5.3 Workflow analysis 23

2.6 Reflection 23

2.6.1 Research gap 23

2.6.2 Justification of design decisions 24

2.6.3 Chapter summary 25

3 Methodology **26**

3.1 Research paradigm 26

3.2 Research approach 26

3.3 Research strategy 27

3.4 Fact collection mechanisms 27

3.5 Research methodology execution workflow 28

3.5.1 Problem identification 28

3.5.2 Relevance justification 28

3.5.3 Comparative analysis and gap justification 30

3.5.4 Define and finalise objectives 30
3.5.5 Design, development, data management and handling 30
3.5.6 Evaluation and communication 30
3.6 Project management methodology 31
3.6.1 Project timeline 31
3.6.2 Ethical considerations 31
3.7 Chapter summary 33

4 System Requirement Specification **34**
4.1 Chapter overview 34
4.2 Stakeholder analysis 34
4.3 Operationalisation process 36
4.4 System and model analysis 38
4.4.1 Use case diagram 38
4.4.2 Class diagram 39
4.4.3 Activity diagram 39
4.4.4 Sequence diagrams 42
4.5 Proposed system architecture 45
4.6 Functional and non-functional requirements 47
4.7 Chapter summary 47

5 Implementation **50**
5.1 Chapter overview 50
5.2 Design of the algorithms 50
5.2.1 Drone Trajectory eXtractor (DTX) 50
5.2.2 SUMO network and demand 52
5.2.3 Virtual LiDAR Sensor (VLS) 55
5.2.4 PRISM control algorithm (PRISM-R) 58
5.2.5 Baseline controllers 68
5.3 Technology selection 71

5.4 Key implementation 74
5.5 Chapter summary 77

6 Testing and Evaluation **79**

6.1 Chapter overview 79
6.2 Test plan and test cases 79
6.2.1 Non-functional testing 79
6.2.2 Functional testing 81
6.3 Testing and evaluation workflow 82
6.3.1 Balanced-demand verification 82
6.3.2 DTX accuracy 83
6.3.3 Calibration and validation 84
6.3.4 Controller comparison 85
6.3.5 Sensitivity analysis 89
6.3.6 Discussion 90
6.4 Review of test strategies used 93
6.5 Chapter summary 94

7 Concluding Remarks **95**

7.1 Chapter overview 95
7.2 Accomplishment of research objectives 95
7.3 Problems encountered 97
7.4 Self-reflection 98
7.4.1 Ideology about the research 98
7.4.2 Benefits gained 98
7.4.3 Learning curves 99
7.5 Business insight 99
7.5.1 Real-world application 99
7.6 Future recommendations 100
7.7 Chapter summary 102

References **103**

A Permission Letters **115**

B Full Parameter Table **122**

C Additional Test Cases **128**

D Additional Code Listings **131**

E Phase Signal-State Strings **136**

F Drone Survey Log **138**

G Supervisor Meeting Log **140**

LIST OF FIGURES

1.1 Aerial view of the Mavithara junction, Piliyandala, taken as a nadir (straight-down) image from the DJI Mini 2 drone during the survey of 2 October 2026; the Piliyandala bypass and the cross road meet diagonally, with channelised left-turn slip lanes at each corner 3

1.2 Rich picture of the proposed solution: drone survey, DTX extraction, calibrated SUMO model driven by the drone-derived scenario generator, virtual LiDAR (VLS), the PRISM-R controller acting on the signal heads in a closed loop, and evaluation against the baselines, with the stakeholders around the workflow 6

2.1 Conceptual map of the literature reviewed in this chapter. Bold items were adopted by this research; dashed arrows show how the strands inform one another in PRISM. 11

2.2 Webster's optimal cycle length (Eq. (2.2)) against the intersection flow ratio. $L = 20$ s corresponds to four phases with a 5 s intergreen each, as at the study site. 16

3.1 Project timeline, June–October 2026 (weeks beginning Monday 1 June; S1–S11 are two-week sprints). 32

4.1 Onion model of the stakeholders of the proposed system 36

4.2 Use case diagram of the PRISM research platform 38

4.3 Class diagram of the system components 41

4.4 Activity diagram of the PRISM-R decision loop 43

4.5 Sequence diagram of the DTX extraction pipeline (UC01) 44

4.6 Sequence diagram of one control step (UC07 and UC08) 44

4.7 Sequence diagram of an experiment run (UC04) 45

4.8 Layered architecture of the proposed system 46

5.1 Flowchart of the PRISM-R decision procedure, with the seven ordered rules R1–R7 and cyclic next-phase selection 65

6.1 Testing and evaluation workflow, with the evidence produced at each stage and the one feedback path that revised the control law. 82

6.2 Balanced-demand verification: average waiting time per vehicle at three demand levels, for the existing plan, the control law as first specified and PRISM-R (evaluation seeds 6–15). 83

6.3 Average waiting time per vehicle by controller and scenario (mean over evaluation seeds 6–15). 87

6.4 Wasted green, averaged over the ten scenarios: seconds per hour in which the green phase has no vehicle within 100 m of its stop line while another phase has a stopped vehicle. 89

6.5 Sensitivity of PRISM-R average waiting time (balanced 500 veh/h, evaluation seeds 6–15) to position noise, detection dropout, the hysteresis θ and the through-phase maximum green. The dashed line is the fixed-time plan at 25.7 s; the whole swept range lies between 19.0 and 20.8 s. 90

7.1 Envisaged city-scale architecture: LiDAR-equipped PRISM junctions share platoon forecasts and publish their state to a cloud junction graph that serves commuter routing and speed advice 100

A.1 Request for permission from NSBM Green University to the Officer in Charge, Piliyandala Police Station (12.06.2026) 116

A.2 Letter from Piliyandala Police Station to the Director General, CAASL, supporting the drone survey of 02.10.2026 (22.09.2026; Sinhala; operator details redacted) 117

A.3 Ministry of Defence security clearance, ref. MOD/MLO/DRONE/03/2026 (7969), page 1 (25.09.2026; operator details redacted) 118

A.4 Ministry of Defence security clearance, ref. MOD/MLO/DRONE/03/2026 (7969), page 2 119

A.5 Civil Aviation Authority of Sri Lanka approval to fly a drone for aerial filming, ref. OP/36/2, page 1 (01.10.2026; address redacted) 120

A.6 Civil Aviation Authority of Sri Lanka approval, ref. OP/36/2, page 2 . . . 121

LIST OF TABLES

1.1 Research objectives 6

1.2 Hardware requirements 7

1.3 Software requirements 8

1.4 Project scope 9

2.1 Comparative assessment of existing signal control systems against the requirements of the study site. 15

2.2 Comparison of candidate detectors and trackers for drone-based vehicle extraction. 20

2.3 Qualitative comparison of traffic sensing technologies for junction control (synthesised from [18, 27, 78, 79, 81]). 21

2.4 Feature comparison of representative works and this research (✓ = addressed, ∼ = partially, × = not addressed, – = not applicable). 24

3.1 Research methodology execution workflow. 29

4.1 Stakeholder roles, viewpoints and influence 35

4.2 Operationalisation of stakeholder needs and dataset fields 37

4.3 Use case specification: UC04 Run experiment 39

4.4 Use case specification: UC08 Decide phase (control step) 40

4.5 Use case specification: UC01 Extract trajectories 40

4.6 Functional requirements with MoSCoW priority and objective mapping . 48

4.7 Non-functional requirements with MoSCoW priority 49

5.1 SUMO vehicle types used for Sri Lankan mixed traffic 54

5.2 The seven situation scenarios written by `gen_scenario_routes.py`, in addition to the three balanced scenarios at 300, 500 and 700 veh/h per approach 54

5.3 PRISM-R design parameters and their justification; values that differ from the original specification are marked 67

5.4 Baseline B1: observed fixed-time plan (cycle 78 s) 69
5.5 Hourly demand extracted by DTX from the two survey sessions, used to calibrate the model and to parameterise the scenario generator (PCU/h; L = slip lane, uncontrolled) 70
5.6 Webster plans for B3, computed per scenario from its mean demand (times in s) 70
5.7 Technology selection: language, models and simulator 72
5.8 Technology selection: libraries and tooling 73

6.1 Test levels, tools and environment. 80
6.2 Principal test cases (TC01–TC10). 80
6.3 Computational cost of a PRISM-R decision against the number of objects in range. 81
6.4 Per-class detection performance of YOLOv8s with SAHI (test split). . . . 83
6.5 Hourly demand and turning shares extracted by DTX (Bypass 1/2 and Cross 1/2 denote the two opposing approaches of each road; turning shares in PCU). 84
6.6 Vehicle composition extracted by DTX (% of vehicles). 84
6.7 Calibration: observed and simulated hourly approach volumes (veh/h) with GEH. 85
6.8 Average waiting time per vehicle (s), mean over evaluation seeds 6–15. Standard deviations are omitted for width; every PRISM-R improvement over B1 holds on all ten seeds individually. 86
6.9 Secondary KPIs, existing fixed-time plan (B1) → PRISM-R (mean over evaluation seeds 6–15). 88
6.10 Exact Wilcoxon signed-rank tests on paired per-seed average waiting times (n = 10 seeds per scenario, two-sided). 88
6.11 Stakeholder-oriented indicators: manual-control trigger (share of 78 s windows with any approach queue over 200 m) and phase changes per hour. Scenarios not listed trigger in 0% of windows under both controllers. . . . 89

7.1 Triangulation of the research objectives against the evidence in this thesis 96
7.2 Indicative annual cost–benefit estimate for one junction (LKR) 100

B.1 Complete parameter list of DTX, SUMO, VLS, PRISM, baselines and experiments 122

C.1 Additional test cases (TC11–TC33). 128

E.1 Assumed link-index order of the traffic-light program 136

E.2 Signal-state strings for the four phases, intergreens and fail-safe plan . . . 137

E.3 Fail-safe (B1) program, cycle 78 s 137

F.1 Flight log for 2 October 2026. 138

F.2 Summary of approvals for the drone survey. 139

F.3 DTX processing parameters for the survey footage. 139

G.1 Supervisor meeting log (placeholder entries) 140

LIST OF ABBREVIATIONS

Abbreviation	Definition
AGL	Above Ground Level
AI	Artificial Intelligence
AP	Average Precision
API	Application Programming Interface
ATSC	Adaptive Traffic Signal Control
BCS	British Computer Society (The Chartered Institute for IT)
CAASL	Civil Aviation Authority of Sri Lanka
CARLA	Car Learning to Act (simulator)
CNN	Convolutional Neural Network
CO_2	Carbon Dioxide
COCO	Common Objects in Context (dataset)
CSV	Comma-Separated Values
CVAT	Computer Vision Annotation Tool
DRL	Deep Reinforcement Learning
DSR	Design Science Research
DTX	Drone Trajectory eXtractor
ETA	Estimated Time of Arrival
EWMA	Exponentially Weighted Moving Average
FHWA	Federal Highway Administration
FR	Functional Requirement
GEH	Geoffrey E. Havers statistic (goodness-of-fit for traffic counts)
GLOSA	Green Light Optimised Speed Advisory
GPU	Graphics Processing Unit
GSD	Ground Sample Distance
GT	Ground Truth
GUI	Graphical User Interface
HBEFA	Handbook Emission Factors for Road Transport
HCM	Highway Capacity Manual
IDF1	Identification F1 score

Abbreviation	Definition
IEEE	Institute of Electrical and Electronics Engineers
IoU	Intersection over Union
ITS	Intelligent Transport Systems
KPI	Key Performance Indicator
LiDAR	Light Detection and Ranging
LKR	Sri Lankan Rupee
LoS	Level of Service
mAP	mean Average Precision
MAPE	Mean Absolute Percentage Error
MoD	Ministry of Defence (Sri Lanka)
MoSCoW	Must have, Should have, Could have, Won't have
MOT	Multiple Object Tracking
MOTA	Multiple Object Tracking Accuracy
NFR	Non-Functional Requirement
OPAC	Optimization Policies for Adaptive Control
ORB	Oriented FAST and Rotated BRIEF
OSM	OpenStreetMap
PCU	Passenger Car Unit
PRISM	Pressure-based Real-time Intersection Signal Management
PRISM-R	PRISM, revised control law (Section 5.2.4)
PRISM-GT	PRISM-R run on a ground-truth (oracle) object list
RANSAC	Random Sample Consensus
RDA	Road Development Authority (Sri Lanka)
RHODES	Real-time Hierarchical Optimized Distributed Effective System
RL	Reinforcement Learning
RPAS	Remotely Piloted Aircraft System
RT-DETR	Real-Time Detection Transformer
SAHI	Slicing Aided Hyper Inference
SCATS	Sydney Coordinated Adaptive Traffic System
SCOOT	Split Cycle Offset Optimisation Technique
SLCAIS	Sri Lanka Civil Aviation Implementing Standard
SORT	Simple Online and Realtime Tracking

Abbreviation	Definition
SRQ	Sub-Research Question
SSD	Solid-State Drive
SUMO	Simulation of Urban MObility
SURTRAC	Scalable Urban Traffic Control
TC	Test Case
TLS	Traffic Light System (SUMO signal program)
TraCI	Traffic Control Interface
UAV	Unmanned Aerial Vehicle
UC	Use Case
UGC	University Grants Commission (Sri Lanka)
UML	Unified Modelling Language
VLOS	Visual Line of Sight
VLS	Virtual LiDAR Sensor
XML	Extensible Markup Language
YOLO	You Only Look Once

1 INTRODUCTION

1.1 Chapter overview

This chapter describes urban congestion in Sri Lanka and the limitations of fixed-time signals, then narrows to the Maviththara junction on the Piliyandala bypass, where the specific problem and research gap are stated. The research question, motivation, aim and objectives follow, and a rich picture summarises the proposed solution, PRISM (Pressure-based Real-time Intersection Signal Management). The chapter closes with the resources used, the project scope and an outline of the remaining chapters.

1.2 Problem background

Road congestion is one of the most persistent constraints on economic and social life in the Colombo Metropolitan Region (CMR). The Urban Transport System Development Project for the Colombo Metropolitan Region and Suburbs, conducted by the Japan International Cooperation Agency together with the Ministry of Transport, documented a rapidly growing private fleet, a declining public-transport share and severe peak-period congestion on the corridors feeding the city, and projected further deterioration without investment in infrastructure and traffic management [1]. More recent studies confirm the trend: CMR commuters report losses of time, fuel, earnings and productivity [2], and the cost of lost workforce time and excess fuel is a substantial drain on the regional economy [3]. Kumarage [4] further argues that Sri Lankan transport policy has for decades lacked effective, evidence-based direction, so that operational improvements have lagged behind demand.

The composition of the Sri Lankan fleet makes the problem distinctive. Of more than 9.2 million registered vehicles, over 5.2 million are motorcycles [5], and motorcycles accounted for 223,228 of the 312,177 new registrations in 2025 alone, when vehicle imports resumed after the economic crisis [6, 7]. Three-wheelers form a further large, distinctively South Asian class. These small vehicles share the carriageway with cars, buses and lorries without strict lane discipline, filtering between larger vehicles to the stop line. A queue of twenty motorcycles and one of twenty buses differ greatly in length and discharge time, which is why traffic engineers express demand in Passenger Car Units (PCU) that weight each class by its effect on capacity [8, 9]. Recent Sri Lankan measurements confirm that locally derived equivalence factors differ markedly from foreign defaults [10].

Most signalised junctions in the CMR are operated by *fixed-time* controllers, which repeat a predetermined cycle with constant green intervals regardless of the traffic actually present. Fixed plans are classically derived from Webster's delay formula [11] using historical counts

and perform reasonably only when demand is stable [12]. Suburban Colombo demand is not stable: school traffic, office shifts, bus bunching and incidents produce surges a fixed plan cannot detect, so one approach queues for hundreds of metres while another, almost empty, receives green. When the plan visibly fails, traffic police officers switch the signals off and direct traffic by hand. Manual control works because the officer responds to visible queues, but it is labour-intensive, inconsistent and hazardous, and its prevalence shows that responsiveness to queues is exactly what the installed controllers lack. Adaptive Traffic Signal Control (ATSC) systems such as SCOOT [13] and SCATS [14] have provided this responsiveness elsewhere for decades, but they depend on dense inductive-loop detection, proprietary hardware and lane-disciplined traffic, and they have seen little adoption in Sri Lanka.

1.3 Problem statement

1.3.1 General problem

Signalised junctions in Sri Lankan cities allocate green time without measuring demand, and the commercial adaptive systems that could respond were designed for lane-disciplined traffic and loop detectors that cannot see filtering motorcycles. Recent local work has begun to apply artificial intelligence and Reinforcement Learning (RL) to signal timing in Sri Lanka [15, 16], but RL controllers require large amounts of training interaction, behave as black boxes and offer no guaranteed bounds on safety or fairness, which remains a major obstacle to their deployment by road authorities [17]. Road users therefore face long, unpredictable waits, and police officers substitute for the controller at the busiest times.

1.3.2 Specific problem

This research concentrates on the Mavithara junction in Piliyandala, Colombo District, where the Piliyandala bypass (the main road) crosses a local cross road (Fig. 1.1). Every arm has two inbound and two outbound lanes, and because Sri Lanka drives on the left, right-turning vehicles must cross the opposing stream. Each corner has a channelised left-turn slip lane controlled by give-way rather than by the signals. Opposite approaches of the same road are served together, giving four signal phases: ϕ_1 bypass through, ϕ_2 bypass protected right turn, ϕ_3 cross-road through and ϕ_4 cross-road protected right turn.

Observation of the junction established the existing fixed-time plan. Each phase is followed by an intergreen of 3 s amber and an all-red interval, assumed in this research to be 2 s because it could not be timed reliably from the footage. The bypass through phase ϕ_1 receives 21 s of green (red 54 s) and the bypass right-turn phase ϕ_2 8 s (red 67 s), giving a cycle of $21 + 3 + 54 = 78$ s. The cross-road through green follows from its observed red of 60 s as $78 - 3 - 60 = 15$ s. The green of ϕ_4 could not be timed directly and was derived from the



Figure 1.1: Aerial view of the Mavilthara junction, Piliyandala, taken as a nadir (straight-down) image from the DJI Mini 2 drone during the survey of 2 October 2026; the Piliyandala bypass and the cross road meet diagonally, with channelised left-turn slip lanes at each corner

cycle constraint $21 + 8 + 15 + G_4 + 4 \times 5 = 78$, which gives $G_4 = 14$ s. The same cycle runs in the saturated morning peak and in the moderate afternoon. Waiting times measured from the drone footage under this plan averaged 104.7 s per vehicle in the morning peak and 46.3 s in the early afternoon, with observed demand of 3,420 veh/h and 2,280 veh/h respectively, motorcycles and three-wheelers together forming 62% of the morning flow.

Three failures of this plan are visible in the footage and are the ones this research set out to remove. First, the signal regularly shows green to an approach on which no vehicle is present, while vehicles stand at red on another arm: because the splits are fixed, green that an empty approach cannot use is simply lost, and this research measures it directly as *wasted green*. Second, the plan cannot react to a surge. A bus platoon or a school-run peak arriving on one arm is served at the same 21, 8, 15 and 14 s as an empty hour, so the queue grows over several cycles until it blocks the slip-lane entries upstream. Third, and most visibly to a road user, the plan makes vehicles wait at a junction that is otherwise empty: at night and in the inter-peak the cross road carries very little traffic, yet the bypass still stops for the full cross-road phases. Each of these is a failure of *allocation* rather than of capacity, which is what makes an adaptive control law, rather than new road space, the appropriate remedy.

At the heaviest periods the Piliyandala Traffic Police switch the lights off and direct traffic manually. In a discussion held on 17 September 2026, the Piliyandala Traffic Police and the Road Development Authority (RDA) expressed two needs: (a) to reduce the time that traffic police officers spend managing the junction manually, and (b) to reduce the waiting time

of every road user. Emergency vehicles currently rely on other traffic making way, and no priority logic was requested at this stage.

Making the junction adaptive requires both *sensing* and a *control law*. Inductive loops measure presence at a point, cannot reliably detect filtering motorcycles and require cutting the road surface; cameras suffer under glare, rain and darkness and capture faces and number plates. A pole-mounted Light Detection and Ranging (LiDAR) sensor measures the position of every object directly [18, 19], but it is expensive, and procuring one before demonstrating its benefit at this site would be irresponsible. The **research gap** is therefore as follows: no existing work has designed and evaluated a transparent, mathematically defined adaptive controller that (i) weights heterogeneous Sri Lankan traffic by PCU, (ii) operates on realistic, imperfect LiDAR observations, (iii) guarantees safety and bounded fairness, and (iv) is evaluated on a simulation model of a real Sri Lankan junction calibrated against per-vehicle waiting times measured from drone video. This research addresses that gap at the Maviththara junction.

1.4 Research question

The research was guided by one principal question and three sub-questions.

Research question (RQ): *How can a LiDAR-driven, PCU-weighted, pressure-based adaptive signal controller, calibrated with drone-observed heterogeneous traffic, reduce the average waiting time of road users at the Maviththara junction compared with the existing fixed-time signal plan?*

SRQ1 What are the demand, composition, turning and waiting characteristics of traffic at the Maviththara junction; how can they be extracted reliably from nadir drone video; and which of the resulting traffic situations make the fixed-time plan inefficient, so that they can be represented as demand scenarios in simulation?

SRQ2 How can a purely mathematical control law account for heterogeneous vehicle classes, fairness between phases and imperfect LiDAR sensing?

SRQ3 How does the proposed controller perform against fixed-time, actuated and Webster-optimised control across representative demand scenarios, and how sensitive is it to sensor noise and to its own parameters?

1.5 Research motivation

The motivation is both personal and societal. A commuter leaving Piliyandala for Colombo at 07:00 spends much of the journey stationary at isolated fixed-time junctions, Maviththara among the first, and every unnecessary second of red is multiplied by thousands of vehicles per peak hour. The author's long-term vision is that every major junction should be

equipped with a LiDAR sensor and an adaptive controller, so that neighbouring junctions can share predicted platoons and a commuter who leaves home can reach the destination in the minimum possible time. Decentralised max-pressure control [20], on which PRISM builds, suits this vision because it needs only local information yet carries network-level throughput guarantees. This thesis is the first, single-junction step towards that goal.

LiDAR was chosen as the target sensor for three reasons. First, it records geometry rather than appearance, capturing the shape, position and speed of a vehicle but not faces or number plates, so a controller that depends only on LiDAR data is privacy-preserving by construction. Second, it is an active sensor that works equally at night and under strong tropical sunlight. Third, it measures position and distance to the stop line directly, rather than inferring them from image coordinates, which allows queues to be measured in PCU over the whole approach.

Finally, machine learning was deliberately kept out of the control loop and used only offline to detect vehicles in drone video. The signal decision is computed by explicit equations whose every output can be explained to a traffic engineer, whose safety and fairness can be bounded and certified, and which require no training data, an important advantage in a country where no historical detector data exist for most junctions [17].

1.6 Research aim

To design, implement and evaluate a LiDAR-driven, PCU-weighted, pressure-based adaptive traffic signal controller (PRISM) that reduces the waiting time of all road users at the Maviththara junction, using a SUMO model calibrated with drone-observed Sri Lankan traffic.

1.7 Research objectives

The aim was decomposed into seven objectives, listed in Table 1.1. Their accomplishment is assessed in Chapter 7.

1.8 Rich picture of the proposed solution

Figure 1.2 presents a rich picture of the proposed solution, emphasising its workflow and the stakeholders around it.

The workflow begins with a drone survey of the junction, approved by the Civil Aviation Authority of Sri Lanka (CAASL) and the Ministry of Defence (MoD) and supported by the Piliyandala Traffic Police. The Drone Trajectory eXtractor (DTX) converts the nadir video into georeferenced vehicle trajectories, from which demand, composition, turning ratios, saturation flow and per-vehicle waiting times are derived. These quantities calibrate

Table 1.1: Research objectives

ID	Objective	SRQ
O1	To identify the operational problems of the Mavithhara junction and the needs of its stakeholders through literature, observation and discussion with the Piliyandala Traffic Police and the RDA.	SRQ1
O2	To analyse existing adaptive signal control systems, control algorithms, sensing technologies and simulation tools critically, and to identify the research gap.	SRQ2
O3	To design and develop a drone-video trajectory extraction pipeline (DTX) that measures demand, composition, turning ratios, saturation flow and per-vehicle waiting time.	SRQ1
O4	To develop a SUMO model of the junction, to calibrate and validate it with the drone-extracted data, and to verify it on a set of demand scenarios representing its operating conditions.	SRQ1
O5	To design and implement a Virtual LiDAR Sensor (VLS) that reproduces realistic LiDAR noise, dropout, occlusion and size-class confusion.	SRQ2
O6	To design and implement the PRISM control algorithm with guaranteed safety, bounded fairness and explainable decisions, using mathematics only (no machine learning in control), and to revise it after evaluation into PRISM-R.	SRQ2
O7	To evaluate PRISM against fixed-time (B1), actuated (B2) and Webster-optimised (B3) control and an oracle-sensor variant (PRISM-GT) over the ten demand scenarios, reporting waiting time, the wasted-green indicator and the computational cost of a decision, with statistical testing and sensitivity analysis.	SRQ3

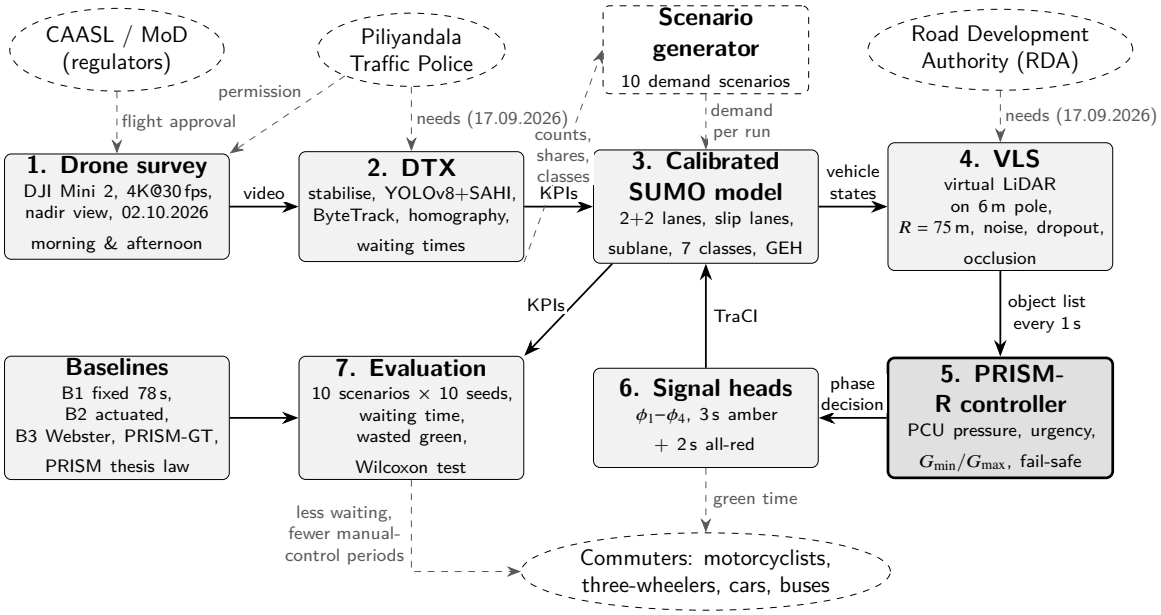


Figure 1.2: Rich picture of the proposed solution: drone survey, DTX extraction, calibrated SUMO model driven by the drone-derived scenario generator, virtual LiDAR (VLS), the PRISM-R controller acting on the signal heads in a closed loop, and evaluation against the baselines, with the stakeholders around the workflow

a microscopic model of the junction in Simulation of Urban MObility (SUMO) [21]. Inside the simulation, a Virtual LiDAR Sensor (VLS) mounted on a pole at the junction centre produces a noisy object list every second, and the PRISM controller, which sees only that list and never the simulator's ground truth, selects phases that are applied to the signal heads through the Traffic Control Interface (TraCI). The resulting indicators are compared with the three baselines and an oracle-sensor variant. Around the workflow, the police and the RDA supplied the needs, the regulators authorised the flights, and commuters benefit from reduced waiting.

The development order is also shown. Because the drone flights could only take place late in the project, all components were first developed and verified against balanced demand, split evenly across the four approaches at 300, 500 and 700 veh/h, which allowed the network, sensor and controller to be built, safety-tested and tuned before the footage was processed. The drone data then calibrated the model and parameterised the scenario generator, whose ten demand scenarios, listed in Section 5.2.2, are the demand used for every definitive experiment: the three balanced levels plus seven situations that the survey showed make the fixed plan inefficient. The controller was itself revised once, after a first evaluation on this scenario set exposed weaknesses in the control law as originally specified; the revised law is denoted PRISM-R and is developed in Section 5.2.4.

1.9 Resource requirements

1.9.1 Hardware requirements

The hardware used is listed in Table 1.2. Detector training used a cloud Graphics Processing Unit (GPU); all other work ran on a student laptop. A physical LiDAR sensor was specified as the eventual deployment target but was not purchased.

Table 1.2: Hardware requirements

Item	Specification	Purpose
Student laptop	Intel Core i7 (8 cores), 16 GB memory, 512 GB SSD	Development, DTX inference and analysis
Linux virtual machine	Ubuntu 24.04 under OrbStack on the laptop, 4 cores, 8 GB memory	Hosts SUMO and the batch experiments; the current macOS no longer runs the SUMO binaries
Google Colab GPU	NVIDIA T4, 16 GB	Fine-tuning YOLOv8 on annotated drone frames
DJI Mini 2 drone	4K video at 30 fps, hovering nadir view	Recording the morning and afternoon survey sessions (flown by a CAASL-registered operator)
Roadside LiDAR (future)	32-channel 360° unit, e.g. Ouster OS1-32 [22]	Target sensor for a real deployment; not purchased, emulated by VLS

1.9.2 Software requirements

The software stack, listed in Table 1.3, consists entirely of free and open-source or free-tier tools, which keeps the solution reproducible and affordable for a public authority.

Table 1.3: Software requirements

Software	Purpose
Python 3.11 (uv)	Main implementation language for DTX, VLS, PRISM and the experiment harness; uv pins and resolves the environment so that a run can be reproduced exactly
Ultralytics YOLOv8 [23]	Vehicle detection in drone frames (seven classes)
SAHI [24]	Sliced (tiled) inference for small objects such as motorcycles
supervision (ByteTrack) [25]	Multi-object tracking with persistent vehicle identities
OpenCV	Video input/output, feature matching, stabilisation and homography
NumPy, SciPy, Pandas	Numerical computation, smoothing, statistics and tabular data
Matplotlib	Plots of results and diagnostics
SUMO 1.20.0 with TraCI / libsumo [21, 26]	Microscopic traffic simulation and online signal control, run inside the Ubuntu 24.04 virtual machine
netconvert, netedit	Building the junction network from the OpenStreetMap extract and inspecting lanes, slip lanes and the signal program
pytest	Automated unit and integration tests
Git and GitHub	Version control and backup of code and configuration
CVAT / Roboflow	Annotation of drone frames for detector fine-tuning
Google Colab	Cloud notebook environment for GPU training
LaTeX	Typesetting of this thesis

1.10 Project scope

The scope was bounded to suit an undergraduate timeframe; Table 1.4 summarises what was and was not included.

Only the Mavithara junction was modelled; spillback from neighbouring junctions was represented only through the boundary conditions observed in the footage. No physical sensor was installed and no live signal was controlled: all control took place in SUMO, with the LiDAR represented by VLS, so that the benefit could be quantified before any money is spent on hardware. Real data came from two sessions flown on 2 October 2026, the morning peak (07:00–08:00) and the afternoon (13:30–14:30). The evening session (17:00–18:00) was approved but could not be recorded because of bad weather, so evening, weekend and wet-weather conditions remain unobserved. The existing four-phase structure was retained for all controllers so that differences in performance can be attributed to timing decisions rather than to changes in phasing or geometry, and the slip lanes were left unsignalised as they are on site. Pedestrians were not modelled; a pedestrian minimum-green parameter

Table 1.4: Project scope

In scope	Out of scope
One junction (Maviththara, Piliyandala bypass)	Coordination of a network of junctions
Simulated LiDAR (VLS) with a realistic error model	Purchase and deployment of real LiDAR hardware
Two drone sessions (morning peak, afternoon)	The approved evening session (lost to bad weather)
Four-phase control (ϕ_1–ϕ_4) with fixed intergreens	Emergency-vehicle pre-emption
Left-turn slip lanes modelled as unsignalised give-way links	Pedestrian phases and pedestrian modelling
Ten drone-derived demand scenarios covering the junction's operating conditions	Simulation of police manual control
Results visualised in SUMO-GUI	A custom web dashboard
Pure-mathematics control (PRISM)	Machine-learning or RL-based control

exists in the controller but was not exercised. Manual police control was used as motivation but not simulated, because an officer's decisions cannot be reproduced reliably. Emergency-vehicle pre-emption, network coordination and a web dashboard are deferred to future work, and results are presented through the SUMO graphical interface only. Machine learning was confined to vehicle detection.

1.11 Chapter summary

This chapter described congestion and fixed-time control in Sri Lanka, defined the specific problem at the Maviththara junction and the research gap, and presented the research question, motivation, aim, objectives, rich picture, resources and scope. The remainder of the thesis is organised as follows. Chapter 2 critically reviews existing signal control systems, control algorithms, sensing technologies and simulation tools, and justifies the research gap. Chapter 3 explains the research paradigm, the Design Science Research strategy, the data collection mechanisms, project management and ethical considerations. Chapter 4 analyses the stakeholders and specifies the requirements and system models. Chapter 5 presents the design and implementation of DTX, the SUMO model, VLS and PRISM. Chapter 6 reports the testing strategy and the evaluation of PRISM against the baselines. Finally, Chapter 7 assesses the accomplishment of the objectives, reflects on the work and recommends future directions.

2 LITERATURE REVIEW

2.1 Chapter overview

This chapter critically reviews the knowledge on which PRISM is built, spanning traffic engineering, control theory, sensing, simulation and computer vision. Each strand is read with one question in mind: what does it contribute to, or leave unsolved for, the control of a single junction carrying heterogeneous, weakly lane-disciplined traffic such as that at Mavilthara? Where a technique is judged unsuitable, the reasons are stated; where it is adopted, the decision is justified.

Section 2.2 maps the literature, Section 2.3 overviews the domain, Section 2.4 compares existing systems, Section 2.5 analyses algorithms, designs and workflows, and Section 2.6 justifies the research gap and design decisions. Foundational theory is cited from its original sources; the state of the art concentrates on work from 2021 onwards.

2.2 Conceptual map of the literature

Figure 2.1 organises the literature around Adaptive Traffic Signal Control (ATSC), with five branches: control strategies, sensing technologies, heterogeneous traffic and Passenger Car Units (PCU), simulation and calibration, and computer vision for trajectory extraction. Bold items were adopted by this research; the others were rejected or used as baselines, for the reasons given in Section 2.6.

2.3 Domain overview

A traffic signal allocates the right of way between conflicting streams in time rather than in space. A *movement* is a stream travelling from one approach to one exit; a *phase* is a set of mutually compatible movements that receive green together; and the *cycle* of length C is one complete sequence of phases. Between consecutive greens an *intergreen* (amber plus all-red) clears the junction; together with the start-up delay of queued drivers it produces *lost time* L during which the junction discharges nothing [27]. The key capacity parameter is the *saturation flow* s, the rate at which a standing queue discharges once the start-up transient has passed. For a movement with arrival flow q and effective green g, capacity is sg/C and the degree of saturation is $x = qC/(sg)$. The Highway Capacity Manual (HCM) measures performance chiefly through *control delay* [9], the basis of the metrics in Chapter 6.

All of these formulations assume a homogeneous stream of "standard" cars travelling in lanes. Traffic in Sri Lanka, as in much of South Asia, violates both assumptions. Motorcycles are the largest category of the registered fleet and three-wheelers form another large

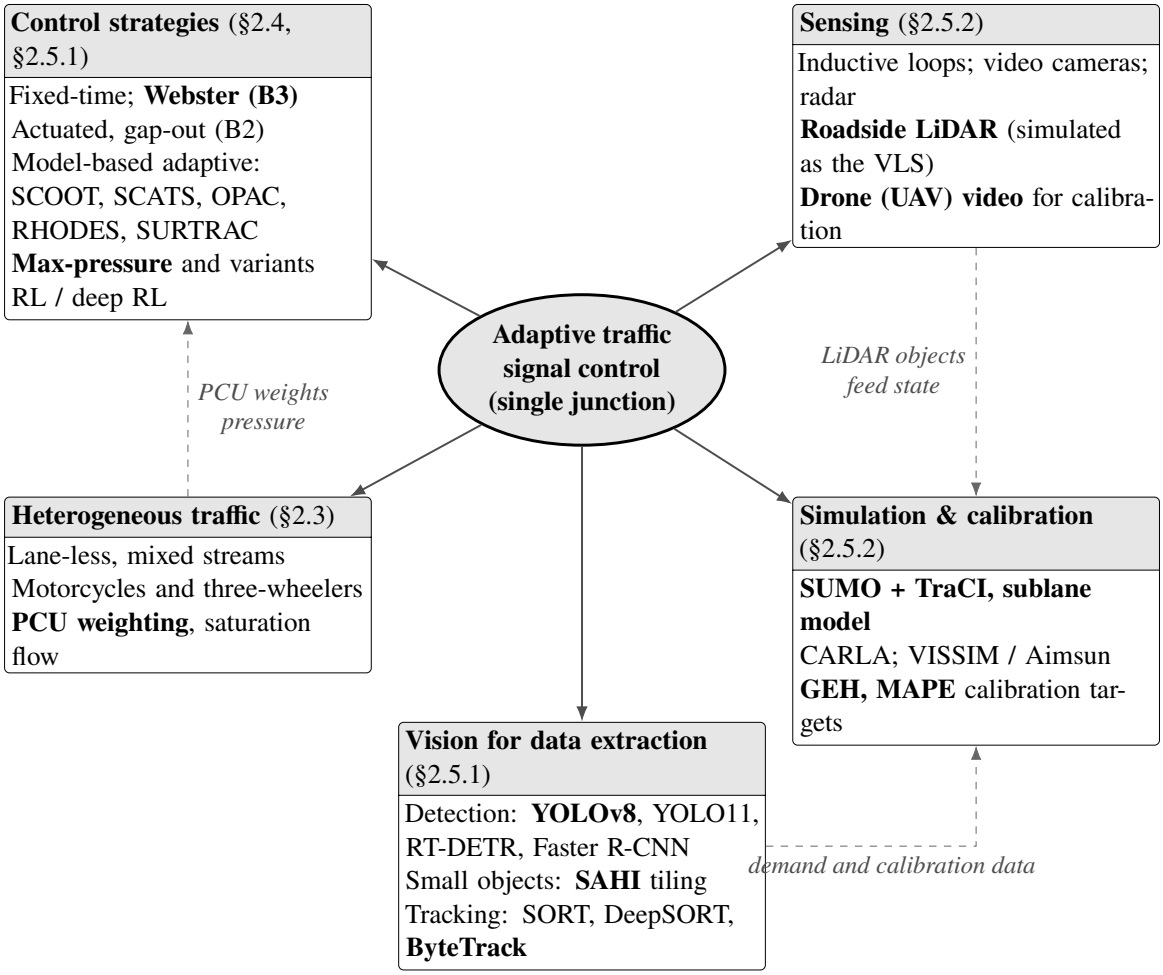


Figure 2.1: Conceptual map of the literature reviewed in this chapter. Bold items were adopted by this research; dashed arrows show how the strands inform one another in PRISM.

share [7]; at the stop line these vehicles weave between cars, travel two or three abreast within a single marked lane and accumulate ahead of larger vehicles. The conventional remedy is to express flows in PCU, weighting each vehicle by its impact on capacity relative to a passenger car [8]. Recent South Asian studies show that PCU values at signalised intersections are not constants but vary with composition, queue position and the share of two-wheelers [10, 28], and that base saturation flow in non-lane-based traffic differs markedly from Western defaults [29]. A 2026 systematic review of Sri Lankan junctions goes further, reporting saturation flows of up to 2574 PCU/h/lane and arguing that motorcycle seepage, lateral filtering and initial surge make static PCU factors and classical timing methods unreliable [30]. The author draws two conclusions from this literature. First, any controller that counts vehicles rather than road-space demand will misjudge a queue of ten motorcycles against a queue of ten buses. Second, because exact PCU values are context-dependent, a controller should treat them as tunable parameters derived from local observation rather than as universal constants, which is the approach taken in this research using the drone-derived saturation flow.

Institutionally, the Colombo metropolitan transport study documented growing congestion and called for better traffic management [1]. Suburban junctions such as Maviththara run fixed-time plans, and in severe congestion the police switch the signals off and direct traffic manually, which is labour-intensive, inconsistent and hazardous for officers. The few recent local adaptive proposals rely on camera-based detection [15] or reinforcement learning [16], both of which raise concerns examined in Section 2.4.

2.4 Existing systems, frameworks and designs

2.4.1 Fixed-time and vehicle-actuated control

Fixed-time control repeats a predetermined cycle, split and phase order, possibly with time-of-day plans. It is cheap, predictable, independent of detectors and easy to coordinate along arterials [27]. Its weakness is that the plan is optimised for an assumed demand: when arrivals deviate, green is wasted on empty approaches while queues grow elsewhere. Maviththara runs a 78 s, four-phase plan. *Vehicle-actuated* control adds local feedback from detectors: each phase runs between a minimum and a maximum green and is extended while vehicles keep arriving, "gapping out" when the headway exceeds a threshold [27, 31]. In the author's assessment, actuation helps at light and moderate demand but is myopic. It considers only the phase being served and never compares competing queues, so under heavy demand every phase runs to its maximum and the controller collapses into a fixed-time plan at maximum settings, precisely the morning-peak condition at Maviththara.

2.4.2 Model-based adaptive systems

Model-based adaptive systems adjust timings continuously from measured demand [12, 32]. **SCOOT** (Split, Cycle and Offset Optimisation Technique) builds cyclic flow profiles from upstream loops and makes frequent small adjustments to splits, offsets and cycle length to minimise a delay-and-stops index [13]. **SCATS** (Sydney Coordinated Adaptive Traffic System) uses stop-line loops to compute a degree-of-saturation measure and adapts cycle, splits and offsets hierarchically [14]. **OPAC** (Optimisation Policies for Adaptive Control) [33] and **RHODES** (Real-time Hierarchical Optimised Distributed Effective System) [34] drop the fixed cycle altogether and optimise switching times over a rolling horizon from predicted arrivals. **SURTRAC** (Scalable Urban Traffic Control) treats each junction as a real-time scheduling problem over clusters of approaching vehicles and coordinates neighbours by exchanging predicted outflows [35]. A recent review of adaptive signal control describes these systems as the established practice against which newer learning-based approaches are measured, and notes their dependence on fixed detector infrastructure [36].

The author regards these systems as the engineering benchmark but questions their transferability to a suburban Sri Lankan junction for three reasons. First, they are proprietary, require central infrastructure and specialist staff, and their cost and detector maintenance are recurrent concerns even in high-income agencies [32]. Second, they rely on inductive loops cut into the carriageway, which register lane-following vehicles well but miss motorcycles filtering along lane edges. Third, their internal traffic models assume lane-disciplined, predominantly four-wheeled flows; a loop occupancy says little about the true PCU load of a queue in which motorcycles and three-wheelers fill every lateral gap.

2.4.3 Max-pressure control and its deployments

Max-pressure control, introduced to road traffic by Varaiya [20] from the back-pressure scheduling of communication networks [37], selects at each decision instant the phase whose movements have the largest difference between upstream and downstream queues. It needs no demand forecast, is decentralised, and is provably throughput-optimal under its modelling assumptions. Since 2021 the family has matured from theory towards practice. The synthesis by Levin of methodological and experimental results [38] traces variants for cyclic phasing, realistic flow models, measurement error and multiple modes, reports that calibrated simulations mostly favour max-pressure over existing timings, and notes that field trials have begun; moreover, corridor studies with realistic phasing have shown that cyclical max-pressure can outperform actuated coordination in microsimulation of real networks [39]. Delay-based [40] and switching-loss-aware [41] variants address two practical weaknesses of the original policy, coordination-aware variants smooth platoons along corridors [42], and a two-layer scheme that combines max-pressure at critical nodes with perimeter control has been shown to remain effective in a large network with finite queues and

spillback [43]. The author finds this trajectory persuasive: max-pressure offers a compact, auditable rule whose behaviour can be explained to a traffic police officer, and its local information requirement matches what a single pole-mounted sensor can provide. However, none of the reviewed max-pressure studies considered lane-less, PCU-heterogeneous traffic or imperfect, sensor-derived queue estimates.

2.4.4 Learning-based systems

Reinforcement Learning (RL) frames signal control as a sequential decision problem, from early tabular Q-learning at an isolated junction [44] and multi-agent schemes [45] to Deep Reinforcement Learning (DRL) agents trained on high-resolution vehicle positions [46] and evaluated on real city data [47]. Recent surveys [17, 48, 49] report rapid progress in simulation, and pressure-aware state designs have become state of the art in benchmarks [50]. Yet the same surveys acknowledge that real-world deployments remain rare and that evaluation is dominated by simulated, lane-disciplined scenarios; the most recent review again names scalability, real-time adaptability and the sim-to-real gap as open problems [51]. Closer to the study site, a Sri Lankan RL controller [16] and a camera-based adaptive proposal for a Matara junction [15] report gains in simulation, and an edge-deployed YOLO-based controller weighting vehicles by passenger car equivalents has been proposed abroad [52]. The author welcomes these as evidence of local interest, but notes that none was calibrated with observed trajectories of the junction concerned, and that the camera-based designs inherit the night-time and privacy weaknesses discussed in Section 2.5.2. The author's view is that a policy whose decisions cannot be explained, which must be retrained when demand or geometry changes and which offers no formal safety guarantee, is unlikely to be accepted by the Road Development Authority (RDA) or the traffic police in the near term. Notably, the strongest RL designs now use pressure as a state feature, which suggests that the explanatory power lies largely in the pressure concept itself.

2.4.5 Comparative assessment

Table 2.1 compares the reviewed systems against criteria that matter for the study site.

The table reveals what the author regards as the crux of this review: the systems that are explainable and cheap (fixed-time, actuated, max-pressure) are blind to vehicle heterogeneity, while the systems that could in principle learn heterogeneity (RL) are neither explainable nor deployable. No reviewed system is simultaneously heterogeneity-aware, explainable and low in data demand.

Table 2.1: Comparative assessment of existing signal control systems against the requirements of the study site.

System	Sensing	Decision method	Hetero. traffic aware	Explain-able	Data needs	Main limitation for the study site
Fixed-time / Webster [11]	None	Offline cycle and split optimisation	No	Yes	Historical counts	Cannot react to demand fluctuations
Actuated [27]	Loops at stop line	Gap-out between min/max green	No	Yes	Low	Myopic; saturates to max green
SCOOT [13]	Upstream loops	Incremental split/offset/cycle optimisation	No	Partly	Dense loops, calibration	Proprietary, costly, lane-based
SCATS [14]	Stop-line loops	Degree-of-saturation plan selection	No	Partly	Dense loops	Proprietary; vehicle counts only
OPAC / RHODES [33, 34]	Upstream loops	Rolling-horizon optimisation	No	Partly	Arrival prediction	Computationally and data intensive
SURTRAC [35]	Loops, video	Real-time cluster scheduling	No	Partly	Approach-level detection	Commercial; lane-based clusters
Max-pressure [20, 38]	Queue detectors	Argmax of phase pressure	No (vehicle counts)	Yes	Local queues	Assumes perfect counts and homogeneous vehicles
Deep RL [17, 48]	Simulated full state	Learned policy	Rarely	No	Large training sets	Opaque; sim-to-real gap; no safety proof
PRISM (this work)	Roadside LiDAR (simulated)	PCU-weighted pressure with fairness	Yes	Yes	Local LiDAR objects	Evaluated in simulation only

2.5 Technological analysis

2.5.1 Algorithmic analysis

2.5.1.1 Webster's fixed-time optimisation

Webster [11] derived the average delay per vehicle on an approach,

$$d = \frac{C(1-\lambda)^2}{2(1-\lambda x)} + \frac{x^2}{2q(1-x)} - 0.65\left(\frac{C}{q^2}\right)^{1/3} x^{(2+5\lambda)}, \tag{2.1}$$

where $\lambda = g/C$. Minimising total delay yields the optimal cycle length

$$C_0 = \frac{1.5L + 5}{1 - Y}, \tag{2.2}$$

with $Y = \sum_i q_i/s_i$ the sum of critical flow ratios, and greens shared as $g_i = (C_0 - L)\, y_i/Y$ [11, 27]. Figure 2.2 shows that C_0 grows without bound as $Y \to 1$. The author draws a practical lesson from this curve: a fixed-time plan is most fragile exactly when demand is highest, so even an optimally recomputed plan cannot absorb the minute-to-minute fluctuations of a morning peak. Webster's method is nevertheless the fairest "best fixed-time" benchmark and is used as baseline B3.

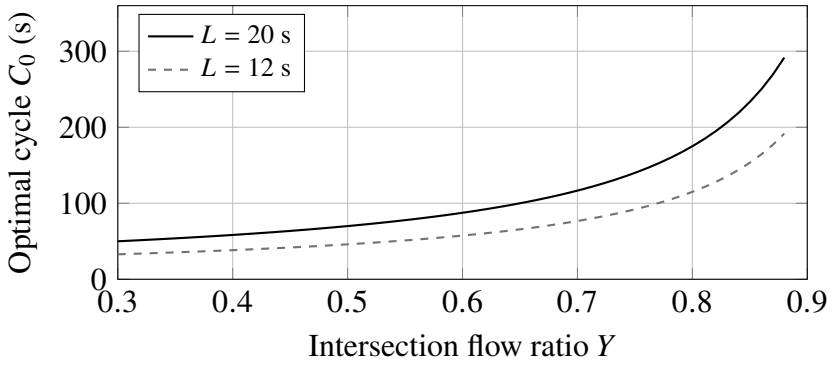


Figure 2.2: Webster's optimal cycle length (Eq. (2.2)) against the intersection flow ratio. $L = 20$ s corresponds to four phases with a 5 s intergreen each, as at the study site.

2.5.1.2 Actuated gap-out logic

Actuated control is an event-driven rule: after the minimum green, each detection resets a passage timer, and the phase terminates when the timer expires (gap-out) or the maximum green is reached (max-out) [27]. SUMO's built-in implementation extends a phase while detected headways stay below a configurable maximum gap within `minDur` and `maxDur` bounds [31], and forms baseline B2 here. The gap-out idea is valuable and is retained in PRISM as one of several termination rules; what is rejected is the use of gap-out as the *only* criterion, because it never weighs the cost of making other queues wait.

Actuated practice also answers a question that pure pressure maximisation does not: what the signal should do when there is nothing to serve. The HCM and the signal-timing handbook describe two conventional resting behaviours [9, 27]. Under *rest in green*, the controller leaves the green on a designated phase, usually the major street, once every other phase has been served, so that traffic on that phase is not stopped needlessly and the next call is answered from a phase that is already running. Under *rest in red*, the controller rests in all-red when no approach is occupied at all, so that the first vehicle to arrive on any arm receives green directly from all-red with only the clearance interval to wait, rather than finding the green committed to an empty arm. The author regards both as directly relevant to Mavithtara, where the empty-side-road and night conditions are exactly the situations in which a fixed plan wastes most of its green; they are adopted as explicit rules in Section 5.2.4 rather than left as an emergent consequence of the pressure score, because a score of zero on every phase does not by itself say *which* phase should hold the green.

2.5.1.3 Max-pressure theory and variants

Let $x_l(t)$ be the queue on link l, $r(l,m)$ the turning ratio and $c(l,m)$ the saturation flow of movement (l,m). Varaiya's movement weight and phase pressure are

$$w_{l,m}(t) = x_l(t) - \sum_{p} r(m,p)\, x_m(t), \qquad P_j(t) = \sum_{(l,m)\in j} w_{l,m}(t)\, c(l,m), \tag{2.3}$$

and the controller activates $j^* = \arg\max_j P_j(t)$ [20]. Under a store-and-forward queueing model this policy stabilises all queues whenever demand lies inside the capacity region, without knowing the demand [20, 37]. Early extensions established distributed implementations [53], simulated arterial performance [54] and normalised queues by link capacity to avoid spillback [55]. Position-weighted back-pressure further weights vehicles by their distance to the stop line so that the state reflects vehicles about to arrive [56]. Post-2021 work has targeted the gaps between theory and practice: the original proof ignores the lost time incurred at every phase change, which motivated switching-loss-aware formulations [41]; delay-based pressure replaces queue counts with accumulated delay to improve fairness at low-demand approaches [40]; and cyclical variants preserve a fixed phase order that drivers expect [38, 39].

Cyclic max-pressure.

The last of these deserves separate treatment, because it bears directly on a defect found in this research. Varaiya's policy is *acyclic*: at each decision instant it activates whichever phase has the greatest pressure, with no constraint on the order in which phases appear. Le et al. [57] showed that a decentralised pressure-like policy remains stable when the greedy choice is constrained, and Levin, Hu and Odell [58] made the cyclic case explicit: they

impose a fixed phase order, allow the controller to *skip* a phase that has no demand, and prove that the resulting policy retains the maximum stability region of acyclic max-pressure while producing the predictable phase sequence that drivers and traffic engineers expect. Their experiments further report that the cyclic variant does not lose, and often gains, against the acyclic one once lost time at phase changes is counted, because unconstrained arg-max selection can revisit the same strong phase repeatedly and pay an intergreen each time. Two consequences matter here. First, a weak movement such as a protected right turn cannot be starved merely because a competing through movement is always stronger, since the cyclic order guarantees it a turn whenever it has demand. Second, the skip rule is what prevents a cyclic order from degenerating into a fixed plan: an empty phase costs nothing. PRISM-R adopts exactly this structure, a cyclic order with skip, in place of the acyclic arg-max of the law as first specified.

From this literature the author identifies four assumptions that do not hold at Maviththarа and that PRISM was designed to relax. (i) Queues are counted in vehicles, implicitly homogeneous. (ii) Phase changes are treated as free, whereas each costs 5 s of intergreen. (iii) Queues on all links are assumed to be observed exactly, whereas a real sensor has limited range, noise, dropout and occlusion. (iv) Stability is a long-run property, so an approach with persistently low pressure can be starved in the short run. The variants above each relax one of these assumptions; none relaxes all four together, and none addresses (i) at all.

2.5.1.4 Queue estimation beyond the detector

Every pressure-based policy assumes that the queue it reads is the queue that exists. A real sensor sees only part of an approach, and the part it cannot see is precisely the part that matters when the junction is congested. Liu et al. [59] addressed this for loop detectors in the oversaturated case, where the queue extends past the advance detector and the detector therefore reports a saturated occupancy that no longer distinguishes a queue of eighty metres from one of two hundred. Their solution identifies the shock-wave structure in the high-resolution detector trace, using the breakpoints at which the arrival and departure waves cross the detector to reconstruct the maximum queue length, so that the estimate is bounded by traffic-flow theory rather than by the detector's position. The general lesson the author takes from this work is that a queue estimate must degrade *conservatively* once it reaches the edge of what the sensor can observe: reporting the visible queue as if it were the whole queue systematically understates demand exactly when demand is highest, and a controller that acts on that understatement will end a green too early. A pole-mounted LiDAR has the same problem in a different geometry. Its range is finite, so beyond that range the controller is blind, and in this research the sensor's $R = 75$ m radius leaves roughly sixty metres of each approach observable from the stop line (Section 5.2.3). Rather than reconstructing shock waves, PRISM-R takes the simpler route appropriate to an object-level sensor: while the

visible queue reaches the edge of the range, arrivals measured at the edge are accumulated into an explicit *hidden queue* that is added to the observed one, and the required green is blended towards its maximum. The principle, that the state must acknowledge what lies beyond the sensor, is Liu et al.'s.

2.5.1.5 Reinforcement learning and its limits for deployment

DRL controllers learn a mapping from a high-dimensional state to a phase choice by maximising cumulative reward [46, 47]. Surveys published since 2021 converge on a common list of obstacles to deployment: sample inefficiency and long training, sensitivity to reward design, poor transfer between junctions and demand patterns, the sim-to-real gap, and the absence of safety guarantees, which must be imposed by external constraint wrappers [17, 48, 49, 51]. Benchmarks also show that well-tuned max-pressure remains a strong baseline that many published agents only marginally beat [50, 60]. The author therefore concludes that, for a single junction whose controller must be approved and supervised by the RDA and the traffic police, the marginal gain RL might offer does not justify its opacity and data demand. Hence the PRISM control law is pure mathematics.

2.5.1.6 Vehicle detection

For the offline extraction of trajectories from drone video, object detection is required. Two-stage detectors such as Faster R-CNN [61] propose and then classify regions, giving high accuracy at high cost. Single-stage detectors descended from YOLO (You Only Look Once) [62] predict boxes in one pass; a 2023 review traces the family from YOLOv1 to YOLOv8 and documents steady gains in the accuracy–speed trade-off [63]. YOLOv8 [23] adopted an anchor-free head and a mature training and export toolchain; YOLOv9 [64] and YOLOv10 [65] improved gradient flow and removed non-maximum suppression respectively, and YOLO11 [66] refined the backbone. Transformer-based RT-DETR (Real-Time Detection Transformer) [67] also eliminates non-maximum suppression with competitive real-time accuracy. The author notes that for offline processing of a few hours of video, inference speed is secondary; what matters is accuracy on very small objects, availability of aerial pre-trained weights and the cost of fine-tuning on a free cloud Graphics Processing Unit (GPU). The newer models offer only incremental accuracy gains on generic benchmarks, whereas YOLOv8 has the widest ecosystem of VisDrone-trained weights and tiling integrations.

2.5.1.7 Small-object detection in aerial imagery

From an altitude of about 90 m a motorcycle spans roughly twenty pixels in a 4K frame, and down-sampling to the network input size erases it. The VisDrone challenge results show that small and densely packed objects remain the dominant source of error for aerial

detectors [68, 69], and dedicated small-object variants of YOLOv8 for UAV imagery report gains mainly by preserving high-resolution features [70]. Slicing Aided Hyper Inference (SAHI) takes a model-agnostic route: it runs the detector on overlapping tiles and merges the results [24]. The author prefers SAHI to an architecturally modified network because it can be applied to a standard, well-supported model and its benefit can be measured simply by switching it on and off.

2.5.1.8 Multi-object tracking

Tracking-by-detection links detections into trajectories. SORT combines a Kalman filter with Hungarian assignment on box overlap [71]; DeepSORT adds an appearance embedding [72]; ByteTrack associates low-confidence boxes in a second stage, recovering partially occluded objects [25]. Successors such as BoT-SORT [73], StrongSORT [74] and OC-SORT [75] add camera-motion compensation, stronger appearance models or observation-centric motion recovery. Performance is reported through Multiple Object Tracking Accuracy (MOTA) [76] and the Identification F1 score (IDF1) [77]. In the author's assessment, appearance features add little in a nadir view where same-class vehicles look almost identical from above, and camera motion is better removed once by frame stabilisation. ByteTrack's motion-only, two-stage association therefore matches the problem best. Table 2.2 summarises the candidates.

Table 2.2: Comparison of candidate detectors and trackers for drone-based vehicle extraction.

Method	Type	Strengths	Weaknesses for this task
Faster R-CNN [61]	Two-stage detector	High accuracy, well understood	Slow on long 4K video; heavier fine-tuning
YOLOv8 [23]	Single-stage, anchor-free	Mature toolchain, VisDrone weights, SAHI support	Small objects need tiling
YOLOv9, YOLOv10, YOLO11 [64–66]	Single-stage	Incremental accuracy and latency gains	Fewer aerial pre-trained weights at time of work
RT-DETR [67]	Transformer	No non-maximum suppression; competitive accuracy	Higher training cost on a free GPU
SORT [71]	IoU + Kalman	Very fast, simple	Drops low-score boxes; identity switches
DeepSORT, StrongSORT [72, 74]	+ appearance	Fewer switches in pedestrian scenes	Re-ID of little use for top-down vehicles
OC-SORT, BoT-SORT [73, 75]	Motion-centric / camera-motion compensation	Robust to occlusion and camera motion	Extra complexity once video is stabilised
ByteTrack [25]	Two-stage association	Keeps occluded, low-score objects; no Re-ID	Sensitive to detection thresholds

2.5.2 Design analysis

2.5.2.1 Sensing technologies

Inductive loops are accurate for lane-following vehicles but intrusive, point-based and prone to missing small vehicles off the lane centre. *Video cameras* provide wide-area, classifiable data but degrade at night, in rain and under glare, and capture identifiable faces and plates. *Radar* is robust to weather and measures speed directly but has limited resolution for closely spaced small vehicles. *Roadside LiDAR* (Light Detection and Ranging) measures three-dimensional geometry with an active laser, independently of ambient light. Early roadside pipelines demonstrated detection, classification and tracking of road users [18, 19], and recent reviews of infrastructure-based LiDAR perception describe a maturing chain of background filtering, clustering, classification and tracking, while identifying occlusion, sparse returns at range and adverse weather as the main residual error sources [78, 79]. Studies since 2021 have derived turning counts, queue lengths, arrivals on green and pedestrian waits directly from LiDAR trajectories at a signalised intersection, in real time rather than after days of aggregation [80]. Table 2.3 compares the options.

Table 2.3: Qualitative comparison of traffic sensing technologies for junction control (synthesised from [18, 27, 78, 79, 81]).

Criterion	Inductive loop	Video camera	Radar	Roadside LiDAR	Connected vehicles
Capital cost	Low per loop, high installation	Low–medium	Medium	Medium–high (falling)	Low if fleet equipped
Coverage	Point	Wide area	Wide area	360°, whole junction	Equipped vehicles only
Night / weather	Robust	Poor at night, in rain	Robust	Robust at night; moderate in heavy rain	Robust
Privacy	High	Low (identifiable)	High	High (no imagery)	Medium
Size / class	Weak	Good	Weak	Good (3-D extent)	Self-reported
Lane-less traffic	Poor	Good	Moderate	Good	Penetration-limited
Occlusion	None	High	Moderate	Moderate (tall vehicles)	None

For Mavithhara the author judges LiDAR the most suitable choice: it works by day and night, records no identifiable imagery, and its three-dimensional extent yields vehicle length, from which a size class and hence a PCU weight follow without a learned classifier. Data reported by connected vehicles [81] would avoid sensing altogether but depends on market penetration, which is negligible in Sri Lanka. LiDAR's weaknesses (cost and occlusion by buses and lorries) are acknowledged and are modelled explicitly in the VLS (Section 5.2.3) rather than assumed away.

2.5.2.2 Simulation platforms

Simulation of Urban MObility (SUMO) [21] is open source, multi-modal and controllable at every step through the Traffic Control Interface (TraCI) [26]. Its default Krauß car-following model [82] and lane-change model [83] are complemented by a *sublane model* that divides lanes into narrow strips so that motorcycles can travel abreast and filter to the stop line [84]. Field measurements of motorcycle lateral clearance while filtering [85] give empirical guidance for its lateral parameters, and SUMO has recently been calibrated for heterogeneous traffic in Khulna, Bangladesh [86] and at the Piliyandala junction itself, where a Mean Absolute Percentage Error (MAPE) below 15% and GEH below 5 were reached with locally defined vehicle types [87]. CARLA [88] offers physically rendered cameras and LiDAR but is built for autonomous-vehicle research and is far too heavy for hour-long, multi-seed experiments. Commercial VISSIM and Aimsun are powerful but licence-bound, conflicting with a reproducible, zero-cost tool chain. The author therefore chose SUMO and wrote a lightweight analytical LiDAR model over its vehicle state, rather than rendering point clouds in CARLA.

2.5.2.3 Drone datasets and data extraction

Drones observe a whole junction from above without occlusion or roadside installation. The highD [89] and pNEUMA [90] datasets demonstrated that aerial video yields accurate trajectories, the latter in dense urban traffic with many powered two-wheelers. A systematic review of UAV-based urban traffic monitoring [91] and newer datasets such as exiD [92] and CitySim [93] show that drone trajectories are now an accepted basis for calibration and digital twins. VisDrone [69] supplies labelled aerial images whose tricycle classes resemble Sri Lankan three-wheelers. The author notes that all of these datasets were recorded in Europe, North America or China; none reflects Sri Lankan composition. Even the local SUMO calibration at Piliyandala relied on roadside video aggregated to 15-minute counts and queues [87], which cannot follow individual motorcycles through the junction. Local nadir footage processed into per-vehicle trajectories was therefore required.

2.5.2.4 Calibration practice

A simulation is credible only if calibrated. The Federal Highway Administration (FHWA) toolbox [94] recommends the GEH statistic,

$$\mathrm{GEH} = \sqrt{\frac{2(M-C)^2}{M+C}}, \tag{2.4}$$

with GEH < 5 for at least 85% of counts, alongside error measures such as MAPE on travel or waiting times. Studies of heterogeneous traffic show that default car-following and lane-change parameters are inadequate and must be tuned to local behaviour [86, 87, 95], while

automated pipelines now fit SUMO demand and signal programmes to observed counts [96]. The author adopts both a count criterion (GEH) and a behavioural criterion (MAPE of waiting time), because matching counts alone does not prove that the model reproduces queues.

2.5.3 Workflow analysis

The reviewed studies share a common workflow: collect traffic data, build and calibrate a simulation, implement the controller, and evaluate it against baselines over several seeds. The author identifies four recurring shortcomings in its execution. First, controllers are commonly fed the simulator's ground truth, so reported gains omit the cost of imperfect sensing. Second, demand is often synthetic or generic rather than observed at the studied junction. Third, heterogeneity is rarely carried through every stage: even studies calibrated with mixed traffic often evaluate a controller that counts vehicles. Fourth, the baseline is frequently a single untuned fixed-time plan, which inflates improvements. Chapter 3 shows how this research closed each shortcoming: drone-observed demand, an explicit sensor model, PCU throughout, and three baselines plus an oracle-sensor variant.

2.6 Reflection

2.6.1 Research gap

Linking the findings above, the author identifies the following gap, summarised in Table 2.4.

1. Model-based adaptive systems are effective but costly, loop-dependent and modelled on lane-disciplined, largely four-wheeled traffic (Section 2.4).
2. Max-pressure is simple, decentralised and provably stabilising, and its recent variants address switching loss, delay and cyclic order [38, 40, 41, 57, 58], yet no variant weights pressure by PCU or combines it with an explicit fairness bound and realistic sensor error, and none accounts for queues extending beyond the range of the sensor that measures them [59].
3. DRL achieves strong simulated results but remains opaque, data-hungry and rarely deployed [17, 48].
4. Roadside LiDAR perception is maturing [78, 79], but its use as the *sole* input to a signal controller, with its errors modelled explicitly, is little explored.
5. Drone trajectory data are established for calibration [91, 93], but have not been used to calibrate a simulation of a Sri Lankan junction for evaluating adaptive control; the only local SUMO calibration used aggregated roadside video and evaluated no controller [87].

Table 2.4: Feature comparison of representative works and this research (✓ = addressed, ~ = partially, × = not addressed, – = not applicable).

Work	PCU weighting	LiDAR input	No-ML control	Fairness bound	Drone calib.	Sri Lankan context
SCOOT / SCATS [13, 14]	×	×	✓	~	×	×
OPAC / RHODES / SURTRAC [33–35]	×	×	✓	~	×	×
Max-pressure and variants [20,38,40]	×	×	✓	~	×	×
Deep RL [48, 50]	×	×	×	×	×	×
Roadside LiDAR [78, 79]	~	✓	–	–	×	×
Heterogeneous PCU studies [8, 28, 30]	✓	×	–	–	×	~
Drone datasets [90, 93]	~	×	–	–	✓	×
This work (PRISM)	✓	✓	✓	✓	✓	✓

2.6.2 Justification of design decisions

Each design decision of this research follows from the review.

- **LiDAR over camera or loops.** LiDAR is the only option in Table 2.3 that is simultaneously light-independent, privacy-preserving and able to measure vehicle size in lane-less traffic. Because a physical unit was unaffordable, it was simulated; to avoid the ground-truth fallacy noted in Section 2.5.3, its noise, dropout, occlusion and size confusion were modelled.
- **Max-pressure over RL.** Max-pressure is explainable, theoretically grounded and needs no training, which suits authorities that must approve and supervise the controller, whereas DRL's deployment obstacles remain unresolved. Its four unrealistic assumptions were relaxed explicitly: PCU weighting, variable green bounded by $G_{\min}$ and $G_{\max}$ with hysteresis, a perception tracker, and an urgency term with a starvation cap.
- **PCU weighting.** Vehicle counts misrepresent load where motorcycles and buses share a queue; PCU, with the saturation flow measured locally, converts LiDAR size classes into road-space demand.
- **Drone calibration.** Existing trajectory datasets do not represent Sri Lankan composition, and loops do not exist at the site; a nadir drone captures every vehicle, including filtering motorcycles, with legal approval and without identifying individuals.
- **YOLOv8, SAHI and ByteTrack.** The combination offers the most mature aerial ecosystem, a model-agnostic remedy for twenty-pixel motorcycles, and a tracker suited to look-alike vehicles seen from above.
- **SUMO with the sublane model.** It is open, scriptable through TraCI, light enough for many seeds and the only reviewed free simulator that represents motorcycles filtering

abreast.

2.6.3 Chapter summary

The review established that lane-less, heterogeneous traffic invalidates vehicle-count models; that no existing system is at once heterogeneity-aware, explainable and frugal in data; and that max-pressure, with its assumptions of homogeneous vehicles, free switching, perfect observation and implicit fairness relaxed, is the most suitable control foundation. Roadside LiDAR, SUMO with its sublane model, and YOLOv8 with SAHI and ByteTrack were selected as sensor, platform and extraction chain. Chapter 3 operationalises these decisions.

3 METHODOLOGY

This chapter explains how the research was planned and conducted. It justifies the philosophical stance, reasoning approach and research strategy against their alternatives, describes how facts were gathered, traces the execution workflow against objectives O1–O7, and closes with project management, the timeline and ethical safeguards.

3.1 Research paradigm

A research paradigm determines what counts as legitimate knowledge and how evidence is judged [97]. This research adopted a *positivist* paradigm with *pragmatist* elements. Its central claim, that a pressure-based controller reduces average waiting time at the Mavithhara junction relative to the fixed-time plan, is an empirical proposition about a physical system whose waiting time, queues and throughput can be measured objectively and repeatedly.

The research also had to *build* something useful for specific stakeholders, the Piliyandala Traffic Police and the Road Development Authority (RDA). Choices such as simulating the LiDAR sensor rather than buying one, or using mock demand before drone data arrived, were driven by what worked within the project's constraints, which is characteristically pragmatist.

An *interpretivist* paradigm, which seeks the subjective meanings people attach to a phenomenon (for example, how officers decide to take manual control), was rejected as the primary stance. Such insight informed the problem framing, but an interpretivist reading could not answer whether one control law produces less delay than another.

3.2 Research approach

The research followed a *deductive*, *quantitative* approach, moving from theory to a testable expectation and then to data. Max-pressure theory states that serving the phase with the greatest weighted difference between upstream and downstream queues stabilises any serviceable demand [20]; signal timing theory states that a demand-blind fixed plan wastes green time whenever arrivals fluctuate [11]. From these premises the working hypothesis was derived that a PCU-weighted pressure controller would lower average waiting time relative to fixed-time, actuated and Webster-optimised control, and that this advantage would shrink as the junction approached saturation. It was tested statistically as paired differences in performance indicators [98].

An *inductive* approach, building theory from observed patterns, was not adopted because strong control theory already existed; the gap in Chapter 2 was a lack of evidence on how it

performs under heterogeneous Sri Lankan traffic and imperfect LiDAR sensing. Inductive reasoning played only a supporting role, for example when motorcycle filtering observed in the drone video shaped the simulation model.

3.3 Research strategy

The research strategy combined *Design Science Research* (DSR) with a *controlled simulation experiment*. DSR creates purposeful artefacts and judges them by their utility [99]. This research produced three: the Drone Trajectory eXtractor (DTX), the Virtual LiDAR Sensor (VLS) and the Pressure-based Real-time Intersection Signal Management (PRISM) controller. The DSR process model of Peffers et al. [100] provided the backbone of the workflow in Section 3.5. Hevner's guidelines of problem relevance, rigorous evaluation and design as a search process [99] were applied by grounding the problem in stakeholder needs, evaluating against three baselines, and iterating the design on mock data before real data arrived.

Because intervening on a live junction was neither safe nor permissible, the artefacts were evaluated through controlled experiments on a calibrated microscopic model built in Simulation of Urban MObility (SUMO) [21, 94]. Simulation allowed identical demand to be replayed under every controller and sensor faults to be injected at chosen intensities; its price is that conclusions are only as valid as the model, hence the emphasis on drone-based calibration.

A *case study* could not compare controllers under identical conditions, a *survey* would have measured perceptions rather than delay, and *action research* would have required changing the real signals, beyond the remit of an undergraduate project, though it suits a future field pilot.

3.4 Fact collection mechanisms

Literature review. A structured review, weighted towards work from 2021 onwards, established the state of the art in signal control, sensing, aerial detection and simulation, and was the systematic route to the research gap (O2; Chapter 2).

Drone video survey. Real traffic was recorded from an Unmanned Aerial Vehicle (UAV) hovering above the junction on 2 October 2026, in Session M (07:00–08:00, morning peak, high congestion) and Session A (13:30–14:30, moderate demand). A third session (17:00–18:00) had been approved but could not be flown because of bad weather. A nadir drone view captures all four approaches at once with little occlusion, whereas manual counts cannot record trajectories or waiting times for hundreds of vehicles and roadside cameras give only oblique views. Drone video is an established trajectory source [89, 90]; flight details are in Appendix F.

Stakeholder discussion. On 17 September 2026 the author met representatives of the Piliyandala Traffic Police and the RDA. The discussion was semi-structured and unrecorded; no individual is named. Two needs emerged: to reduce the time officers spend directing traffic manually, and to reduce the waiting time of every road user. Emergency vehicles were reported to rely on other traffic making way, so pre-emption was noted as future work. Only practitioners could confirm the problem and define success for them (O1).

Simulation experiments. Controlled experiments in SUMO generated the evaluation data (O7). Development began with balanced demand, split evenly across the four approaches at 300, 500 and 700 veh/h, which allowed the complete sensing and control stack to be verified months before the flights; symmetric demand made errors easy to detect. The drone data then calibrated the model and parameterised a *scenario generator*, and the definitive experiments were run on the ten demand scenarios it produces: the three balanced levels, together with seven situations that the survey showed make the fixed plan inefficient (an empty side road, unbalanced roads, a peak, a platoon surge, night, a heavy-vehicle mix and a sensor failure). Composition, turning shares and saturation flow in every scenario come from the footage. Random seeds were partitioned so that no evaluation seed influenced any design choice: seeds 1–5 were used for tuning and seeds 6–15, held out, for evaluation, giving ten paired runs per scenario and controller. Differences were assessed with the exact two-sided Wilcoxon signed-rank test, paired by seed, for which $n = 10$ gives a smallest attainable p-value of 0.002.

3.5 Research methodology execution workflow

Table 3.1 summarises each DSR stage, its outputs and objectives; the subsections explain each stage.

3.5.1 Problem identification

Repeated observation showed that the junction's fixed-time plan runs four phases (bypass through 21 s, bypass protected right 8 s, cross-road through 15 s, cross-road protected right 14 s) in a 78 s cycle, each followed by 3 s amber and 2 s all-red, regardless of demand. Morning-peak queues on the Piliyandala bypass routinely forced the police to take manual control. After a permission request to the Piliyandala Police (12 June 2026), the problem was confirmed with the police and the RDA on 17 September 2026.

3.5.2 Relevance justification

Relevance was established by linking the local symptoms to wider evidence: chronic congestion in the Colombo region [1], a vehicle fleet dominated by motorcycles and three-wheelers [7], and the absence of any adaptive control at the junction.

Table 3.1: Research methodology execution workflow.

Phase	Activities	Outputs	Obj.
Problem identification	Site observation of the 78 s fixed plan; permission request (12.06.2026); police/RDA discussion (17.09.2026)	Problem statement; stakeholder needs	O1
Relevance justification	Review of congestion, mixed-traffic and manual-control context in Sri Lanka	Motivation; relevance argument	O1, O2
Comparative analysis & gap justification	Critical review of fixed-time, actuated, adaptive, max-pressure and learning controllers, sensors and simulators	Comparison of existing systems; research gap	O2
Define & finalise objectives	Translation of the gap and stakeholder needs into research question, aim, objectives and requirements	O1–O7; FR/NFR lists	O1–O7
Design, development & data handling	SUMO model on balanced demand; DTX; calibration; scenario generator; VLS; PRISM and baselines; versioned data and configurations	Working artefacts; calibrated model; ten demand scenarios; anonymous trajectory store	O3–O6
Evaluation, revision & communication	Paired-seed experiments on seeds 6–15; exact Wilcoxon tests; diagnosis of the first evaluation and revision of the control law into PRISM-R; sensitivity analysis; supervisor reviews; thesis	Results; revised control law; thesis; reproducible repository	O6, O7

3.5.3 Comparative analysis and gap justification

Existing systems and algorithms were compared on sensing needs, treatment of heterogeneous traffic, fairness and explainability (Sections 2.4 and 2.6). The comparison showed that pressure-based control is theoretically attractive but had rarely been evaluated with PCU weighting, realistic LiDAR imperfections and drone-calibrated mixed traffic, which justified the research gap.

3.5.4 Define and finalise objectives

The gap and the stakeholder needs were translated into one research question, an aim and seven objectives (Section 1.7), and then into functional and non-functional requirements (Chapter 4). The decision to keep the controller purely mathematical, without machine learning, was fixed at this stage to secure explainability.

3.5.5 Design, development, data management and handling

First, a SUMO model of the junction (two lanes in and two out on every arm, give-way slip lanes for left turns, and the four-phase signal) was built and driven by balanced demand. Second, DTX was developed on sample clips to stabilise video, detect and track the seven vehicle classes and compute per-vehicle waiting time. Third, after the flights, the model was calibrated against the DTX outputs using the GEH statistic for counts and the Mean Absolute Percentage Error (MAPE) for average waiting time [94], and the scenario generator was parameterised from the same outputs. Fourth, VLS was built to turn simulated vehicles into a noisy LiDAR object list. Fifth, PRISM and the baselines were implemented and tuned. Sixth, after a first evaluation on the ten scenarios, the control law was diagnosed and revised into PRISM-R; every revision was implemented as a parameter, so that the law as first specified can still be run as an ablation on identical sensing. The VLS and PRISM prototypes were in fact first written against balanced demand and refined after calibration.

Only anonymous trajectories (track identifier, class, time, position and speed) and aggregated flows were stored; raw video was kept offline. All scenario and parameter files were held in a Git repository, so every result is traceable to a configuration version. Random seeds were partitioned: seeds 1–5 were used only for parameter tuning, and seeds 6–15 were reserved for the final evaluation to avoid optimistic bias.

3.5.6 Evaluation and communication

Each controller was run for a 300 s warm-up followed by 3600 s of measurement on every evaluation seed and in every scenario, with all controllers sharing identical demand realisations so that comparisons were paired. Differences were tested with the exact two-sided Wilcoxon signed-rank test [98], and sensitivity to sensor noise, dropout and controller pa-

rameters was examined (Chapter 6). The first round of results was treated as evidence about the design rather than as the final answer: where the control law lost to the fixed plan, the cause was diagnosed and the law revised, and only then were the held-out seeds reported for both the original and the revised law. Results were communicated through sprint reviews with the supervisor and this thesis.

3.6 Project management methodology

The project was managed with *Scrum* [101], adapted for a solo researcher. Work ran in two-week sprints, each with one increment as its goal (for example, "VLS with noise and dropout"). The supervisor acted as product owner, reviewing each increment and updating the backlog of repository issues. The author combined the developer and Scrum Master roles.

Scrum suited the problem because uncertainties such as detector accuracy on small motorcycles and controller stability under noisy sensing could only be resolved by building and testing. A *Waterfall* model was rejected because committing to a full design up front would have risked late and costly rework once drone data arrived. *Kanban* lacks the fixed review rhythm and time-boxes needed to protect the flight-date milestone. *PRINCE2* targets organisational projects with boards and formal stage controls, an overhead disproportionate for a one-person project.

3.6.1 Project timeline

Figure 3.1 shows the timeline from June to October 2026 at weekly resolution, with the two-week sprints along the top. The drone permission chain was the critical path, so every component was verified on mock data before the flights, leaving the final weeks for extraction, calibration and evaluation.

3.6.2 Ethical considerations

Because the only human-related data were aerial recordings of public road users, the safeguards focused on lawful flight and anonymity.

Regulatory approvals. The flights were authorised through a supporting letter from the Piliyandala Police Station to the Director General of the Civil Aviation Authority of Sri Lanka (CAASL) (22 September 2026); Ministry of Defence (MoD) security clearance for educational use on 2 October 2026 between 06:00 and 18:00 (25 September 2026; reference MOD/MLO/DRONE/03/2026 (7969)); and CAASL approval, reference OP/36/2, under Sri Lanka Civil Aviation Implementing Standard (SLCAIS) 053 Rev.03 [102] (1 October 2026). The conditions, a maximum height of 400 ft Above Ground Level (AGL), visual line of sight at all times and prior notification of the Sri Lanka Air Force and the police, were

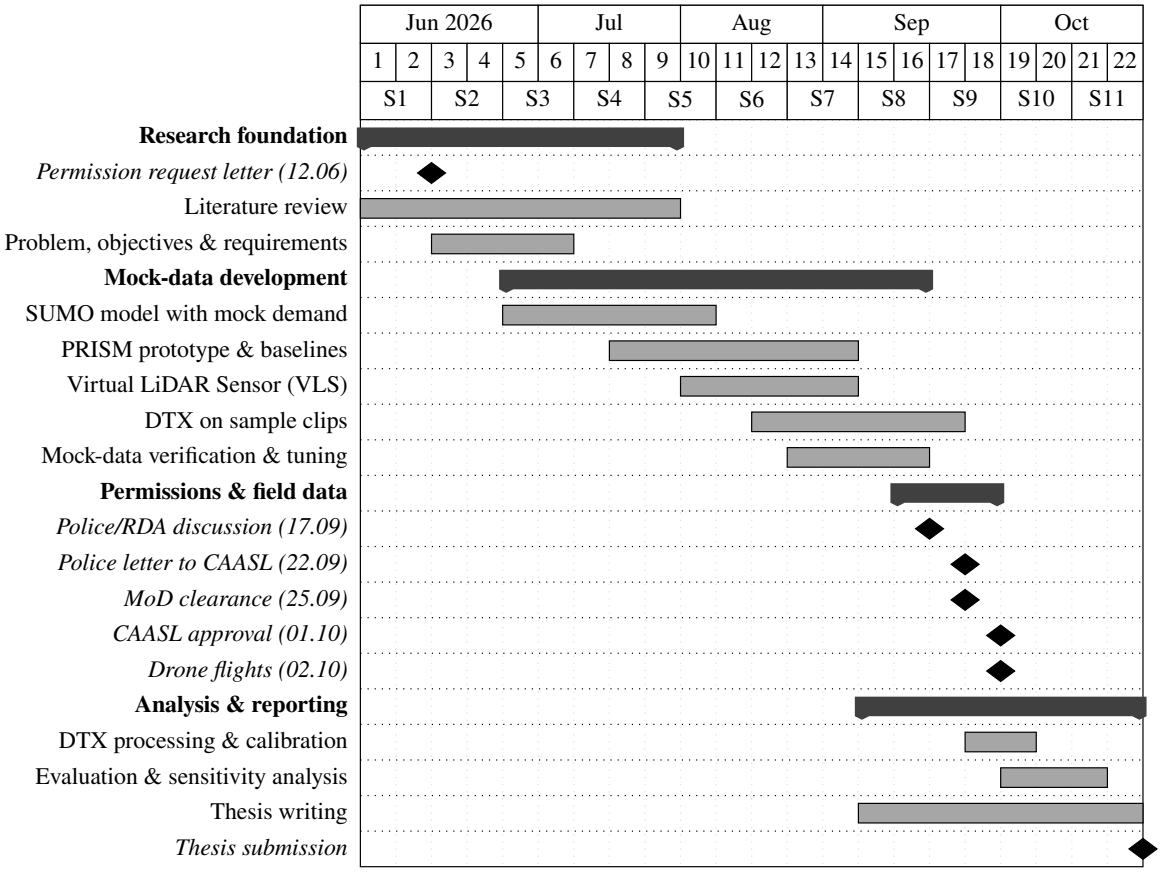


Figure 3.1: Project timeline, June–October 2026 (weeks beginning Monday 1 June; S1–S11 are two-week sprints).

observed. A CAASL-registered operator flew the aircraft. Redacted copies of the letters are in Appendix A.

Privacy and anonymity. Privacy was protected by design: from a flight altitude of about 90 m, faces and number plates occupy only a few pixels and cannot be identified. Only anonymous trajectories were kept; no personal data were processed. As a matter of policy, raw video was kept offline on an encrypted drive and is deleted once processing and examination are complete. These measures follow the data-minimisation and storage-limitation principles of Sri Lanka's Personal Data Protection Act [103]. Individual road-user consent was neither required nor practicable; the police and the RDA consented institutionally to the use of their input, with no officer named.

Professional conduct. The work followed the BCS Code of Conduct [104]. In the *public interest*, safety constraints were placed above efficiency and the limits of simulation-based evidence were stated honestly. *Professional competence and integrity* were maintained by acknowledging all sources and reporting unfavourable as well as favourable results. *Duty to relevant authority* was met by respecting aviation and defence regulations and institutional procedures.

3.7 Chapter summary

This chapter justified a positivist paradigm with pragmatist elements, a deductive quantitative approach, and a strategy combining Design Science Research with simulation experiments. Facts came from literature, a two-session drone survey, the police/RDA discussion and controlled simulation over ten drone-derived demand scenarios. The workflow linked each stage to O1–O7; the project ran as Scrum sprints with the supervisor as product owner; and ethical safeguards ensured lawful flights, anonymity and professional conduct.

4 SYSTEM REQUIREMENT SPECIFICATION

4.1 Chapter overview

Chapter 3 explained how this research was conducted; this chapter specifies *what* had to be built, bridging the stakeholder needs and objectives of Section 1.7 and the implementation of Chapter 5. It identifies the stakeholders of the Pressure-based Real-time Intersection Signal Management (PRISM) controller with an onion model (Section 4.2), and shows how the concerns of the Piliyandala Traffic Police and the Road Development Authority (RDA), together with the drone dataset fields, were operationalised into variables, objectives and requirements (Section 4.3). The system is then analysed with Unified Modelling Language (UML) use case, class, activity and sequence diagrams (Sections 4.4.1 to 4.4.4), followed by the layered architecture (Section 4.5) and the prioritised requirements (Section 4.6) that later guided the test cases of Chapter 6.

The system has no web front-end: results were inspected through SUMO-GUI, the graphical interface of the Simulation of Urban MObility (SUMO) package, and through exported tables and plots. Artificial intelligence was used only for vehicle detection; the signal controller is pure mathematics.

4.2 Stakeholder analysis

Stakeholders were identified with the onion model of Alexander [105], which arranges them in concentric layers by how directly they interact with the product. The innermost layer is the system itself: the Drone Trajectory eXtractor (DTX), the Virtual LiDAR Sensor (VLS) and PRISM. The *containing system* holds those who operate it: the traffic engineer or researcher, and the traffic police who currently control the junction and would operate a deployed controller. The *wider environment* contains those affected without operating it: commuters, motorcyclists and three-wheeler drivers, bus operators, pedestrians, the RDA as road owner, and the supervisor. An outer *regulatory* layer was added because the drone survey required the Civil Aviation Authority of Sri Lanka (CAASL) and the Ministry of Defence (MoD) [102]. Figure 4.1 shows the model and Table 4.1 each stakeholder's role, viewpoint and influence.

The analysis exposed a tension that shaped the design. The police and the RDA wanted lower waiting for *every* road user, whereas a vehicle-count controller would favour the approach with the most vehicles regardless of size. Because motorcycles and three-wheelers dominated the observed traffic while buses carry many passengers, demand was weighted in Passenger Car Units (PCU) and a starvation cap ensured that no approach was neglected.

Table 4.1: Stakeholder roles, viewpoints and influence

Stakeholder	Layer	Role	Interest / viewpoint	Influence
Traffic engineer / researcher	Containing	Builds, configures and runs DTX, SUMO, VLS and PRISM	Valid, reproducible and explainable results	High
Piliyandala Traffic Police	Containing	Controls the junction; switches lights off and directs traffic manually at peaks	Less officer time spent on manual control; safe operation	High
Commuters	Wider	End users of the junction	Shorter waiting and travel times	Medium
Motorcyclists and three-wheeler drivers	Wider	Majority of the vehicle mix; filter to the stop line	Fair treatment despite small size; not ignored by sensing	Low
Bus operators	Wider	Run scheduled services along the bypass	Reliable crossing times; high-occupancy vehicles weighted by PCU	Low
Pedestrians	Wider	Cross the arms between signal phases	Adequate and safe crossing time (future push-button phase)	Low
RDA	Wider	Owns the road and signal infrastructure	Lower congestion; a low-cost, deployable upgrade path	High
Academic supervisor	Wider	Guides and assesses the research	Academic rigour and ethical conduct	High
CAASL	Regulatory	Approves drone operations (SLCAIS 053)	Safe, lawful flights below 400 ft	High
MoD	Regulatory	Grants security clearance for flights	National security; restricted flight window	High

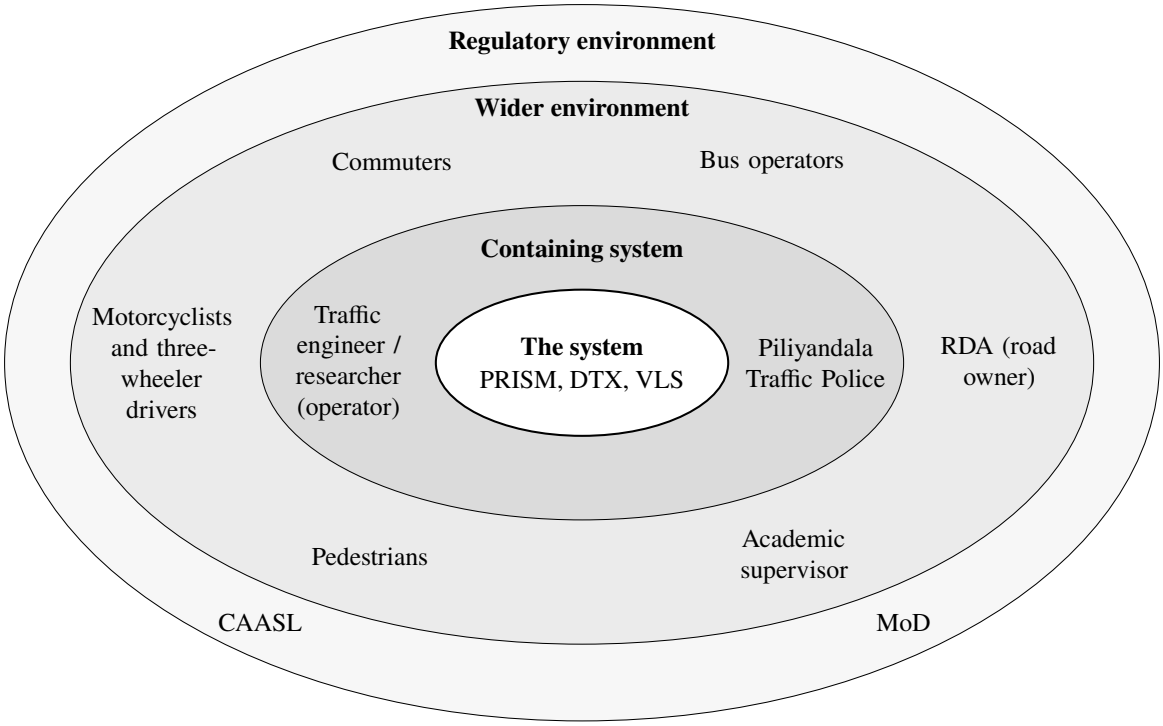


Figure 4.1: Onion model of the stakeholders of the proposed system

Pedestrian and emergency-vehicle features were placed out of scope as future work. Influence was rated by the ability to stop or redirect the project: the police, the RDA and the regulators could withhold permission, and the supervisor approved the objectives, whereas road users influenced the design only indirectly through the data recorded about them.

4.3 Operationalisation process

Operationalisation turns abstract concerns into measurable variables [106]. Two primary sources were used: the discussion with the Piliyandala Traffic Police and the RDA on 17.09.2026, which produced two explicit needs and one explicit exclusion, and the trajectory dataset from the drone sessions of 02.10.2026, from whose fields every Key Performance Indicator (KPI) was computed. The literature gap of Section 2.6 was a secondary source. Table 4.2 traces each item to a variable, to objectives O1–O7 and to the functional (FR) and non-functional (NFR) requirements of Section 4.6.

The first need, less manual control by officers, cannot be observed directly because officer behaviour was not simulated. Since officers intervene when queues grow so long that the fixed plan visibly fails, the share of cycles in which any approach queue exceeded 200 m was adopted as a "manual-control trigger". The second need was operationalised as mean waiting time per vehicle, complemented by the 95th-percentile and per-class waiting so that "every road user" was not averaged away.

Validity was justified in two ways. *Construct validity* was supported by defining waiting

Table 4.2: Operationalisation of stakeholder needs and dataset fields into variables, objectives and requirements

Source item	Measurable variable / KPI	Objective	Requirements
(a) Police / RDA discussion, 17.09.2026			
Reduce officer time spent on manual traffic control	Share of cycles in which any approach queue exceeds 200 m ("manual-control trigger"); maximum queue length	O1, O6, O7	FR10, FR12, FR14, FR16
Reduce waiting time for every road user	Mean waiting time per vehicle; 95th-percentile waiting time; waiting time per vehicle class	O1, O6, O7	FR10, FR11, FR16, FR17
Emergency vehicles currently "make way"; no priority logic requested	None in this study; recorded as future work	O1	FR19 (W)
(b) Drone trajectory dataset fields			
Timestamp	Sample interval Δt; time in frame; flow per 5-min bin	O3, O4	FR01, FR05, FR06
Vehicle class	Composition (%); PCU flow; class-specific waiting	O3, O4, O6	FR02, FR06, FR10
Position (pixel → metre)	Trajectory; queue length; distance to stop line	O3, O4	FR03, FR04
Speed	Free-flow speed; discharge headways giving saturation flow	O3, O4	FR05, FR06
Zone (approach / exit)	Turning movement; turning ratios; per-approach demand	O3, O4	FR06, FR07
Waiting flag ($v < 0.1$ m/s)	Per-vehicle waiting time W_i; observed mean waiting used as calibration target	O3, O4, O7	FR05, FR16
(c) Literature gap			
Heterogeneous traffic, imperfect sensing, explainability	PCU-weighted pressure; sensor-noise cost (PRISM vs PRISM-GT); decision log	O2, O5, O6	FR08, FR09, FR15, NFR03, NFR04

identically on both sides of the comparison: DTX counted a vehicle as waiting below 0.1 m/s inside an approach zone, the same threshold SUMO uses [21], so observed waiting was a legitimate calibration target. *Triangulation* came from drawing requirements from three independent sources: practitioners, drone measurements and literature. FR10 (pressure-based phase selection) appears in all three, whereas FR19 (emergency pre-emption) rested on one remark and was deprioritised. The dataset itself was validated by the accuracy tests of Section 6.3.2, and the calibrated model was checked against observed counts and waiting times before any controller was compared, so that the requirements rested on measured rather than assumed traffic.

4.4 System and model analysis

4.4.1 Use case diagram

Figure 4.2 presents the use case diagram. The *traffic engineer or researcher* extracts trajectories, calibrates the model, configures scenarios, runs experiments and exports KPIs. The *traffic police* observe the junction in SUMO-GUI; manual override is dashed because it is future work. Two system actors were modelled: SUMO supplies vehicle states and receives signal states through the Traffic Control Interface (TraCI) [26], and the Virtual LiDAR is the only information source for *Scan junction*. Every experiment includes the phase decision, every decision includes a scan, and the fail-safe extends the decision when the sensor stops returning objects.

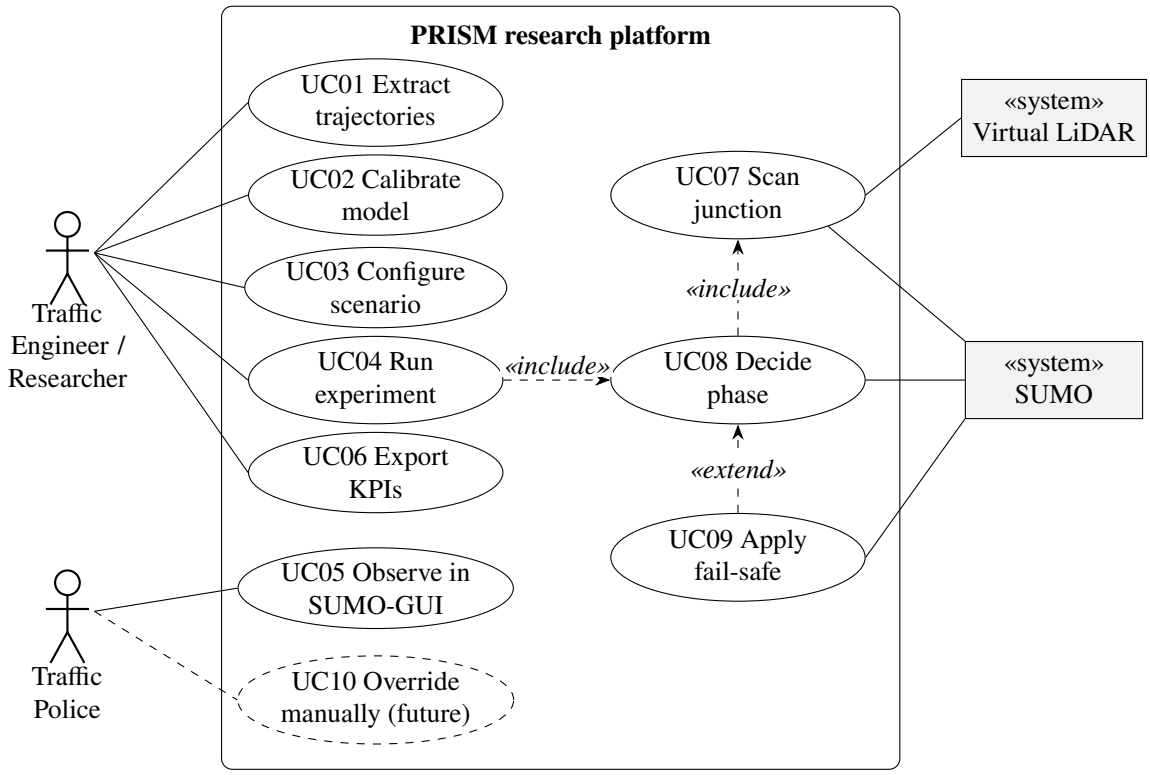


Figure 4.2: Use case diagram of the PRISM research platform

The three most important use cases are specified in Tables 4.3 to 4.5; the others are simple configuration or viewing actions.

Table 4.3: Use case specification: UC04 Run experiment

Field	Description
ID and name	UC04 – Run experiment
Actors	Traffic engineer / researcher (primary); SUMO (system)
Description	Executes every combination of scenario, controller and seed and stores the KPIs of each run.
Pre-conditions	Network, route and vehicle-type files exist; `scenarios.yaml` lists the ten demand scenarios, the controllers (B1, B2, B3, the control law as first specified, PRISM-R and PRISM-R-GT) and seeds 6–15.
Main flow	1. Researcher starts the runner with the configuration file. 2. For each scenario, controller and seed the runner starts SUMO with the seed. 3. The simulation is stepped for 300 s of warm-up and 3,600 s of measurement; UC08 is included every 1 s. 4. SUMO trip, queue and emission outputs are parsed into a KPI row. 5. After all runs, statistics and plots are produced (FR17).
Alternative flow	2a. SUMO fails to start or a run aborts: the error is logged, the run is marked failed and the runner continues with the next combination.
Post-conditions	One KPI Comma-Separated Values (CSV) file per run, a summary table and plots exist; configuration and seeds are stored with the results.

4.4.2 Class diagram

Figure 4.3 shows the static structure in three groups matching the artefacts. The extraction group is a pipeline from `Stabiliser` to `SumoExporter`, whose route files parameterise a `Scenario`. In the sensing group, `VirtualLidar` produces a noisy point list and `PerceptionTracker` aggregates confirmed `DetectedObject` instances. The control group centres on a `SignalController` interface with one `decide` operation: `PrismController` realises it and is composed of four `PhaseDefinition` objects and one `MovementState` per movement, while the abstract `BaselineController` is specialised into fixed-time, actuated and Webster controllers. Because `ExperimentRunner` depends only on the interface, every controller ran through the same code path, removing a common source of comparison bias. PRISM-GT is the same `PrismController` fed noise-free objects, and the control law as first specified is the same class with the revised behaviours switched off by parameter, so the ablation differs from PRISM-R in configuration only.

4.4.3 Activity diagram

Figure 4.4 formalises the decision loop implemented in Section 5.2.4, run every $\Delta = 1$ s. After five consecutive empty scans the controller assumes sensor failure and falls back to

Table 4.4: Use case specification: UC08 Decide phase (control step)

Field	Description
ID and name	UC08 – Decide phase (control step)
Actors	SUMO and Virtual LiDAR (system actors); initiated by UC04
Description	Every 1 s the controller decides whether to extend the current green or to terminate it, and which phase follows.
Pre-conditions	A phase is green; minimum and maximum greens, intergreen and starvation cap are configured.
Main flow	1. UC07 returns the tracked object list. 2. PRISM computes movement pressures, the hidden queue and phase scores; the required green was frozen at the start of the current green. 3. The seven rules of Section 4.4.3 are evaluated in order; the result is "extend" or "terminate". 4. On termination, 3 s amber and 2 s all-red are applied and the next phase is selected cyclically, skipping phases that have nothing to discharge. 5. The signal state string is sent to SUMO and the decision and its reason are logged.
Alternative flow	1a. Five consecutive empty scans: UC09 applies the observed 78 s fixed-time plan until objects reappear; a gap of one to four scans holds the current green. 3a. An exit link is above 85% occupancy: phases feeding it are limited to their minimum green and are not forced in by the starvation rule. 3b. No approach is occupied: the green ends and the signal rests in all-red until a vehicle is detected.
Post-conditions	No conflicting greens were issued; the decision record holds the scores and the rule that fired.

Table 4.5: Use case specification: UC01 Extract trajectories

Field	Description
ID and name	UC01 – Extract trajectories
Actors	Traffic engineer / researcher
Description	Converts a nadir drone video into metric vehicle trajectories, observed KPIs and SUMO route files.
Pre-conditions	Video file available; zone polygons, stop lines and at least four ground control points are defined; detector weights are fine-tuned.
Main flow	1. Each frame is stabilised to a reference frame. 2. Vehicles of seven classes are detected with tiled inference. 3. Detections are linked into tracks with persistent identities. 4. Pixel tracks are mapped to metres, smoothed and assigned to zones. 5. Waiting time, delay, flows, turning ratios and saturation flow are computed. 6. Route files and the observed-KPI CSV are written.
Alternative flow	1a. Too few feature matches: the frame is skipped and logged. 4a. A track is cut by the start or end of the video: it is counted in flows but excluded from averages (censored).
Post-conditions	Anonymous trajectory file, observed-KPI CSV and `.rou.xml` files exist; no images are stored.

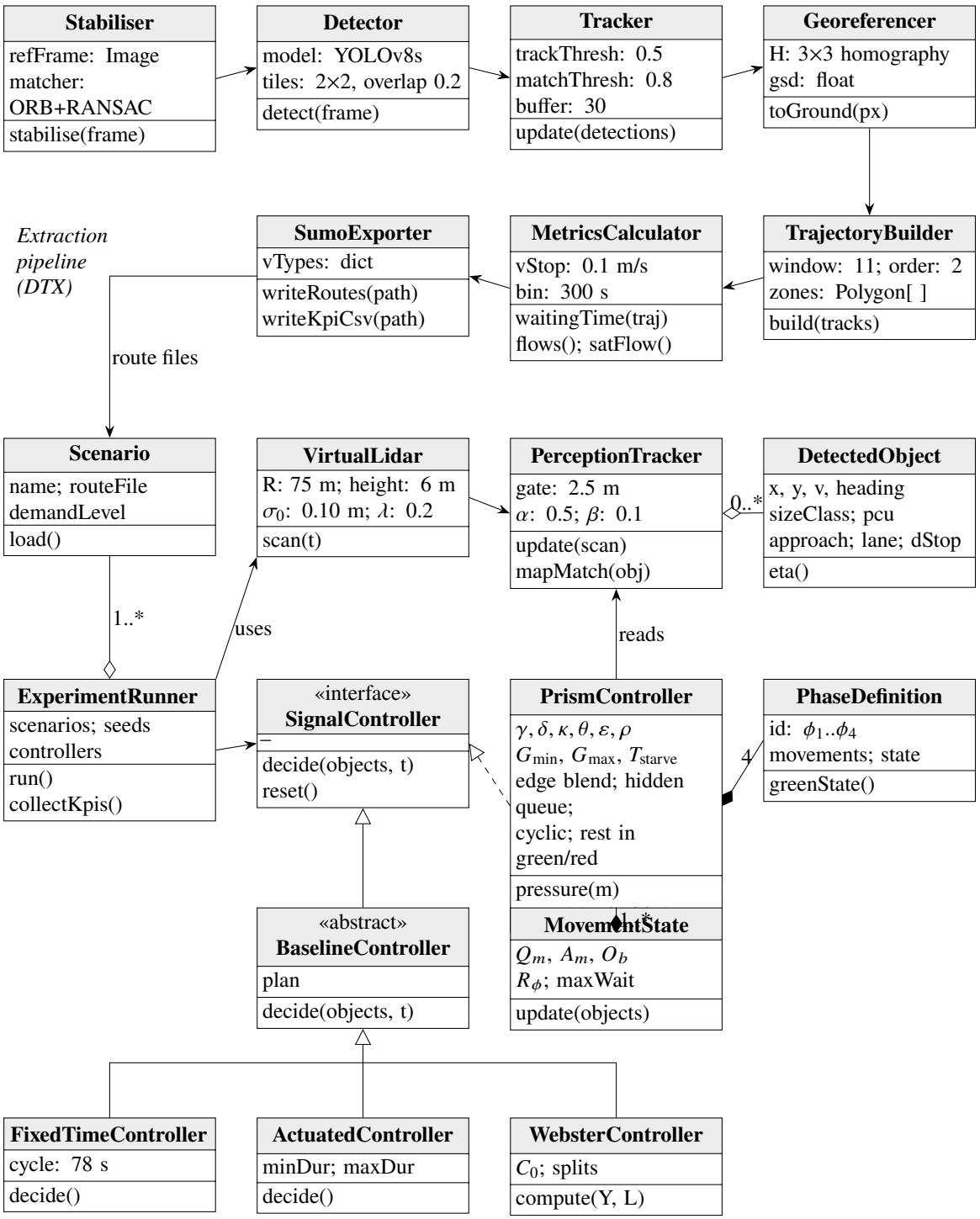


Figure 4.3: Class diagram of the extraction, sensing, control and experiment components

the observed 78 s fixed-time plan. Otherwise it updates the movement states and evaluates seven ordered rules. A green never ends before its minimum (10 s through, 5 s right turn). It ends at its maximum (50 s and 25 s) only while some other phase is actually waiting, and the spillback guard ends it after the minimum when an exit it feeds is over 85% occupied. The fairness override ends it when another phase that has waiting demand, and whose own exit is not blocked, has waited $T_{\text{starve}} = 120$ s. Two resting rules then follow: if the whole junction is empty the green ends and the signal rests in all-red, so that the next vehicle to arrive on any arm receives green directly from the clearance interval; and if no *other* phase has vehicles it could discharge, the current green is held rather than handed to an empty approach. Gap-out then ends a green serving nobody, and the pressure switch ends it when another phase that can actually discharge scores above the current one by a hysteresis margin, after a fraction ρ of the green required at the start of this green. Safety and fairness rules precede efficiency rules, so they can never be overridden. This ordering also bounds the worst-case red time of any phase by $T_{\text{starve}} + G_{\max} + 5$ s, and because every termination is attributable to exactly one rule, each decision can be logged with its reason, which satisfies the explainability requirement NFR04. Termination always inserts 3 s amber and 2 s all-red; the next phase is the oldest starved phase, otherwise the first phase after the current one in the order $\phi_1 \rightarrow \phi_2 \rightarrow \phi_3 \rightarrow \phi_4$ that has vehicles it can discharge, so that selection is cyclic with skipping rather than a free arg-max.

4.4.4 Sequence diagrams

Three sequence diagrams, one per key use case, describe the run-time interactions. Figure 4.5 corresponds to UC01: each stabilised frame passes through YOLOv8 [23] with Slicing Aided Hyper Inference (SAHI) [24], so that small motorcycles are found, and then ByteTrack [25]. Georeferencing, smoothing and metrics follow the loop because smoothing and censoring need complete trajectories.

Figure 4.6 shows one control step (UC08). The `VirtualLidar` reads ground truth from SUMO, but `PrismController` receives only the perception tracker's object list; its command returns to SUMO as a TraCI signal-state string. Keeping this boundary explicit in the design made it possible to swap the VLS for the noise-free oracle without touching the controller.

Figure 4.7 shows an experiment (UC04): ten demand scenarios (balanced 300, 500 and 700 veh/h per approach, empty side road, unbalanced roads, peak, surge, night, heavy vehicles and sensor failure) × six controllers (B1, B2, B3, the control law as first specified, PRISM-R and PRISM-R-GT) × ten evaluation seeds gave 600 runs. Statistics were computed after all runs so that paired seeds could be compared with the Wilcoxon signed-rank test [98].

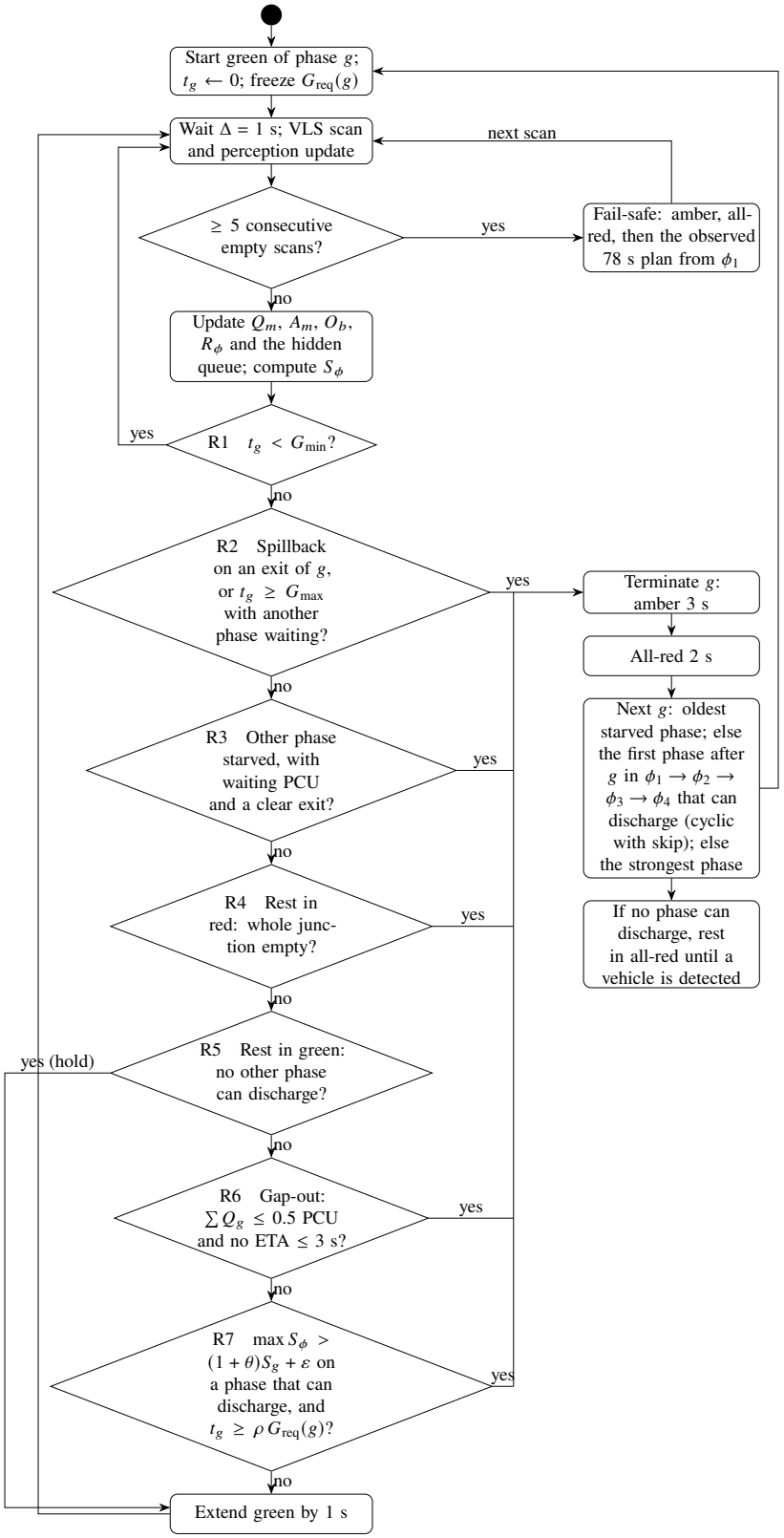


Figure 4.4: Activity diagram of the PRISM-R decision loop, executed every 1 s of simulated time; R1–R7 are the seven ordered rules and the next phase is chosen cyclically with skipping

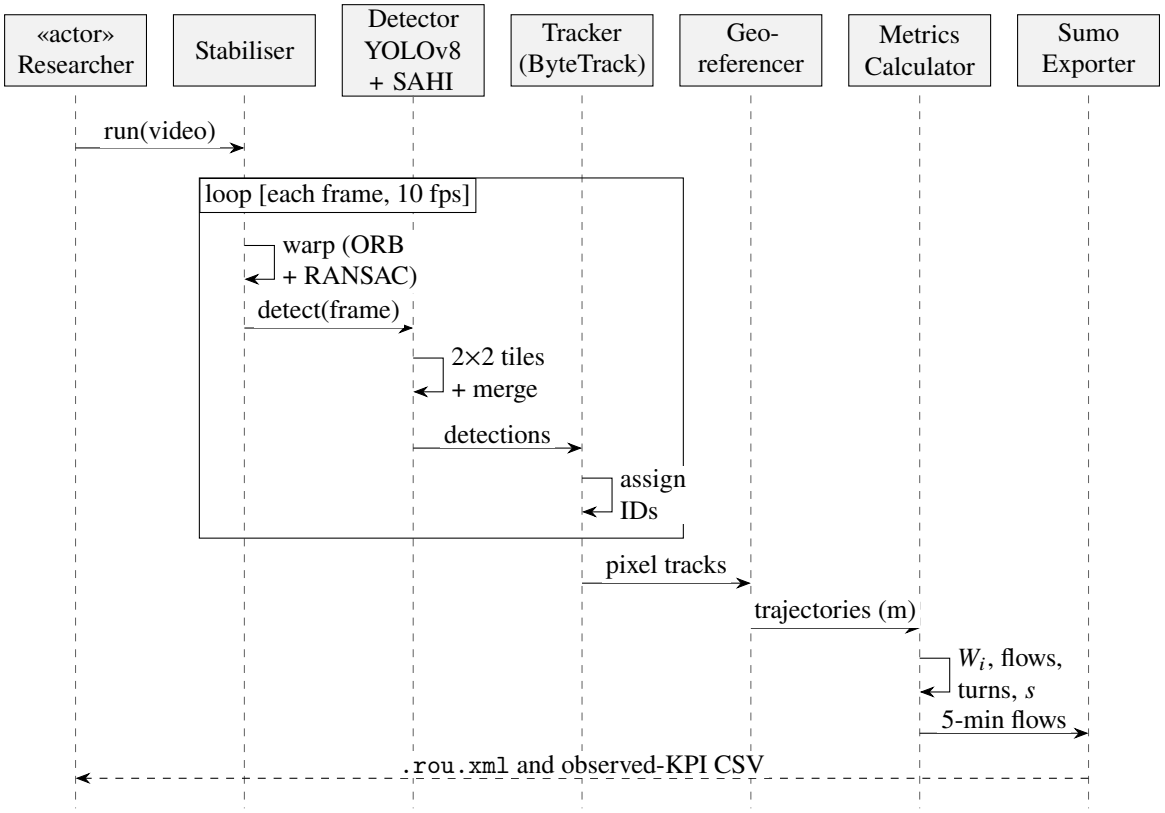


Figure 4.5: Sequence diagram of the DTX extraction pipeline (UC01)

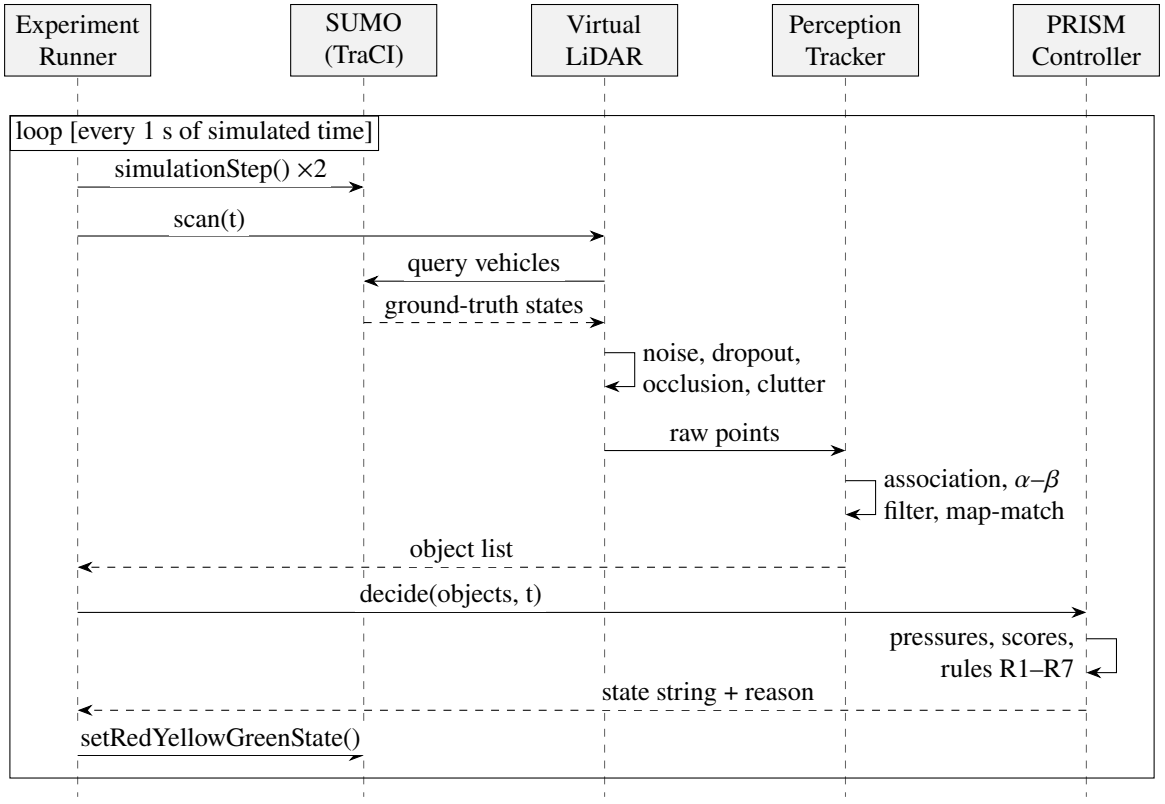


Figure 4.6: Sequence diagram of one control step (UC07 and UC08)

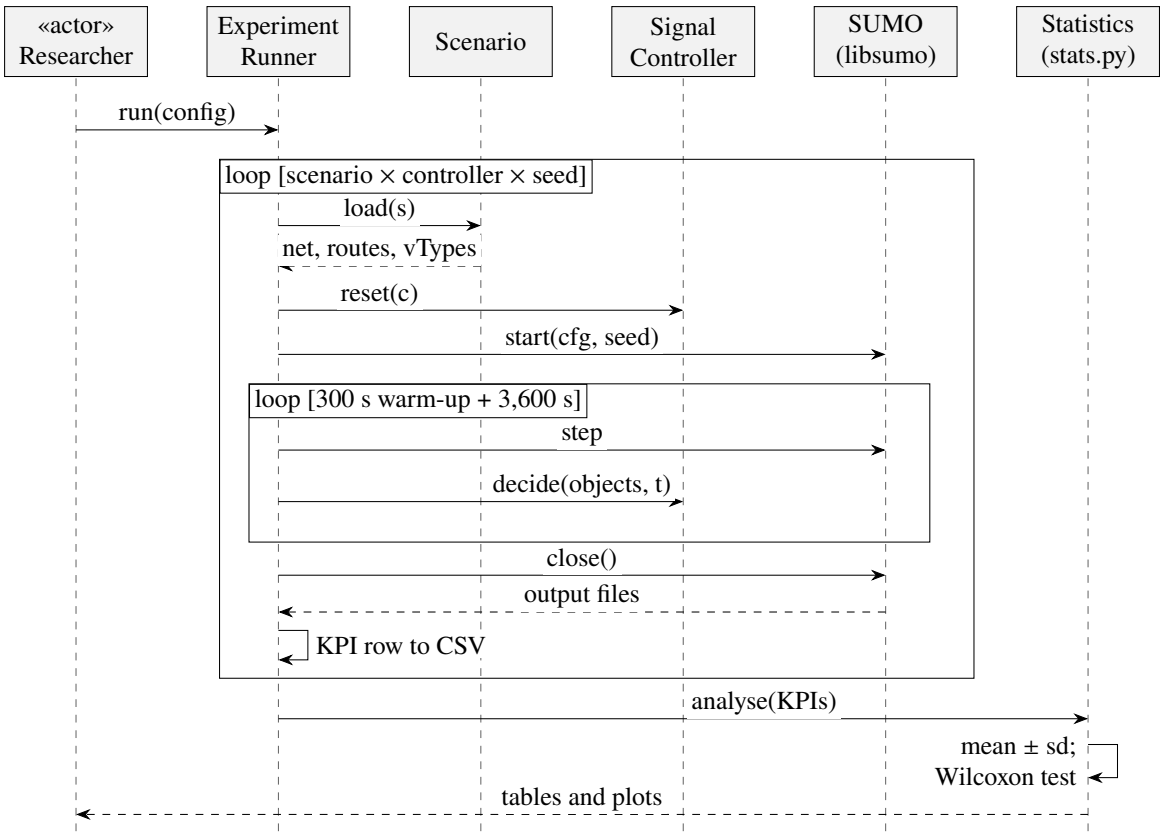


Figure 4.7: Sequence diagram of an experiment run over scenarios, controllers and seeds (UC04)

4.5 Proposed system architecture

Figure 4.8 presents six layers, each consuming the output of the one above. The *data acquisition layer* holds the two nadir DJI Mini 2 videos recorded under the permissions of Section 3.6.2. The *extraction layer* (DTX) produces anonymous trajectories, observed KPIs and route files. The *simulation layer* is the SUMO model: two lanes in and out per arm, give-way left-turn slip lanes, per-movement routes and Sri Lankan vehicle types with PCU values and sublane behaviour; it was fed first with evenly split balanced demand and later by the drone-derived scenario generator. The *sensing layer* holds the VLS and perception tracker, the *control layer* PRISM, the baselines and safety guards, and the *evaluation layer* the experiment runner, statistics and SUMO-GUI replay.

The key property is the sensing–control interface. The controller sees only the object list (position, speed, heading, size class, PCU, approach, lane, distance to stop line), never SUMO ground truth. Because the interface is sensor-agnostic, a physical pole-mounted 32-channel LiDAR [22] could replace the VLS without changing PRISM, serving the RDA's interest in a deployable upgrade path; and since PRISM-GT bypasses the error model, the cost of imperfect sensing can be measured honestly. TraCI closes the loop between control and simulation as a signal cabinet would on the street, and the layering let each layer be tested

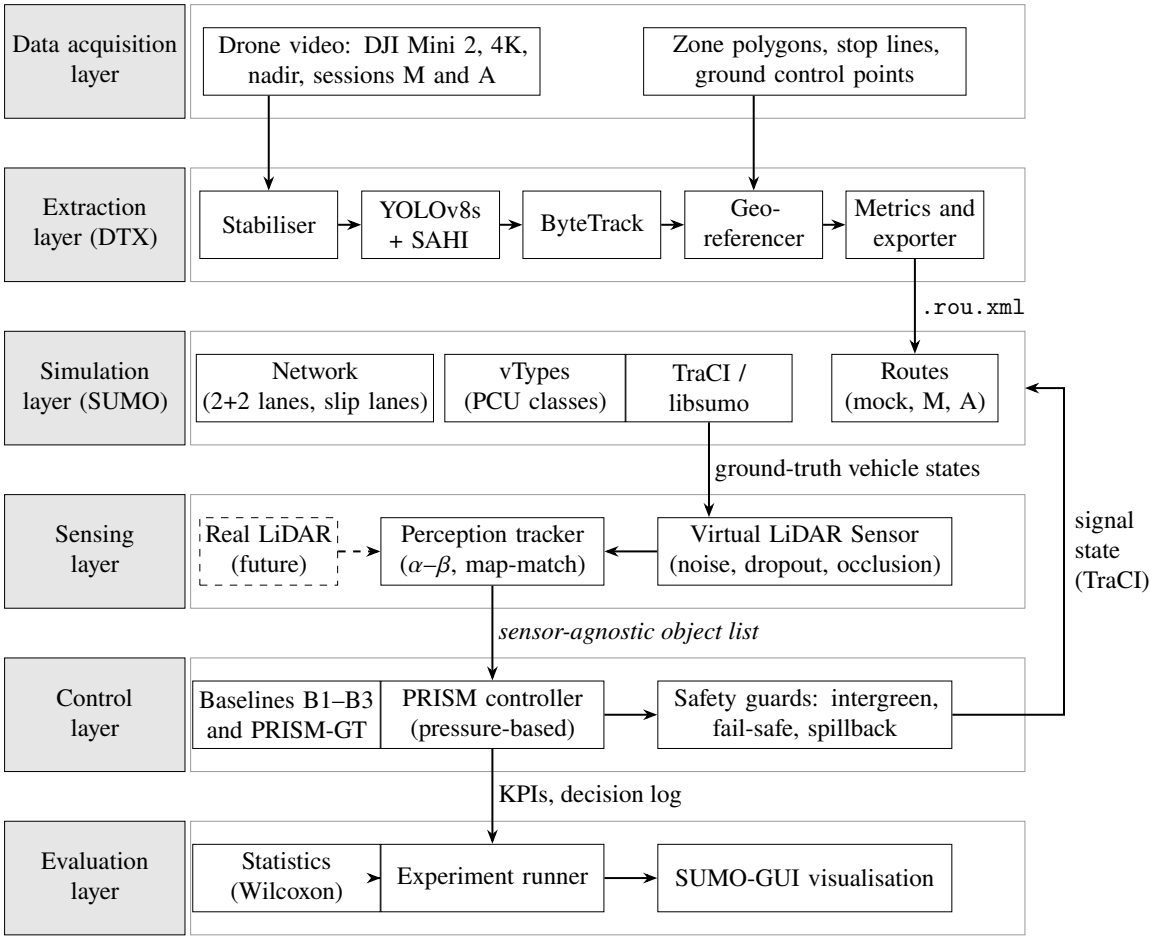


Figure 4.8: Layered architecture of the proposed system

in isolation with mock inputs before the drone data arrived.

4.6 Functional and non-functional requirements

The requirements were prioritised with MoSCoW (Must have, Should have, Could have, Won't have) [106], chosen over numerical techniques such as the analytic hierarchy process because a single developer with a fixed deadline needed a clear boundary between the minimum viable artefact and optional features. "Must" requirements are needed to answer the research question, "Should" ones add robustness and rigour, the "Could" one improves presentation, and the two "Won't" ones were consciously excluded. Tables 4.6 and 4.7 list the twenty functional and nine non-functional requirements, each traced to the objectives of Section 1.7. Each non-functional requirement was given a fit criterion so that it could be verified by a test rather than judged subjectively: the 10 ms decision budget, for example, leaves ample margin within the 1 s control interval of a real signal cabinet, and the three-minute limit per simulated hour kept the 600 runs feasible on a student laptop.

Every objective from O3 to O7 is covered by at least one "Must" requirement, confirming that the minimum artefact suffices to answer the research question; O1 and O2 are research activities whose outputs are this specification and the literature review. The "Won't" items were decided, not forgotten: the police did not request emergency pre-emption, and a web dashboard would add effort without serving the research question, since SUMO-GUI already shows the controller at work.

4.7 Chapter summary

This chapter specified the requirements of the PRISM research platform. The onion model identified stakeholders from the operating researcher to the regulators and showed why heterogeneous road users had to be treated fairly. Operationalisation mapped the police and RDA needs and the drone dataset fields to variables, objectives and requirements, justified by construct validity and triangulation. The UML analysis described the system from four viewpoints, the architecture isolated the controller behind a sensor-agnostic interface, and twenty functional and nine non-functional requirements were prioritised with MoSCoW. Chapter 5 describes their design and implementation.

Table 4.6: Functional requirements with MoSCoW priority and objective mapping

ID	Requirement	Priority	Objective
FR01	Stabilise drone video against hover drift (Oriented FAST and Rotated BRIEF (ORB) features and a Random Sample Consensus (RANSAC) homography to a reference frame).	Must	O3
FR02	Detect the seven vehicle classes (motorcycle, three-wheeler, car, van, bus, lorry, bicycle).	Must	O3
FR03	Track vehicles with persistent identities across frames.	Must	O3
FR04	Convert pixel tracks to metric ground coordinates using control points.	Must	O3
FR05	Compute per-vehicle waiting time and control delay.	Must	O3
FR06	Derive flows, turning ratios and saturation flow, and export SUMO route files and an observed-KPI CSV.	Must	O3, O4
FR07	Provide a SUMO network with 2+2 lanes per arm, give-way slip lanes and a 4-phase signal.	Must	O4
FR08	Generate a noisy LiDAR object list every 1 s (noise, dropout, occlusion, size-class confusion, clutter).	Must	O5
FR09	Track and map-match LiDAR objects to approaches and lanes.	Must	O5
FR10	Compute movement and phase pressures and select phases.	Must	O6
FR11	Enforce $G_{\min}$, $G_{\max}$, amber and all-red.	Must	O6
FR12	Enforce the starvation cap T_{starve}.	Must	O6
FR13	Fall back to the observed fixed-time plan on sensor failure.	Should	O6
FR14	Apply the spillback guard.	Should	O6
FR15	Run baselines B1–B3 and PRISM-GT.	Must	O7
FR16	Run batch experiments over seeds and export KPIs.	Must	O7
FR17	Perform statistical tests and produce plots.	Should	O7
FR18	Visualise runs in SUMO-GUI with phase and decision overlays.	Could	O7
FR19	Emergency-vehicle pre-emption (future work).	Won't	O1
FR20	Web dashboard (results are shown through SUMO-GUI only).	Won't	–

Table 4.7: Non-functional requirements with MoSCoW priority, fit criterion and objective mapping

ID	Quality	Requirement and fit criterion	Priority	Objective
NFR01	Performance	A control decision takes at most 10 ms per step.	Must	O6
NFR02	Safety	Never conflicting greens; a full intergreen (3 s amber + 2 s all-red) always precedes a new green.	Must	O6
NFR03	Robustness	Graceful degradation under sensor noise and dropout; no unsafe state when the sensor fails.	Must	O5, O7
NFR04	Explainability	Every decision is logged with its reason and phase scores.	Should	O6
NFR05	Reproducibility	Fixed seeds and versioned configuration files reproduce every result; the network is rebuilt from the OpenStreetMap extract by one script, so no manual editing step lies between the source data and the model.	Must	O4, O7
NFR06	Scalability / efficiency	One simulated hour runs in at most 3 min of wall-clock time.	Should	O7
NFR07	Privacy	No personally identifiable data (faces, number plates, images) is stored.	Must	O1, O3
NFR08	Portability	The controller consumes only a sensor-agnostic object list.	Should	O5, O6
NFR09	Maintainability	Modular code with automated tests.	Should	O3–O7

5 IMPLEMENTATION

5.1 Chapter overview

This chapter explains how the artefact specified in Chapter 4 was designed and built. It has four components: the Drone Trajectory eXtractor (DTX), which turns the drone videos into trajectories, waiting times and calibration data; a Simulation of Urban MObility (SUMO) model of the junction; the Virtual LiDAR Sensor (VLS), which imitates an imperfect pole-mounted Light Detection and Ranging (LiDAR) unit; and the Pressure-based Real-time Intersection Signal Management (PRISM) controller, the main contribution. Sections 5.2.1 to 5.2.4 present each algorithm and Section 5.2.5 the baselines. Section 5.3 justifies the technology stack and Section 5.4 presents the key code.

The SUMO network, the VLS and PRISM were first developed and tested against balanced demand split evenly across the four approaches; the drone-derived data were introduced afterwards, calibrating the model and parameterising the ten demand scenarios on which every reported experiment runs. The control law itself is purely mathematical; artificial intelligence is used only for vehicle detection inside DTX. Section 5.2.4 also records how the control law was revised into PRISM-R after its first evaluation, and why.

5.2 Design of the algorithms

5.2.1 Drone Trajectory eXtractor (DTX)

DTX converts each one-hour 4K video (30 frames per second, processed at 10) into metric trajectories and calibration quantities (Algorithm 5.1).

(i) Stabilisation.

Wind caused slow drift of the hovering drone, so every frame was registered to a reference frame with Oriented FAST and Rotated BRIEF (ORB) features [107], filtered by Random Sample Consensus (RANSAC) [108]. The estimated 3×3 matrix $\mathbf{H}_t$ warps frame t onto the reference.

(ii) Detection.

Vehicles were detected with YOLOv8s [23], a single-stage You Only Look Once (YOLO) detector, initialised from weights trained on the aerial VisDrone2019-DET dataset [69], whose *tricycle* and *awning-tricycle* classes resemble Sri Lankan three-wheelers. The model was fine-tuned on 450 annotated frames from the two sessions with the seven classes of Table 5.1. At about 90 m altitude a motorcycle occupies roughly 20 pixels, so inference used Slicing

Aided Hyper Inference (SAHI) [24] with 2×2 tiles and an overlap ratio of 0.2.

(iii) Tracking.

Detections were linked into tracks with ByteTrack [25] (track threshold 0.5, match threshold 0.8, buffer 30 frames). Its second pass over low-confidence boxes recovers partly hidden motorcycles. The class of a track was fixed by majority vote over its frames.

(iv) Pixel-to-metre conversion.

A ground-plane homography was estimated from at least four control points (stop-line ends and lane-marking corners, using the 3.5 m lane width and satellite imagery). A stabilised pixel (u, v) maps to ground coordinates (x, y) by

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \mathbf{G} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \qquad \mathbf{G} \in \mathbb{R}^{3\times 3}, \tag{5.1}$$

where λ is a scale factor [109]. The resulting ground sample distance (GSD) was $\approx$2.6 cm/px and the reprojection error on held-out points 0.18 m.

(v) Smoothing and speed.

Positions were smoothed with a Savitzky–Golay filter [110] (window of 11 samples, polynomial order 2), which removes jitter without the lag of a moving average. The speed at sample k is

$$v_k = \frac{\|\tilde{\mathbf{p}}_k - \tilde{\mathbf{p}}_{k-1}\|}{\Delta t}, \qquad \Delta t = 0.1 \text{ s}, \tag{5.2}$$

where $\tilde{\mathbf{p}}_k$ is the smoothed position.

(vi) Zoning and movements.

Polygons were drawn for each approach, each exit and each slip lane, together with stop-line segments. The entry and exit polygons crossed by a track define its turning movement (left via slip lane, through, right or U-turn).

(vii) Waiting time and delay.

As in SUMO, a vehicle is waiting in any sample where its speed is below 0.1 m/s while it is inside an approach zone $\mathcal{Z}$. The waiting time of vehicle i is

$$W_i = \sum_k \Delta t \cdot \mathbb{1}\left[v_k < 0.1 \text{ m/s} \ \wedge \ \tilde{\mathbf{p}}_k \in \mathcal{Z}\right], \tag{5.3}$$

and its control delay is

$$d_i = T_{\text{frame},i} - \frac{L_{\text{path},i}}{v_{\text{free}}}, \tag{5.4}$$

where $T_{\text{frame},i}$ is the time spent in the field of view, $L_{\text{path},i}$ the length of the smoothed path and v_{free} the observed free-flow speed of 38 km/h. Tracks that were already present when the video began, or still present when it ended, are *censored*: they were counted in flows but excluded from averages.

(viii) Derived quantities.

Flows per approach and class were aggregated into 5-minute bins in vehicles per hour and in Passenger Car Units (PCU) per hour, with turning ratios and queue lengths. Saturation flow was estimated from queue-discharge headways at the start of green, ignoring the first three vehicles [9]. Expressed per PCU,

$$s = 3600 \cdot \frac{\sum_{j \geq 4} \text{PCU}_j}{\sum_{j \geq 4} h_j}, \tag{5.5}$$

where h_j is the stop-line headway of the j-th discharging vehicle. Pooled over both sessions this gave 1850 PCU/h/lane.

(ix) Output.

Finally, DTX wrote a SUMO route file (a vehicle-type distribution per session and one `<flow>` per movement per 5-minute bin) and a comma-separated values (CSV) file of observed key performance indicators (KPIs) for calibration. In the morning the observed demand was 3,420 veh/h, or 3,150 PCU/h, and in the afternoon it was 2,280 veh/h, or 2,085 PCU/h. The observed average waiting times under the existing fixed-time plan were 104.7 s and 46.3 s respectively.

5.2.2 SUMO network and demand

The network was taken from OpenStreetMap (OSM) [111] as an Overpass extract covering the two roads and the four slip links within 500 m of 6.801 33° N, 79.935 02° E, and converted with `netconvert --lefthand` because Sri Lanka drives on the left. The geometry needed correcting in several places, but the corrections were *not* made by hand in `netedit`: each one is recorded as a `netconvert` patch file (edge type overrides, connection definitions and the traffic-light program), so a single script rebuilds the network identically from the extract. This matters for reproducibility (NFR05), because a hand-edited `.net.xml` cannot be regenerated once the source data change. All side streets in the extract were deleted; only the bypass, the cross road and the four slip links are modelled, since the side streets carry negligible flow and are not signalised. The four arms are numbered clockwise from

Algorithm 5.1: DTX drone trajectory extraction

Input: video V (30 fps), zone polygons $\mathcal{Z}$, control points, model weights
Output: trajectory table $\mathcal{T}$, route file `.rou.xml`, observed-KPI CSV

$F_0 \leftarrow$ reference frame; $\mathcal{T} \leftarrow \emptyset$
foreach *frame* F_t *of* V *sampled at 10 fps* **do**
  $\mathbf{H}_t \leftarrow \text{RANSAC}(\text{ORBMatch}(F_t, F_0))$; $\hat{F}_t \leftarrow \text{Warp}(F_t, \mathbf{H}_t)$
  $D_t \leftarrow$ SAHI(YOLOv8s, $\hat{F}_t$, tiles 2×2, overlap 0.2)
  $\mathcal{T} \leftarrow \text{ByteTrack}(\mathcal{T}, D_t)$
foreach *track* $\tau \in \mathcal{T}$ **do**
  class(τ) $\leftarrow$ majority vote of per-frame classes
  map pixels to metres with $\mathbf{G}$ (Eq. (5.1)); smooth (Savitzky–Golay 11/2)
  compute v_k (Eq. (5.2)), W_τ (Eq. (5.3)) and d_τ (Eq. (5.4))
  movement(τ) $\leftarrow$ (entry zone, exit zone)
  if τ *touches the video start or end* **then** mark τ censored
aggregate flows (veh/h, PCU/h) per approach, class and 5-min bin; turning ratios; queues
$s \leftarrow$ Eq. (5.5) over discharge headways of 4th and later vehicles
write `vTypeDistribution` and `<flow>` elements; write KPI CSV (uncensored averages)

the northern bypass arm as Bypass 1 (B1), Cross 1 (X1), Bypass 2 (B2) and Cross 2 (X2); Bypass 1 is therefore the northern bypass arm, and this numbering is used throughout the thesis and in every output file.

Each arm has two entry and two exit lanes; the kerb lane carries through traffic and the offside lane carries through and right-turning traffic. Each corner's channelised left-turn slip lane was modelled as a separate edge that merges into the exit arm through an unsignalised *yield* link. The slip lanes are therefore not part of the Traffic Light System (TLS) program and are ignored by the controller except as downstream exit occupancy. The central node is a `traffic_light` junction with the custom four-phase program of Appendix E: ϕ_1 bypass through, ϕ_2 bypass protected right, ϕ_3 cross-road through and ϕ_4 cross-road protected right, each serving both opposing directions and each followed by 3 s amber and 2 s all-red.

Heterogeneous traffic was represented with the sublane model [84] at a lateral resolution of 0.8 m. Motorcycles were given a high `lcSublane` eagerness and `minGapLat` of 0.3 m so that they filter between queued cars to the stop line, which is the dominant behaviour at Maviththarа. Longitudinal motion used the Krauss model [82] with driver imperfection $\sigma = 0.5$ and reaction time $\tau = 1.0$ s, and lateral decisions used the model of Erdmann [83]. A custom `threewheeler` type was created because SUMO has no such class. The vehicle types are listed in Table 5.1; the PCU values are adopted values for urban mixed traffic [8].

The *height* column was set explicitly rather than left at SUMO's defaults, because the occlusion rule of the VLS (Section 5.2.3) treats any vehicle at least 2.5 m tall as an occluder, on

the reasoning that only buses and lorries block the sensor's line of sight. SUMO's defaults contradict that rule: a `delivery` van defaults to 2.86 m and a `truck` to 2.4 m, so with the defaults every van would occlude and no lorry would. The values in Table 5.1 are typical kerb heights for the Sri Lankan fleet and place exactly the buses and lorries above the threshold.

Table 5.1: SUMO vehicle types used for Sri Lankan mixed traffic

Class	vClass	Len. (m)	Ht. (m)	PCU	Max (km/h)	Accel. (m/s^2)	Share M/A (%)
Motorcycle	`motorcycle`	2.0	1.5	0.5	70	2.8	38 / 34
Three-wheeler	`passenger`*	2.7	1.7	0.8	50	1.8	24 / 27
Car	`passenger`	4.5	1.5	1.0	70	2.6	21 / 22
Van/SUV	`delivery`	5.0	2.0	1.0	70	2.6	7 / 8
Bus	`bus`	11.5	3.4	3.0	60	1.2	6 / 5
Lorry	`truck`	8.5	3.5	2.5	55	1.3	3 / 3
Bicycle	`bicycle`	1.8	1.7	0.3	20	1.2	1 / 1

*Custom type `threewheeler` (width 1.4 m). M/A = morning/afternoon.

Demand for every experiment is written by a *scenario generator* parameterised from the DTX outputs. Two scripts make up the generator. `sumo/gen_mock_routes.py` writes the three balanced scenarios, with equal demand on every approach at 300, 500 and 700 veh/h; these are the scenarios on which the components were developed, unit-tested and tuned. `sumo/gen_scenario_routes.py` writes the seven situations that the drone survey showed make the fixed plan inefficient, listed in Table 5.2. Every scenario shares the turning shares of 25% left, 50% through and 25% right and the composition of 40% motorcycles, 20% three-wheelers, 25% cars, 7% vans, 5% buses and 3% lorries, which averages 0.905 PCU per vehicle, and in every scenario vehicles are released as a Poisson process at the stated rate, so that the ten seeds of a scenario differ in arrival realisation but not in expected demand. Shares and composition come from the footage; the ten scenarios are therefore demand conditions observed at or derived from the junction, and the generator exists so that each condition can be replayed identically under every controller.

Table 5.2: The seven situation scenarios written by `gen_scenario_routes.py`, in addition to the three balanced scenarios at 300, 500 and 700 veh/h per approach

ID	Name	Demand
sc1	Empty side road	Bypass 650 veh/h; X1 40 veh/h; X2 0 veh/h
sc2	Unbalanced roads	Bypass 700 veh/h; cross road 250 veh/h
sc3	Peak	250 → 750 → 250 veh/h in 10-minute blocks
sc4	Platoon surge	400 veh/h throughout, plus 1,500 veh/h on X1 for 5 minutes
sc5	Night	80 veh/h on every approach
sc6	Heavy vehicles	450 veh/h with 30% buses and lorries
sc7	Sensor failure	500 veh/h with LiDAR outages at 1,200–1,500 s and 2,400–2,460 s

Simulations used a step length of 0.5 s, a controller decision every 1 s, a 300 s warm-up that was discarded and 3,600 s of measurement, with tuning seeds 1–5 and evaluation seeds 6–

15. Trip information, summary, queue and emission outputs (HBEFA4 [112]) were recorded. The calibration of `sigma`, `tau`, `minGap`, `lcSublane` and the speed factor distribution against the observed counts and waiting times is reported in Section 6.3.3.

(i) Software environment.

SUMO 1.20.0 is run inside an Ubuntu 24.04 virtual machine hosted by OrbStack on the author's Mac, rather than natively, because the published macOS binaries no longer run on the current version of macOS. The virtual machine also fixes the SUMO build for the whole experiment campaign, which removes a source of irreproducibility between runs made weeks apart.

5.2.3 Virtual LiDAR Sensor (VLS)

The VLS reproduces the output of a 360° unit, comparable to a 32-channel Ouster OS1-32 [22], mounted on a 6 m pole at the junction centre. Rather than simulating individual laser returns, VLS works at the *object-list* level produced by roadside LiDAR perception software [18, 19]. Every second it queries all vehicles through the Traffic Control Interface (TraCI) [26], keeps those within $R = 75$ m and applies the error model below (Algorithm 5.2).

Each vehicle is represented by the centre of its bounding rectangle. This conversion is necessary because TraCI reports a vehicle's position at the *front bumper*, whereas a LiDAR perception stack reports the centroid of the cluster it detects; using the bumper position unchanged would displace every object forward by half its length, which is 5.75 m for a bus and would corrupt both the range and the distance to the stop line. The rectangle is also what gives the object its angular extent: the azimuth span $[\alpha_i^-, \alpha_i^+]$ used in Eq. (5.8) is the smallest interval containing the bearings of the rectangle's four corners, so a bus seen broadside occludes a wide arc and the same bus seen end-on occludes a narrow one.

Position noise grows with range r:

$$\hat{\mathbf{p}} = \mathbf{p} + \boldsymbol{\eta}, \qquad \boldsymbol{\eta} \sim \mathcal{N}\left(\mathbf{0}, \sigma(r)^2 \mathbf{I}\right), \qquad \sigma(r) = 0.10 + 0.002\, r \text{ m}. \tag{5.6}$$

Each object is independently missed with probability

$$p_{\text{miss}}(r) = 0.01 + 0.04 \left(\frac{r}{R}\right)^2, \tag{5.7}$$

which reflects the thinning of beams with distance. Occlusion is computed by ray casting in the horizontal plane. Let the interval $[\alpha_i^-, \alpha_i^+]$ be the azimuth range that object i subtends at the sensor. Object i is hidden if a nearer object j that is at least 2.5 m tall (a bus or lorry)

satisfies

$$\frac{\left|[\alpha_i^-, \alpha_i^+] \cap [\alpha_j^-, \alpha_j^+]\right|}{\alpha_i^+ - \alpha_i^-} \;\geq\; 0.7. \tag{5.8}$$

Clutter is added as $N_c \sim \text{Poisson}(\lambda = 0.2)$ false objects per scan at uniformly random positions within range.

A real LiDAR does not know vehicle classes, so VLS reports only a *size class* derived from the measured length: *small* below 3.2 m (motorcycles, bicycles and three-wheelers) mapped to 0.6 PCU, *medium* from 3.2 to 6 m (cars and vans) mapped to 1.0 PCU, and *large* above 6 m (buses and lorries) mapped to 2.8 PCU. The small and large values are demand-weighted compromises between the classes they contain. The controller never sees the true class.

Raw detections then pass through a perception tracker that associates each detection with the nearest predicted track within a 2.5 m gate and filters it with an α–β filter:

$$\hat{\mathbf{p}}_k^- = \hat{\mathbf{p}}_{k-1} + \hat{\mathbf{v}}_{k-1}\,\Delta t, \tag{5.9}$$

$$\hat{\mathbf{p}}_k = \hat{\mathbf{p}}_k^- + \alpha\,(\mathbf{z}_k - \hat{\mathbf{p}}_k^-), \tag{5.10}$$

$$\hat{\mathbf{v}}_k = \hat{\mathbf{v}}_{k-1} + \frac{\beta}{\Delta t}\,(\mathbf{z}_k - \hat{\mathbf{p}}_k^-), \tag{5.11}$$

with $\alpha = 0.5$, $\beta = 0.1$ and $\Delta t = 1$ s.

(i) Track initiation.

A new detection starts a *tentative* track with unknown velocity. Its second detection is associated using a wider *initiation gate* of $v_{\max}\Delta t \approx 20$ m rather than the steady-state 2.5 m gate, and the two positions set the initial velocity directly, $\hat{\mathbf{v}} = (\mathbf{z}_2 - \mathbf{z}_1)/\Delta t$. Only from the third scan onwards does the track use the 2.5 m gate and the α–β filter of Eqs. (5.9) to (5.11). The wider gate is not a detail. A tentative track initialised with $\hat{\mathbf{v}} = \mathbf{0}$ predicts that the vehicle has not moved, so with a 2.5 m gate and a 1 s scan period no vehicle travelling faster than 2.5 m/s, or 9 km/h, can ever be associated with its own second detection and confirmed. Measured on the balanced 500 veh/h scenario, the original single-gate scheme confirmed only 21% of moving vehicles; two-point initiation removes the speed ceiling, because the gate now spans the furthest a vehicle could legally travel between scans. A track is deleted after three consecutive misses, which suppresses most single-scan clutter.

(ii) Coasting tracks.

Only tracks that were updated by a detection in the current scan are reported to the controller. A track that was not associated this scan is still propagated internally by Eq. (5.9), so that it survives a short dropout and can be re-associated, but it is withheld from the object list. Reporting coasting tracks was found to be actively harmful: they made up 30% of the object list, and 84% of those were more than 2.5 m from any real vehicle, so the controller was

counting queue that had already discharged. Withholding them costs nothing, because a vehicle missed in one scan is almost always detected in the next and rejoins the list with its position corrected.

(iii) Map matching.

Reported tracks are matched to the network by point-in-polygon tests against lane polygons. A track inside an approach-lane polygon within 75 m of the stop line yields an approach, a lane and a distance to the stop line measured along the road. A track inside an exit-lane polygon yields the exit link it occupies, which feeds the downstream occupancy term. Tracks in the junction box itself, on the unsignalised slip lanes, or further than 75 m along an arm match nothing and are discarded, since the controller has no state in which to place them.

One feature of the OSM geometry had to be handled explicitly. OSM splits every arm at an intermediate node 8 to 17 m short of the stop line, where the carriageway widens for the slip lane. SUMO inserts an internal lane across that node, and an internal lane belongs to no edge, so a vehicle standing on it matched no approach polygon at all. The lane polygons therefore include a *connector* spanning the internal lane, and distance along the road is measured through the connector as well as the two edges it joins. Without this, the controller saw only 27–31% of the stopped vehicles on the B1 and X1 approaches, precisely the vehicles nearest the stop line and so precisely those whose queue matters most.

(iv) Effective queue range.

The 75 m radius is centred on the sensor at the junction centre, not on the stop lines, and the stop lines lie 13 to 16 m from the centre. The sensor therefore observes queues to roughly **60 m** back from the stop line, not 75 m. Two statements elsewhere in the original specification assumed the larger figure and are corrected accordingly: the queue term Q_m of Section 5.2.4 counts objects within the *observable* range rather than "within 75 m of the stop line", and the exit capacity $C_b = 2 \times 75/6.5 \approx 23$ PCU is similarly optimistic, since the exit links are observable over the same shorter distance. This is the limitation that motivates the hidden-queue state of Section 5.2.4.

(v) Randomness.

The VLS draws its noise, dropout and clutter from its own random generator, seeded with the run seed and separate from SUMO's, so that sensor realisations are reproducible and independent of how many random numbers the traffic model happens to consume. For the dropout sweep of Section 6.3.5 the range-dependent $p_{\text{miss}}(r)$ of Eq. (5.7) is replaced by a uniform miss probability applied to every object regardless of range, so that the swept quantity is a single interpretable number.

This list is the only input to PRISM, so a real sensor could replace it unchanged.

Algorithm 5.2: VLS scan and perception

Input: ground-truth vehicles $\mathcal{V}$ from TraCI, perception tracks $\mathcal{K}$
Output: object list $\mathcal{O}$ for the controller

$\mathbf{p}_i \leftarrow$ rectangle centre of i (TraCI reports the front bumper)
$\mathcal{D} \leftarrow \{i \in \mathcal{V} : r_i \le R\}$
foreach $i \in \mathcal{D}$ *in increasing range* **do**
    $[\alpha_i^-, \alpha_i^+] \leftarrow$ azimuth span of the four corners of i's rectangle
    if $\exists j : h_j \ge 2.5\ m,\ r_j < r_i,\ overlap \ge 0.7$ *(Eq.* (5.8)*)* **then** remove i; **continue**
    if $U(0,1) < p_{miss}(r_i)$ **then** remove i; **continue**
    $\hat{\mathbf{p}}_i \leftarrow \mathbf{p}_i + \mathcal{N}(\mathbf{0}, \sigma(r_i)^2\mathbf{I})$; size class and PCU from length
add $N_c \sim \text{Poisson}(0.2)$ clutter objects uniformly within R
foreach *tentative track with one hit* **do**
    associate within the initiation gate $v_{\max}\Delta t \approx 20$ m; set $\hat{\mathbf{v}} \leftarrow (\mathbf{z}_2 - \mathbf{z}_1)/\Delta t$
associate remaining detections to $\mathcal{K}$ by nearest neighbour (gate 2.5 m); update with Eqs. (5.9) to (5.11)
coast unassociated tracks by Eq. (5.9) but do *not* report them; delete after 3 misses
$\mathcal{O} \leftarrow$ tracks updated this scan, map-matched to approach, lane and d_{stop} through the intermediate-node connectors; junction box, slip lanes and anything beyond 75 m match nothing

5.2.4 PRISM control algorithm (PRISM-R)

PRISM belongs to the max-pressure family [20, 37] but was adapted to three realities of the Mavithara junction: vehicles differ greatly in size, a starved approach is socially unacceptable even if it is theoretically optimal, and the sensor is imperfect. It contains no learned component.

The law as first specified did not work. On its first evaluation it lost to the existing fixed-time plan at both 500 and 700 veh/h, and the loss at 700 was severe. Rather than retune the parameters, the author instrumented the decision log and diagnosed five distinct causes, each of which is a modelling error rather than a bad constant:

1. **The map-matching gap.** Vehicles standing on the internal lane at the intermediate OSM node, nearest the stop line, matched no approach, so the controller saw only 27–31% of the stopped vehicles on B1 and X1 (Section 5.2.3). It was optimising against a queue that was mostly invisible.

2. G_{req} **recomputed every second.** Equation (5.17) was evaluated at each decision step from the queue *currently* present. As the green discharged the queue, G_{req} shrank with it, so the condition $t_g \ge \rho\, G_{\text{req}}$ became satisfiable almost immediately. Greens ended after about 11 s and the controller made roughly 250 phase changes an hour, each paying 5 s of intergreen: about 21 minutes of every hour was spent clearing the junction.

3. **The 60 m horizon.** Because the sensor sees roughly 60 m back from the stop line, a queue

that is longer than that reports a constant value, and the controller cannot distinguish a queue of 60 m from one of 250 m. At 700 veh/h this is the normal condition, so the pressure signal saturated exactly when it was needed.

4. **Right-turn starvation.** Under acyclic arg-max selection a protected right turn, which serves at most a quarter of the flow of the through phase beside it, is almost never the strongest phase. It was served only when the starvation override fired at 120 s, and the resulting right-turn queues then blocked the offside lane and degraded the through movement as well.

5. **Occlusion in heavy traffic.** A stopped vehicle hidden behind a bus simply vanished from the object list, so its PCU left the queue estimate although the vehicle had not moved.

The revisions described in the remainder of this section address these five causes. Every revision is implemented as a parameter rather than as a rewrite, so the original formulation can still be executed on identical sensing as an ablation, referred to throughout Chapter 6 as "PRISM, thesis law". The revised controller is denoted PRISM-R, and wherever the two differ the text says which is meant.

5.2.4.1 State

A *movement* m is a pair (approach, through or right). Left turns use the slip lanes and are not controlled. With the lane assumption stated in Section 5.2.2, the controller computes for each movement:

- Q_m, the queued PCU: the sum of PCU of objects in m's lanes with $v < 1.4$ m/s within the observable range of the approach, which is about 60 m from the stop line (Section 5.2.3). Offside-lane objects are split between through and right, because the sensor cannot know a stationary vehicle's intention;
- A_m, the arriving PCU: the sum of PCU of moving objects whose estimated time of arrival (ETA) d_{stop}/v is at most $H = 10$ s;
- O_b, the PCU occupancy of exit link b, whose nominal capacity is $C_b \approx 23$ PCU (2×75 m/6.5 m per PCU). This figure shares the optimism noted in Section 5.2.3, since the exit is observable over roughly 60 m rather than 75 m; it is retained because the spillback guard compares a ratio against a fixed threshold, so a consistently scaled capacity shifts only where that threshold sits.

For each phase ϕ, R_ϕ is the time since ϕ last had green. PRISM-R adds three further state elements.

(i) Offside-lane split.

The offside lane carries both through and right-turning traffic, and the kerb lane carries only through traffic. Splitting an offside-lane object by the raw turning shares is therefore wrong, because it ignores the fact that part of the through traffic is not in that lane at all. If T and R are the through and right turning shares and a fraction $f_k = 0.75$ of through traffic uses the kerb lane, the share of offside-lane traffic that is turning right is

$$\pi_R = \frac{R}{R + (1 - f_k)\,T}, \tag{5.12}$$

which for the observed shares gives 0.667 rather than the 0.33 that the raw shares would suggest. A stationary offside-lane object contributes π_R of its PCU to the right movement and $1 - \pi_R$ to the through movement.

(ii) Hidden queue.

While a movement's visible queue reaches the edge of the sensor range, that is, while some stopped object of the movement lies at least 50 m back, the controller can no longer measure the queue and must instead account for it. Arrivals at the learnt rate λ_m are accumulated into a *hidden queue* Q_m^{hid}, where λ_m is an exponentially weighted moving average, with time constant $\tau_\lambda = 120$ s, of the rate at which tracks enter the range beyond 45 m. A track that enters the observable range is removed from the hidden queue, so the same vehicle is never counted twice. The hidden queue is capped at 60 PCU, which is roughly the standing capacity of an approach beyond the sensor, and is cleared entirely whenever every object of the movement is nearer than 30 m, since the queue has then demonstrably dissipated. It is added to Q_m in proportion to T and R. This is the device discussed in Section 2.5.1: it makes the state degrade conservatively, rather than silently, at the range edge.

(iii) Occlusion persistence.

A stopped object on an approach that disappears from the object list without ever having been seen moving is held in the state, at its last position, for a further 5 s. A vehicle that stops behind a bus does not cease to occupy road space, and the alternative, deleting it immediately, was what allowed heavy-vehicle traffic to erase its own queue from the estimate. Five seconds is short enough that a vehicle which genuinely departed during an occlusion is forgotten within one or two decisions.

5.2.4.2 Equations

The movement pressure is

$$p_m = Q_m + \gamma A_m - \delta\, O_{\mathrm{exit}(m)}, \qquad \gamma = 0.5,\ \delta = 0.6. \tag{5.13}$$

The terms are the upstream queue, the platoon about to arrive and the downstream penalty that discourages feeding a congested exit [55]. Phase pressure aggregates its movements:

$$P_\phi = \sum_{m \in \phi} p_m. \tag{5.14}$$

To encode fairness, each phase receives an urgency multiplier that grows with the square of its red time,

$$U_\phi = 1 + \kappa \left(\frac{R_\phi}{T_{\text{starve}}} \right)^2, \qquad \kappa = 1.5, \tag{5.15}$$

so that urgency is negligible for short reds but reaches $1 + \kappa = 2.5$ at the starvation cap. The phase score is

$$S_\phi = \max(P_\phi, 0) \cdot U_\phi. \tag{5.16}$$

Clipping at zero stops an empty phase being promoted. The green time required to discharge the current demand is estimated as

$$G_{\text{req}}(\phi) = \text{clip}\left(l_s + h_s \cdot \frac{\sum_{m \in \phi} (Q_m + \gamma A_m)}{n_{\text{lanes}}(\phi)},\ G_{\min}(\phi),\ G_{\max}(\phi) \right), \tag{5.17}$$

with start-up lost time $l_s = 2.0$ s, saturation headway $h_s = 3600/1850 \approx 1.95$ s/PCU and $n_{\text{lanes}}(\phi)$ the number of approach lanes discharging during ϕ (four for a through phase, two for a right-turn phase).

Equations (5.13) to (5.16) are unchanged in PRISM-R. Equation (5.17) is amended in two ways, which between them fix causes 2 and 3 of the diagnosis.

(i) Range-edge blend.

If any movement of the phase has a queue that reaches the edge of the sensor range, the required green cannot be computed from the visible queue, because that queue is a lower bound of unknown slackness. The estimate is then blended halfway towards the maximum,

$$G_{\text{req}} \leftarrow G_{\text{req}} + \eta \left(G_{\max}(\phi) - G_{\text{req}} \right), \qquad \eta = 0.5, \tag{5.18}$$

so that a saturated approach is given roughly the green it would need if the queue continued beyond what can be seen. Blending rather than jumping to $G_{\max}$ keeps the controller responsive if the queue turns out to be short.

(ii) Frozen target.

$G_{\text{req}}(\phi)$ is evaluated *once*, at the instant the green of ϕ begins, and held constant for the whole of that green. The original specification recomputed it at every decision step, which made the target chase the shrinking queue it was meant to discharge and produced the 11 s

greens described above. Freezing it restores the intended meaning of the quantity: the green needed to clear the queue that was present when the green started.

5.2.4.3 Decision loop

Every $\Delta = 1$ s during the green of phase g with elapsed green t_g, the controller evaluates seven rules in order and acts on the first that fires. Say that a phase *can discharge* if it has at least one queued or arriving PCU on a movement whose exit is not spilling back.

1. **Minimum green.** If $t_g < G_{\min}(g)$, extend.
2. **Spillback or max-out.** If an exit fed by g has $O_b/C_b > 0.85$, terminate. Terminate also if $t_g \geq G_{\max}(g)$, but *only* while some other phase is waiting: a green that has reached its maximum with nothing else to serve is not worth the 5 s of intergreen it would cost to end.
3. **Starvation.** If any other phase ϕ has $R_\phi \geq T_{\text{starve}}$ and is waiting, terminate. "Waiting" means any queued or arriving PCU, $Q_\phi + A_\phi > 0$, rather than the original $P_\phi > 0$: a phase whose pressure is driven negative by a congested exit is still waiting, and under the original test it could be passed over indefinitely. A phase whose own exit is spilling back is *not* forced in by this rule, since releasing traffic into a blocked exit helps no one.
4. **Rest in red.** If no approach anywhere at the junction is occupied, terminate and rest in all-red. The first vehicle subsequently detected on any arm receives its green directly from all-red, so it waits only the clearance interval instead of finding the green committed to an empty approach.
5. **Rest in green.** If no other phase can discharge, extend. There is nothing to be gained by taking the green away from the only movement able to use it.
6. **Gap-out.** If $\sum_{m \in g} Q_m \leq 0.5$ PCU and no object of g has ETA $\leq \tau_{\text{gap}} = 3$ s, terminate.
7. **Pressure switch.** If $\max_{\phi \neq g} S_\phi > (1 + \theta) S_g + \varepsilon$ for a phase that can discharge, with $\theta = 0.10$ and $\varepsilon = 2$ PCU, and $t_g \geq \rho\, G_{\text{req}}(g)$ with $\rho = 1.0$ evaluated against the frozen target, terminate.

If no rule fires, the green is extended by 1 s.

On termination, 3 s amber and 2 s all-red follow. At the end of the all-red the next phase is chosen. Starved phases are served first, oldest first. Otherwise selection is *cyclic with skipping*: the controller takes the first phase after g in the fixed order $\phi_1 \to \phi_2 \to \phi_3 \to \phi_4$ that can discharge, passing over any phase with nothing to serve. If no phase can discharge, the strongest phase by S_ϕ is chosen. This replaces the acyclic arg-max of the original specification and follows the cyclic max-pressure construction of Levin, Hu and Odell [57,58]: it guarantees a protected right turn its turn whenever it has demand, which the arg-max never did, while the skip rule prevents the fixed order from degenerating into a fixed-time plan.

Two further behaviours, left unspecified in the original design, were settled during the revision. A *short sensor gap* of one to four scans holds the current green rather than taking any decision, because a single missed scan is far more likely than a junction that has emptied instantaneously. Entry to the *fail-safe* is not instantaneous either: the controller runs the normal 3 s amber and 2 s all-red before handing over, and the fixed plan then starts at the beginning of its ϕ_1, so that drivers see a lawful transition and a familiar plan from its natural start. Recovery is likewise deferred until B1 is again at the start of ϕ_1, which prevents the adaptive law from resuming mid-cycle.

Two guards complete the design. The *spillback guard* applies when any exit has $O_b/C_b > 0.85$: every phase feeding b has its effective maximum green reduced to $G_{\min}$, keeping vehicles out of a blocked junction box. The *fail-safe* switches to the observed 78 s fixed-time plan (B1) when the VLS delivers no data for five consecutive scans, and PRISM resumes at the next cycle boundary once ten consecutive valid scans have been received. The complete logic is given in Algorithm 5.3 and Fig. 5.1.

5.2.4.4 Properties

(i) Safety.

PRISM never writes a signal state directly. It selects one of four pre-verified green states, and every transition passes through the 3 s amber and 2 s all-red states of Appendix E, so conflicting greens and truncated intergreens are impossible by construction. This was also checked by an automated test over every simulated step (Appendix D).

(ii) Bounded fairness.

Let phase ϕ end its green at time t_0. It becomes starved at $t_0 + T_{\text{starve}}$. From then on, rule 3 terminates any running green at the first decision step after that green reaches $G_{\min}$, and the oldest-first rule serves ϕ after the following 5 s intergreen. If another phase became starved earlier, it is served first, but its green is itself cut at $G_{\min}$ because ϕ is starved. The extra time beyond T_{starve} therefore consists of at most minimum greens and intergreens, which together are always shorter than one maximum through green plus one intergreen.

The bound survives the revisions of PRISM-R because the starvation rule still outranks both resting rules and the cyclic order: rest in green (rule 5) and the cyclic selection are consulted only after rule 3 has found no starved phase, so neither can hold the green against a phase that has reached T_{starve}. The corrected definition of "waiting" strengthens the guarantee rather than weakening it, since a phase now qualifies on $Q_\phi + A_\phi > 0$ and no longer loses its claim merely because a congested exit has driven its pressure negative. The single exception is deliberate: a phase whose own exit is spilling back is not forced in, because its vehicles could not discharge in any case, and it reclaims its turn as soon as the exit clears. A conservative

Algorithm 5.3: PRISM-R decision procedure (executed every $\Delta = 1$ s)

Input: object list O, current green g, elapsed t_g, red timers R_ϕ
Output: action $\in$ {EXTEND, TERMINATE} and a logged reason

Function `Decide(`O, g, t_g`)`**:**
  if *O missing for 1–4 consecutive scans* **then return** (EXTEND, "sensor gap")
  if *O missing for 5 consecutive scans* **then return** FAILSAFE(amber, all-red, B1 plan from ϕ_1)
  update hidden queues and held occluded objects; compute Q_m, A_m, O_b from O with the offside split (Eq. (5.12))
  p_m (Eq. (5.13)); P_ϕ, U_ϕ, S_ϕ (Eqs. (5.14) to (5.16)); $G_{\text{req}}(g)$ is the value frozen at the start of this green
  if $t_g < G_{\min}(g)$ **then return** (EXTEND, "min green")
  // R1 **if** *g feeds an exit with $O_b/C_b > 0.85$* **then return** (TERMINATE, "spillback")
  // R2 **if** $t_g \geq G_{\max}(g)$ ***and*** *some $\phi \neq g$ is waiting* **then return** (TERMINATE, "max out")
  if *$\exists \phi \neq g$ waiting, exit clear, $R_\phi \geq T_{starve}$* **then return** (TERMINATE, "starvation")
  // R3 **if** *no approach at the junction is occupied* **then return** (TERMINATE, "rest in red")
  // R4 **if** *no $\phi \neq g$ can discharge* **then return** (EXTEND, "rest in green")
  // R5 **if** $\sum_{m \in g} Q_m \leq 0.5$ *and no* $ETA_g \leq \tau_{gap}$ **then return** (TERMINATE, "gap out")
  // R6 **if** *$\exists \phi \neq g$ that can discharge with* $S_\phi > (1+\theta)S_g + \varepsilon$ ***and*** $t_g \geq \rho\, G_{req}(g)$ **then**
    return (TERMINATE, "pressure switch")
  // R7 **return** (EXTEND, "extend")

On terminate: amber 3 s; all-red 2 s; recompute S_ϕ
if *starved phases exist* **then**
  $g' \leftarrow$ the starved phase with largest R_ϕ
else if *some phase can discharge* **then**
  $g' \leftarrow$ the first phase after g in $\phi_1 \to \phi_2 \to \phi_3 \to \phi_4$ that can discharge
  `// cyclic with skip`
else
  $g' \leftarrow \arg\max_\phi S_\phi$, or rest in all-red if the junction is empty
start green of g'; freeze $G_{\text{req}}(g')$; $R_g \leftarrow 0$ at end of g's green; log decision record

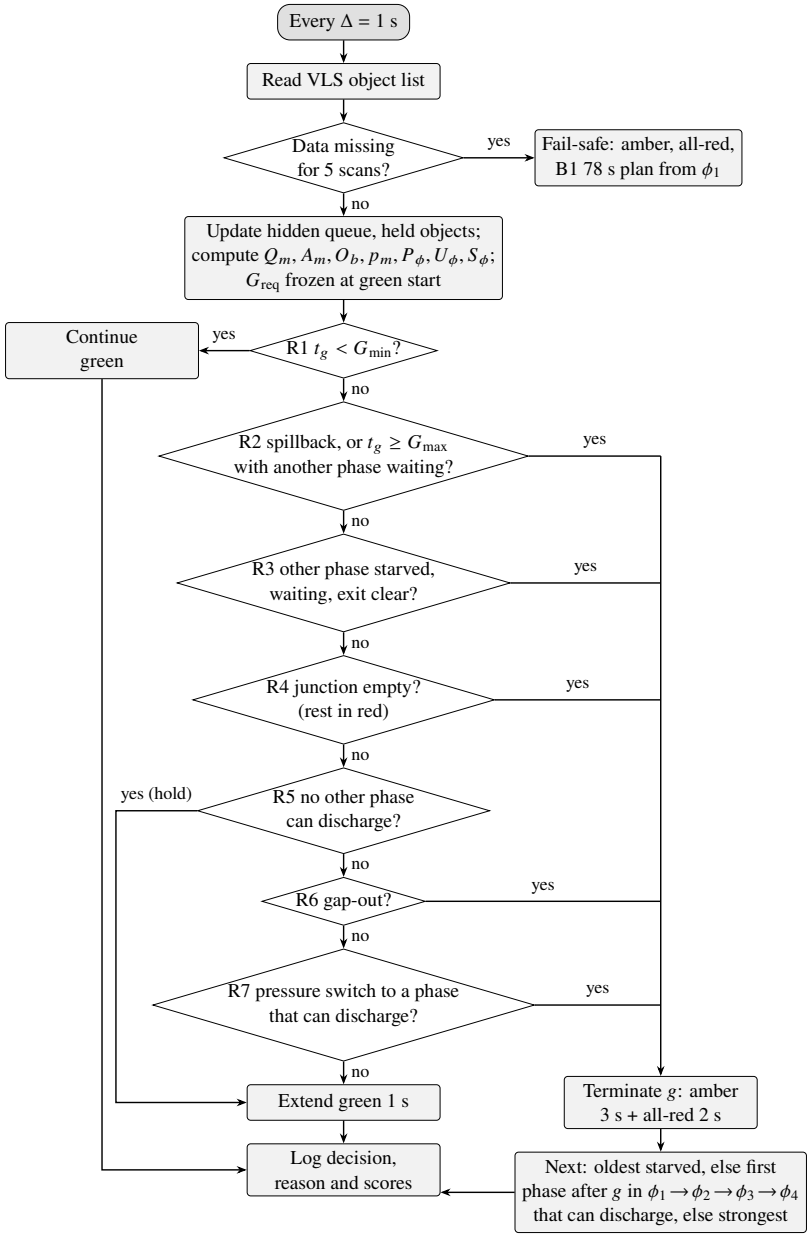


Figure 5.1: Flowchart of the PRISM-R decision procedure, with the seven ordered rules R1–R7 and cyclic next-phase selection

bound on the red time of any phase is

$$R^{\text{worst}} \leq T_{\text{starve}} + G_{\max} + 5 \text{ s} = 120 + 50 + 5 = 175 \text{ s}, \tag{5.19}$$

using the through-phase $G_{\max}$ of 50 s, which is the longest green any phase can hold.

(iii) Stability intuition.

Varaiya [20] proved that serving the phase of maximum pressure stabilises queues for any demand inside the capacity region, using only local information. PRISM keeps that core because phases are ranked by pressure. However, the minimum and maximum greens, the hysteresis θ and the urgency factor mean that the formal proof does not transfer exactly. This trade-off for fairness is also made by cycle-based variants [54]. PRISM-R's cyclic selection makes the position stronger rather than weaker. Levin, Hu and Odell [58] prove that imposing a fixed phase order, with skipping of phases that have no demand, preserves the maximum stability region of acyclic max-pressure, and Le et al. [57] reach a comparable conclusion for constrained decentralised policies. The cyclic-with-skip rule used here is exactly that construction, so the departure from Varaiya's policy now lies only in the green bounds, the hysteresis and the urgency term.

(iv) Complexity.

Each object is assigned to one movement and one exit, so state computation is $O(n)$ in the number of objects n, and the memory held is likewise $O(n)$: one record per track, plus the track identifiers and occlusion-held objects, which are retained for 5 s. The mean decision time measured was 0.25 ms at 400 objects, with about 162 KiB retained (Section 6.2.1); this supersedes the 0.4 ms reported for the original formulation.

(v) Explainability.

Every decision writes a record containing time, phase, t_g, the rule that fired, all P_ϕ, U_ϕ and S_ϕ, and G_{req}. For example, "pressure switch: $S_{\phi_3} = 18.4 > 1.25 \times 11.2 + 2$".

(vi) Parameter justification.

Table 5.3 lists the design parameters and the reasoning behind each value. They were fixed on the balanced 300, 500 and 700 veh/h scenarios using tuning seeds 1–5 only, by a grid search over the edge blend η, the discharge ratio ρ and the hysteresis θ; no evaluation seed and none of the seven situation scenarios took part in the tuning.

Table 5.3: PRISM-R design parameters and their justification; values that differ from the original specification are marked

Parameter	Value	Justification
γ (arrival weight)	0.5	Arrivals within H are uncertain and not yet delaying anyone, so they count half as much as queued PCU.
δ (exit weight)	0.6	Penalises congested exits without letting downstream noise dominate the upstream queue ($\delta < 1$).
κ (urgency gain)	1.5	Score at most 2.5× at T_{starve}; the quadratic form keeps short reds almost unaffected.
θ (hysteresis)	**0.10**	*Was 0.25.* With $\rho = 1$ already preventing early switching, a large hysteresis only delayed switches that were justified; a challenger must still be 10% stronger, which remains enough to absorb sensor noise.
ε (margin)	2 PCU	Absolute margin for light traffic, where relative margins are meaningless.
ρ (discharge ratio)	**1.0**	*Was 0.6.* A pressure switch is now allowed only once the green has run for the whole required time, so the queue present at green start has had time to discharge. With the frozen target this is a meaningful commitment rather than, as before, a moving one.
η (range-edge blend)	0.5	Moves G_{req} halfway to $G_{\max}$ when the queue reaches the range edge (Eq. (5.18)); blending rather than saturating keeps the controller responsive if the hidden queue is short.
Range-edge distance	50 m	A stopped object at least this far back is treated as touching the edge of the observable range.
Hidden-queue entry / clear radius	45 / 30 m	Tracks entering beyond 45 m feed the learnt arrival rate; the hidden queue is cleared when every object is nearer than 30 m.
Hidden-queue EWMA constant	120 s	Long enough to average out Poisson arrival noise, short enough to follow a peak.
Hidden-queue cap	60 PCU	Roughly the standing capacity of an approach beyond the sensor; prevents unbounded growth if the clear condition is never met.
Occlusion hold	5 s	A stopped object that vanishes unseen keeps counting; short enough to forget a vehicle that genuinely departed.

continued on next page

Parameter	Value	Justification
Next-phase selection	cyclic with skip	Fixed order $\phi_1 \rightarrow \phi_4$, skipping phases that cannot discharge; guarantees right turns their turn [58].
Rest in green	enabled	Hold the current green when no other phase can discharge.
Rest in red	enabled	Rest in all-red when the junction is empty, so the next arrival waits only the clearance interval.
H (horizon)	10 s	About 100 m at free-flow speed, which covers the whole observable approach.
τ_{gap} (gap time)	3 s	Standard actuated gap; equal to the B2 `max-gap` for a fair comparison.
l_s (start-up lost time)	2.0 s	Typical start-up lost time [9].
h_s (headway per PCU)	1.95 s	3600/1850, from the DTX saturation flow.
$G_{\min}$	10 / 5 s	Through / right; driver expectation and clearance of the queue head.
$G_{\max}$	50 / 25 s	Through / right; limits red time of the other three phases.
T_{starve}	120 s	Longer than any observed red (67 s) and acceptable to waiting drivers.
Amber / all-red	3 s / 2 s	Same as the existing signal.
Δ (decision period)	1 s	Two SUMO steps; matches the VLS scan rate.

5.2.5 Baseline controllers

Three baselines, an oracle variant and one ablation were implemented.

(i) B1: observed fixed-time plan.

The existing signal runs the 78 s plan of Table 5.4. The greens of ϕ_1 and ϕ_2 and the red of ϕ_3 were timed from the footage. Since every cycle contains four intergreens of $3+2 = 5$ s, the ϕ_3 green is $78-3-60 = 15$ s, and the remaining green follows from $21+8+15+G_{\phi_4}+4\times5 = 78$, giving 14 s for ϕ_4.

(ii) B2: SUMO actuated.

SUMO's built-in gap-based actuated controller [31] was run on the same four phases with `minDur` 10/5 s and `maxDur` 50/25 s for through/right phases, its *default* induction loops and a maximum gap of 3 s. It extends a green while vehicles keep arriving but ignores vehicle size and the demand on other phases. One limitation of this configuration must be stated

Table 5.4: Baseline B1: observed fixed-time plan (cycle 78 s)

Phase	Movement (both directions)	Green (s)	Source
ϕ_1	Bypass through	21	Observed (red 54 s)
ϕ_2	Bypass protected right	8	Observed (red 67 s)
ϕ_3	Cross-road through	15	$78 - 3 - 60$
ϕ_4	Cross-road protected right	14	$78 - (21 + 8 + 15) - 4 \times 5$
Intergreen after each phase		$3 + 2$	Amber + all-red

plainly, because it affects the comparison. SUMO generates default detectors for lanes that serve a phase's *major* movements, and the protected right-turn phases ϕ_2 and ϕ_4 receive no detectors of their own. Those phases therefore cannot gap out on their own demand and always run to `minDur`, which starves right turns under heavy demand and is the main reason B2 deteriorates so sharply at 700 veh/h (Section 6.3.4). Adding hand-placed detectors would have made B2 stronger, but it would also have made it a different, hand-tuned controller; the default configuration was kept because it is what an authority deploying SUMO-style actuation out of the box would obtain, and the weakness is reported rather than concealed.

(iii) B3: Webster-optimised fixed time.

This separates adaptivity from simple re-timing. Webster's method [11] was applied *per scenario*, to the mean demand that `gen_scenario_routes.py` writes for that scenario, so that each scenario is compared against the best fixed plan for its own demand rather than against one plan for all ten. Left turns use the slip lanes and were excluded. Because waiting right-turners occupy the offside lane during through phases, DTX showed that 75% of through traffic used the kerb lane, so the critical lane flow of a through phase is 0.75 of the heavier approach's through flow. For a right-turn phase it is the full right-turn flow in the offside lane. The flow ratio of phase i is $y_i = q_i/s$ with $s = 1850$ PCU/h/lane, $Y = \sum_i y_i$, the lost time is $L = 4 \times 5 = 20$ s, and

$$C_0 = \frac{1.5L + 5}{1 - Y}, \qquad g_i = (C_0 - L)\,\frac{y_i}{Y}. \tag{5.20}$$

At the demand levels of most scenarios Y is small, so Eq. (5.20) returns a cycle whose effective green $C_0 - L$ is shorter than the sum of the minimum greens, $\sum_i G_{\min}(i) = 10 + 5 + 10 + 5 = 30$ s. In that case the cycle is raised to $\sum_i G_{\min}(i) + L$ and the greens are set to the minima, since a plan that cannot clear the head of a queue is not a fair benchmark. The result is a 50 s cycle at 10/5/10/5 s for most scenarios, and a 61 s cycle at 13/8/12/8 s for the heavy-vehicle scenario and for balanced 700 veh/h, where Y is large enough for Eq. (5.20) to bind (Table 5.6).

Two observations follow. First, in eight of the ten scenarios Webster's formula asks for less green than the minimum greens allow, so B3 reduces to the shortest lawful cycle. This is

Table 5.5: Hourly demand extracted by DTX from the two survey sessions, used to calibrate the model and to parameterise the scenario generator (PCU/h; L = slip lane, uncontrolled)

	Morning				Afternoon			
Approach	L	T	R	Total	L	T	R	Total
Bypass 1	150	960	170	1,280	90	570	120	780
Bypass 2	120	680	130	930	80	480	100	660
Cross 1	100	340	150	590	60	300	110	470
Cross 2	60	210	80	350	35	100	40	175
Total	430	2,190	530	3,150	265	1,450	370	2,085

Table 5.6: Webster plans for B3, computed per scenario from its mean demand (times in s)

Scenario	Y	Cycle (s)	Greens $\phi_1/\phi_2/\phi_3/\phi_4$ (s)
Balanced 300	0.17	50	10 / 5 / 10 / 5
Balanced 500	0.29	50	10 / 5 / 10 / 5
Balanced 700	0.40	61	13 / 8 / 12 / 8
Empty side road	0.21	50	10 / 5 / 10 / 5
Unbalanced roads	0.27	50	10 / 5 / 10 / 5
Peak (mean)	0.24	50	10 / 5 / 10 / 5
Platoon surge (mean)	0.25	50	10 / 5 / 10 / 5
Night	0.05	50	10 / 5 / 10 / 5
Heavy vehicles	0.42	61	13 / 8 / 12 / 8
Sensor failure	0.29	50	10 / 5 / 10 / 5

A cycle of 50 s is $\sum_i G_{\min}(i) + L = 30 + 20$, applied where $C_0 - L < \sum_i G_{\min}(i)$.

not a defect of the method but a consequence of the junction being undersaturated for much of the day, and it already represents a large improvement on the existing plan, whose 78 s cycle spends 20 s of every cycle on intergreens regardless of demand. Second, where the formula does bind, at balanced 700 veh/h and in the heavy-vehicle scenario, it lengthens the cycle to 61 s and shifts green towards the bypass. The comparison with the existing plan is instructive in its own right: under B1, ϕ_1 carries a degree of saturation $x = y\,C/g$ well above unity at the observed morning demand, so the bypass through movement is heavily oversaturated while ϕ_4 runs far below capacity. The existing plan therefore allocates its green time to the wrong phases and uses one cycle length for very different demand levels. However, Webster's formula assumes stationary random arrivals and undersaturation, and it ignores queues that spill back past the slip-lane entries, so it cannot respond to the peak, the surge or the empty side road at all. The simulated benefit of B3 was therefore expected to be smaller than the analytical one (Section 6.3.4).

(iv) PRISM-GT (oracle).

The same PRISM-R code was run with the VLS replaced by an exact ground-truth object list, with true positions, no dropout or occlusion and true-class PCU rather than size-class PCU. The gap between PRISM-R and PRISM-GT measures the cost of imperfect sensing.

(v) PRISM, thesis law (ablation).

The control law exactly as first specified in this chapter, before the revisions of Section 5.2.4, was run on identical sensing: recomputed G_{req}, no range-edge blend, no hidden queue, no occlusion persistence, $\theta = 0.25$, $\rho = 0.6$, six rules and acyclic arg-max selection. Because every revision is a parameter, this ablation differs from PRISM-R only in configuration, so the difference between them isolates the effect of the revisions rather than of any other change. It is reported alongside the baselines throughout Chapter 6.

5.3 Technology selection

Every technology was selected by comparing realistic alternatives against the needs of this research: free availability to a student, maturity, support for small objects in aerial video, support for heterogeneous lane-less traffic and scriptable control of the signal. Tables 5.7 and 5.8 summarise the decisions. No web front-end or custom dashboard was built. Results and live decisions were presented through SUMO-GUI only, with the VLS points and the current PRISM reason overlaid. This kept the effort on the control law.

Table 5.7: Technology selection: language, models and simulator

Concern	Alternatives	Chosen	Reason
Programming language	C++, Java, MATLAB, Python	Python 3.11	Native TraCI/libsumo bindings; Ultralytics, OpenCV and SciPy in one language; fast prototyping. C++ speed is unnecessary because a decision takes under 1 ms. MATLAB is licensed and has weak SUMO support.
Detection model	YOLO11, RT-DETR, Faster R-CNN, YOLOv8s	YOLOv8s	Mature, well documented, VisDrone weights available, trains at 1280 px on a free T4. YOLO11 [113] offered marginal gains with less community evidence at the time; RT-DETR [67] needs more GPU memory; Faster R-CNN [61] is two-stage and too slow for two hours of video.
Tracker	SORT, DeepSORT, BoT-SORT, ByteTrack	ByteTrack	Recovers low-score boxes of small motorcycles; needs no re-identification network, which is of little use for top-down vehicles. SORT [71] loses occluded objects; DeepSORT [72] and BoT-SORT [73] add appearance models with little benefit here.
Simulator	VISSIM, Aimsun, CARLA, SUMO	SUMO	Open source; sublane model for motorcycle filtering; TraCI for per-second signal control. VISSIM [114] and Aimsun [115] are commercial; CARLA [88] targets single-vehicle autonomy, not junction throughput.

Table 5.8: Technology selection: libraries and tooling

Need	Chosen (alternatives)	Reason
Image processing	OpenCV [116] (scikit-image)	ORB, RANSAC homography and warping in optimised C++ behind a Python interface.
Numerics	NumPy/SciPy [117, 118]	Vectorised geometry; Savitzky–Golay filter; Wilcoxon test.
Tabular data	Pandas [119] (plain CSV)	Grouping trajectories by track, class and 5-min bin.
Tracking utilities	supervision [120]	Maintained ByteTrack implementation, zone polygons and annotators.
Small objects	SAHI [24] (higher input size only)	Tiled inference raises recall for 20-pixel motorcycles without retraining at 4K.
Simulation API	libsumo for batches, TraCI for GUI runs	libsumo runs in-process and avoids socket overhead; TraCI is required to drive SUMO-GUI.
Annotation	CVAT and Roboflow	Free; export directly to YOLO format; Roboflow handled the 70/15/15 split.
Testing	pytest (unittest)	Concise fixtures and parametrised tests for phase safety and edge cases.
Version control	Git and GitHub	History of every configuration and seed, which supports reproducibility.
Compute	Google Colab T4 GPU (local CPU)	Free GPU for fine-tuning; simulation runs on the student laptop.
Front-end	None (SUMO-GUI only)	A dashboard was a “Won’t have” requirement; SUMO-GUI shows the network, vehicles and signal states.

5.4 Key implementation

The code follows the repository layout planned in Section 4.5: `extraction/` for DTX, `sumo/` for the network and demand, `sensing/` for the VLS, `control/` for PRISM and the baselines, `experiments/` for batch runs and statistics, and `tests/`. Only the fragments that embody the contribution are shown here; longer listings are in Appendix D.

(i) Waiting-time computation.

Listing 5.1 implements Eqs. (5.2) to (5.4). The decisive detail is that the 0.1 m/s threshold is the same constant SUMO uses, so observed and simulated waiting times are directly comparable during calibration.

Listing 5.1: Per-vehicle waiting time and control delay (`extraction/metrics.py`)

```
WAIT_SPEED = 0.1 # m/s, identical to SUMO's waitingTime threshold
DT = 0.1 # s, video processed at 10 fps

def vehicle_metrics(trk: pd.DataFrame, zones: ZoneMap, v_free: float) -> dict:
    """Waiting time W_i, time in frame and control delay d_i for one track."""
    xy = savgol_filter(trk[["x", "y"]].to_numpy(), 11, 2, axis=0)
    step = np.linalg.norm(np.diff(xy, axis=0), axis=1)
    v = np.r_[step[0], step] / DT # speed per sample
    in_approach = zones.in_approach(xy) # point-in-polygon
    W = DT * np.count_nonzero((v < WAIT_SPEED) & in_approach)
    t_frame = trk.t.iloc[-1] - trk.t.iloc[0]
    delay = max(t_frame - step.sum() / v_free, 0.0)
    censored = (trk.t.iloc[0] <= zones.t_start + DT or
                trk.t.iloc[-1] >= zones.t_end - DT)
    return {"track_id": int(trk.track_id.iloc[0]),
            "cls": trk.cls.mode().iat[0], # majority vote
            "movement": zones.movement(xy),
            "W": W, "t_frame": t_frame, "delay": delay,
            "censored": censored}
```

(ii) VLS scan.

Listing 5.2 shows the error model. Vehicles are processed in increasing range so that the list of potential occluders only contains nearer tall vehicles, which keeps the occlusion test cheap.

Listing 5.2: Noisy, occluded LiDAR scan (`sensing/virtual_lidar.py`)

```
def scan(self, vehicles: list[GTVehicle]) -> list[Detection]:
    near = sorted((v for v in vehicles if self.range(v) <= self.R), key=self.
        range)
    dets, blockers = [], [] # blockers: tall vehicles seen so far
    for v in near:
        r = self.range(v)
        span = self.azimuth_span(v)
        if any(overlap(span, b) >= 0.7 * width(span) for b in blockers):
            continue # occluded by bus/lorry
        if v.height >= 2.5:
```

```
            blockers.append(span)
        if self.rng.random() < 0.01 + 0.04 * (r / self.R) ** 2:
            continue # range-dependent dropout
        sigma = 0.10 + 0.002 * r
        x, y = v.pos + self.rng.normal(0.0, sigma, 2)
        length = v.length + self.rng.normal(0.0, sigma)
        dets.append(Detection(x, y, size_class(length)))
    for _ in range(self.rng.poisson(0.2)): # clutter
        dets.append(Detection(*self.random_point_in_range(), "small"))
    return dets

def size_class(length: float) -> str:
    return "small" if length < 3.2 else "medium" if length <= 6.0 else "large"
```

(iii) Pressure and score.

Listing 5.3 computes Eqs. (5.13) to (5.16) in a single pass over the object list. This is the source of the $O(n)$ complexity.

Listing 5.3: Movement pressure and phase score (`control/prism.py`)

```
def movement_state(self, objs: list[TrackedObject]) -> dict[Movement, MovState]:

    st = {m: MovState() for m in MOVEMENTS}
    seen = {m: [] for m in MOVEMENTS}
    for o in objs + self.held_occluded(self.t_now): # single O(n) pass
        if o.approach is None: # exit link
            self.occ[o.exit_link] += o.pcu
            continue
        for m, share in self.split(o): # offside lane: eq. (5.x)
            if o.v < Q_SPEED and o.d_stop <= self.p.range_m:
                st[m].Q += share * o.pcu
                seen[m].append(o.d_stop)
            elif o.v >= Q_SPEED and o.d_stop / o.v <= self.p.H:
                st[m].A += share * o.pcu
                st[m].min_eta = min(st[m].min_eta, o.d_stop / o.v)
    for m, ds in seen.items(): # hidden queue at the range edge
        st[m].at_edge = bool(ds) and max(ds) >= self.p.edge_m # 50 m
        self.hidden[m] = self.update_hidden(m, st[m].at_edge, objs)
        st[m].Q += self.hidden[m]
    return st

def scores(self, st, t_now: float) -> dict[int, float]:
    S = {}
    for ph in PHASES:
        P = sum(st[m].Q + self.p.gamma * st[m].A
                - self.p.delta * self.occ[EXIT[m]] for m in ph.movements)
        U = 1.0 + self.p.kappa * (self.red_time(ph, t_now) / self.p.T_starve) **
            2
        S[ph.id] = max(P, 0.0) * U
    return S
```

(iv) Decision loop.

Listing 5.4 is a direct transcription of the seven rules. Each branch returns a reason string, which provides the explainability. Note that `self.g_req_frozen` is set once, in `start_green()`, and is not recomputed here.

Listing 5.4: The `decide()` rule cascade (`control/prism.py`)

```
def decide(self, objs, t_now: float) -> Decision:
    st = self.movement_state(objs)
    S = self.scores(st, t_now)
    g, tg, p = self.green, t_now - self.green_start, self.p
    others = [ph for ph in PHASES if ph is not g]
    waiting = [ph for ph in others if self.is_waiting(ph, st)]
    ready = [ph for ph in others if self.can_discharge(ph, st)]
    if tg < g.gmin: # R1
        return Decision(EXTEND, "min_green", S)
    if self.spillback(g): # R2
        return Decision(TERMINATE, "spillback", S)
    if tg >= g.gmax and waiting:
        return Decision(TERMINATE, "max_out", S)
    if self.starved(st, t_now, exclude=g): # R3
        return Decision(TERMINATE, "starvation", S)
    if not self.any_occupied(st): # R4
        return Decision(TERMINATE, "rest_in_red", S)
    if not ready: # R5
        return Decision(EXTEND, "rest_in_green", S)
    queue_g = sum(st[m].Q for m in g.movements)
    eta_g = min(st[m].min_eta for m in g.movements)
    if queue_g <= 0.5 and eta_g > p.tau_gap: # R6
        return Decision(TERMINATE, "gap_out", S)
    best = max(S[ph.id] for ph in ready) # R7
    if (best > (1 + p.theta) * S[g.id] + p.eps
            and tg >= p.rho * self.g_req_frozen):
        return Decision(TERMINATE, "pressure_switch", S)
    return Decision(EXTEND, "extend", S)
```

(v) TraCI integration.

Listing 5.5 shows the experiment loop. The controller is called every second step, which gives the 1 s decision period with a 0.5 s simulation step. Signal states are set only through the phase manager, which inserts amber and all-red. The same loop drives B1–B3, PRISM and PRISM-GT, selected from `scenarios.yaml`, so the comparison is like-for-like.

Listing 5.5: Simulation loop (`experiments/run_experiment.py`)

```
def run(scn: Scenario, seed: int, gui: bool = False) -> Path:
    sim = traci if gui else libsumo
    sim.start(["sumo-gui" if gui else "sumo", "-c", scn.cfg, "--seed", str(seed),

               "--step-length", "0.5", "--lateral-resolution", "0.8",
               "--tripinfo-output", scn.out(seed, "tripinfo.xml")])
    sensor = GroundTruth(sim) if scn.oracle else VirtualLidar(sim, seed=seed)
    tracker = PerceptionTracker(alpha=0.5, beta=0.1, gate=2.5)
    ctrl = make_controller(scn.controller, PhaseManager(sim, TLS_ID))
```

```
    log, step = DecisionLog(scn.out(seed, "decisions.csv")), 0
    while sim.simulation.getMinExpectedNumber() > 0 and step < 2 * 3900:
        sim.simulationStep()
        step += 1
        if step % 2: # decide every 1 s (two 0.5 s steps)
            continue
        t = sim.simulation.getTime()
        objs = tracker.update(sensor.scan(), t)
        decision = ctrl.step(objs, t) # PRISM: decide(); B1/B3: plan; B2: SUMO
        log.write(t, decision)
        if gui:
            show_overlay(sim, objs, decision) # VLS points and reason
    sim.close()
    return scn.out(seed, "tripinfo.xml")
```

(vi) SUMO configuration.

Listing 5.6 gives an excerpt of the vehicle types and the B1 program. The fail-safe re-uses exactly this program, so the fallback is the plan that road users already know.

Listing 5.6: Excerpt of `vtypes.add.xml` and the B1 signal program

```
<vType id="motorcycle" vClass="motorcycle" length="2.0" maxSpeed="19.44"
      accel="2.8" sigma="0.5" tau="1.0" lcSublane="10" minGapLat="0.3"
      speedFactor="normc(1,0.1)"/>
<vType id="threewheeler" vClass="passenger" length="2.7" width="1.4"
      maxSpeed="13.89" accel="1.8" sigma="0.5" tau="1.0" guiShape="passenger/
          hatchback"/>
<vType id="bus" vClass="bus" length="11.5" maxSpeed="16.67" accel="1.2"/>
<tlLogic id="maviththarа" type="static" programID="B1_fixed78" offset="0">
  <phase duration="21" state="GGrrrrGGrrrr"/> <!-- phi1 bypass through -->
  <phase duration="3" state="yyrrrryyrrrr"/>
  <phase duration="2" state="rrrrrrrrrrrr"/>
  <phase duration="8" state="rrGrrrrrGrrr"/> <!-- phi2 bypass right -->
  <phase duration="3" state="rryrrrrryrrr"/>
  <phase duration="2" state="rrrrrrrrrrrr"/>
  <phase duration="15" state="rrrGGrrrrGGr"/> <!-- phi3 cross through -->
  <phase duration="3" state="rrryyrrrryyr"/>
  <phase duration="2" state="rrrrrrrrrrrr"/>
  <phase duration="14" state="rrrrrGrrrrrG"/> <!-- phi4 cross right -->
  <phase duration="3" state="rrrrryrrrrry"/>
  <phase duration="2" state="rrrrrrrrrrrr"/>
</tlLogic>
```

5.5 Chapter summary

This chapter described how the artefact was designed and built. DTX turned the drone videos into anonymous metric trajectories through stabilisation, YOLOv8s detection with SAHI, ByteTrack tracking, homography and Savitzky–Golay smoothing, and it measured waiting time with the same 0.1 m/s definition that SUMO uses. The SUMO model reproduced the left-hand, 2+2-lane junction with give-way slip lanes, sublane motorcycles and a four-phase signal, rebuilt reproducibly from the OpenStreetMap extract by `netconvert`

patches and driven by ten drone-derived demand scenarios. The VLS supplied a noisy, occluded, class-blind object list, with two-point track initiation, coasting tracks withheld and map matching carried across the intermediate OSM nodes. PRISM combined PCU-weighted movement pressure, an arrival term, a downstream penalty and a quadratic urgency factor; after a first evaluation exposed five distinct modelling errors, it was revised into PRISM-R, which adds a hidden-queue state, a frozen green target, a range-edge blend, occlusion persistence, resting in green and in red, and cyclic phase selection with skipping, in seven transparent rules, with proven safety, a 175 s fairness bound, $O(n)$ cost and a logged reason for every decision. Three baselines were implemented, including a Webster plan retimed per scenario, together with a ground-truth oracle and an ablation that runs the original law on identical sensing.

Chapter 6 now tests whether these designs meet the requirements and how PRISM performs against the baselines.

6 TESTING AND EVALUATION

6.1 Chapter overview

This chapter reports how the three software components built in this research—the Drone Trajectory eXtractor (DTX), the Virtual LiDAR Sensor (VLS) and the Pressure-based Real-time Intersection Signal Management (PRISM) controller—were verified and then evaluated against the objectives of Section 1.7. Following the development order of Chapter 3, components were verified in isolation, exercised in closed loop on balanced demand, and finally evaluated over the ten drone-derived demand scenarios of Section 5.2.2. Unless stated otherwise, every simulation result is the mean over evaluation seeds 6–15 (300 s warm-up, 3,600 s measured). The chapter also reports the one place where evaluation fed back into design: the first comparison showed the control law losing to the fixed plan under heavy demand, and Section 6.3.4 reports both the revised law, PRISM-R, and the original formulation run as an ablation on identical sensing.

6.2 Test plan and test cases

Because an erroneous PRISM decision could display conflicting greens, correctness was established bottom-up before any performance claim; a layered strategy was preferred to a single end-to-end evaluation, which would have hidden the origin of defects. Four test levels were used (Table 6.1). Unit tests targeted functions whose correct output can be derived by hand; integration tests coupled Simulation of Urban MObility (SUMO), the Traffic Control Interface (TraCI), the VLS and the controller on short scenarios; system tests ran complete one-hour experiments with safety invariants asserted at every step; and acceptance testing compared the outcomes with objectives O3–O7.

The unit-test suite comprised 62 test functions and achieved 87% statement coverage of the `control/` and `sensing/` packages; it was re-run after every change, preventing regressions during tuning. Ten principal test cases (TC01–TC10) are reported in Table 6.2, each linked to the functional (FR) and non-functional (NFR) requirements of Section 4.6; a further twenty-three cases (TC11–TC33) are listed in Appendix C.

6.2.1 Non-functional testing

Performance. The controller must decide well within the 1 s control interval (NFR01). A PRISM-R decision took 0.25 ms on average at 400 objects in range, and the controller retained about 162 KiB. One simulated hour ran well inside the three-minute budget with libsumo, so NFR06 was also met with a wide margin (Table 6.3).

Table 6.1: Test levels, tools and environment.

Level	Scope and method	Tools
Unit	Single functions with hand-derived oracles; statistical tests for the VLS error model	`pytest`, `pytest-cov`, NumPy, SciPy
Integration	SUMO–TraCI–VLS–controller loop on 600 s mock scenarios; fault injection	SUMO 1.20.0, TraCI
System	One-hour closed-loop runs, all controllers and seeds, invariants asserted every step	libsumo, `run_experiment.py`
Acceptance	Key Performance Indicators (KPIs) compared with objectives and stakeholder needs	Pandas, SciPy, Matplotlib
Environment	Python 3.11 on the author's laptop (Table 1.2); detector evaluation on a Google Colab T4 Graphics Processing Unit (GPU)	

Table 6.2: Principal test cases (TC01–TC10).

ID	Description	FR/NFR	Input	Expected	Actual	Status
TC01	Phase safety and closed-loop invariant	FR10, FR11, NFR02	10,000 random state vectors; all closed-loop runs	Only one of ϕ_1–ϕ_4 green at any instant; no conflicting signal state	0 conflicting states; 0 collisions	Pass
TC02	Intergreen, $G_{\min}$ and $G_{\max}$	FR11	Zero demand on ϕ_1; saturated ϕ_1	Hold to 10 s; terminate at 50 s; 3 s amber + 2 s all-red after every green	10 s / 50 s; intergreen on 100% of transitions	Pass
TC03	Gap-out	FR10, FR11	$\sum Q \le 0.5$ PCU on ϕ_3, no object with ETA ≤ 3 s	Terminate at first step with $t_g \ge G_{\min}$	Terminated at $t_g = 10$ s	Pass
TC04	Pressure switch with hysteresis	FR10	24 boundary cases around $S_j = 1.10\,S_g + 2$ and $t_g = G_{\text{req}}$ (frozen)	Switch only if both conditions hold and the challenger can discharge	Correct in 24/24	Pass
TC05	Starvation override	FR12	ϕ_4 given near-zero pressure under heavy ϕ_1/ϕ_3 demand	ϕ_4 served once $R_{\phi_4} \ge 120$ s	Served; longest red 128 s (≤ 175 s bound)	Pass
TC06	Sensor fail-safe	FR13, NFR03	VLS output removed for 600 s	Hold green for 1–4 empty scans; revert to the 78 s plan at the 5th, through amber and all-red, starting at ϕ_1	Held, then reverted at the 5th empty scan with a full intergreen	Pass
TC07	Spillback guard	FR14	Exit link of ϕ_1 blocked to $O_b/C_b = 0.87$	Phases feeding the exit held to $G_{\min}$	ϕ_1 ended at 10 s	Pass
TC08	VLS noise and dropout statistics	FR08, NFR03	10^5 scans at $r = 50$ m and $r = 75$ m	$\sigma(50) = 0.20$ m; $p_{\text{miss}}(75) = 0.05$	0.201 m; 0.0497	Pass
TC09	DTX waiting time on synthetic tracks	FR05	Track halting 20 s with jitter below 0.1 m/s	$W = 20.0$ s	20.0 s	Pass
TC10	Computational performance	NFR01, NFR06	Synthetic object lists of 100–800 objects; one simulated hour	Decision ≤ 10 ms; hour ≤ 180 s	0.25 ms at 400 objects, 0.48 ms at 800; 95 s	Pass

Scalability. Decision time was measured against a synthetic object list of increasing size, which isolates the controller's cost from the simulator's. It rose from 0.07 ms at 100 objects to 0.48 ms at 800, that is, linearly in the number of objects n, confirming the $O(n)$ analysis of Section 5.2.4. Even at 800 objects, roughly four times the largest count observed in any scenario, a decision uses under 0.05% of the 1 s control interval, so the margin against NFR01 is three orders of magnitude.

Table 6.3: Computational cost of a PRISM-R decision against the number of objects in range.

Objects in range n	100	200	**400**	600	800
Decision time (ms)	0.07	0.13	**0.25**	0.37	0.48
Per object (μs)	0.70	0.65	0.63	0.62	0.60

Retained memory at 400 objects: 162 KiB. The near-constant per-object cost confirms $O(n)$ scaling.

Robustness. Graceful degradation under sensor imperfection (NFR03) was tested statistically (TC08), by noise and dropout sweeps (Section 6.3.5), by fault injection (TC06) and by the dedicated sensor-failure scenario, in which two LiDAR outages occur during the measured hour. With both outages the hourly average waiting time rose only from 19.4 to 20.9 s, against 25.7 s for the fixed plan, so the controller remained the better option even while blind for 6% of the hour.

Safety invariant monitoring. During every system run, assertions checked that no conflicting movements were green together, that every green was followed by the full 5 s intergreen and that no green was shorter than $G_{\min}$. Across 370 simulated hours (all controllers, scenarios, seeds and sensitivity runs) zero violations and zero SUMO collisions were recorded, supporting the safety argument of Section 5.2.4.

6.2.2 Functional testing

Functional tests targeted each rule of the decision loop (Algorithm 5.3) in isolation with hand-constructed states. TC01, the most important, drove the decision function with randomly generated queues, arrivals and red times and asserted that exactly one of the four phases (bypass through, bypass right, cross through, cross right) was green or that the junction was in intergreen. TC02–TC05 verified the timing rules: minimum and maximum green, gap-out, the pressure switch with hysteresis $\theta = 0.10$ and $\varepsilon = 2$ PCU (Passenger Car Unit) against the frozen G_{req}, and the starvation override at $T_{\text{starve}} = 120$ s. Two further cases covered the resting rules of PRISM-R: with no object anywhere, the controller ended the green and held all-red until a vehicle appeared, which then received green after the clearance interval alone; with demand on the running phase only, the green was held rather than surrendered to an empty approach.

TC08 verified that the VLS error model produced the intended statistics: a Kolmogorov–

Smirnov test of position errors against $\mathcal{N}(0, \sigma(r))$ with $\sigma(r) = 0.10 + 0.002r$ gave $p = 0.62$, and the empirical miss rate at the range limit lay within a binomial 99% confidence interval around $p_{\text{miss}}(R) = 0.05$. TC09 confirmed that the DTX waiting-time definition ($v < 0.1$ m/s) reproduces SUMO's `waitingTime`, a prerequisite for Section 6.3.3. All ten cases passed.

6.3 Testing and evaluation workflow

After verification, evaluation proceeded through the five stages shown in Fig. 6.1. Each stage produced evidence on which the next relied.

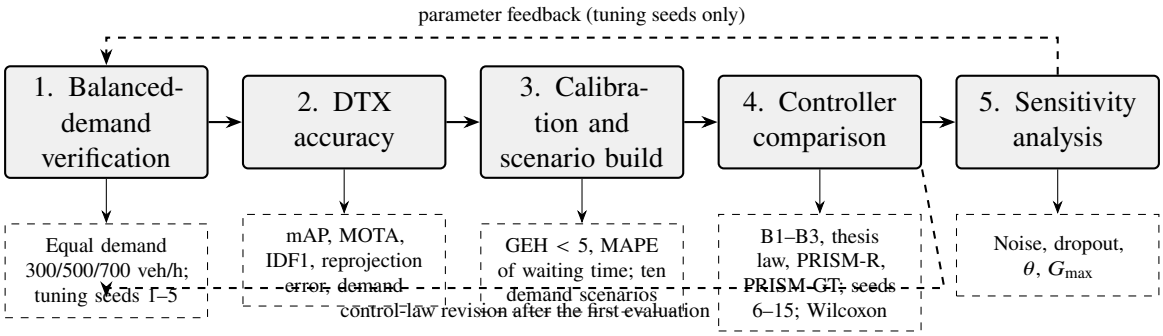


Figure 6.1: Testing and evaluation workflow, with the evidence produced at each stage and the one feedback path that revised the control law.

6.3.1 Balanced-demand verification

Before the drone data were processed, the closed loop was exercised on balanced demand: equal flow on all four approaches, turning shares of 25% left, 50% through and 25% right, and a fixed composition (Section 5.2.2). This stage supplied the parameter grid search on tuning seeds 1–5; Fig. 6.2 shows the held-out results for the three balanced scenarios. PRISM-R reduced waiting time against the observed 78 s plan at every level: by 26% (18.1 → 13.4 s) at 300 veh/h per approach, 25% (25.7 → 19.4 s) at 500 and 33% (84.4 → 56.5 s) at 700. Unlike the original law, whose gain fell away as demand rose, the revised law gains *most* at the highest balanced demand, because the hidden queue and the range-edge blend keep the state informative once queues exceed what the sensor can see.

The same figure records the finding that prompted the revision. The control law as first specified beat the fixed plan at 300 veh/h but lost at 500 (31.5 against 25.7 s) and collapsed at 700 (190.1 against 84.4 s). A controller that is worse than the plan it replaces, by a factor of more than two at the demand that matters most, is not a tuning problem, and the diagnosis of Section 5.2.4 followed from it.

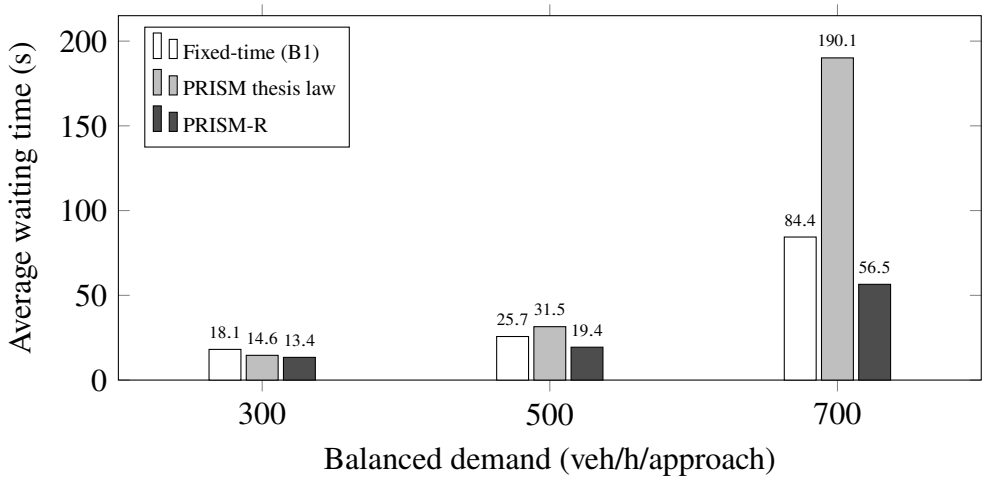


Figure 6.2: Balanced-demand verification: average waiting time per vehicle at three demand levels, for the existing plan, the control law as first specified and PRISM-R (evaluation seeds 6–15).

6.3.2 DTX accuracy

The fine-tuned YOLOv8s detector [23] with Slicing Aided Hyper Inference (SAHI) [24] was evaluated on the 15% test split of the 450 annotated frames. It reached a mean Average Precision (mAP) at 0.5 Intersection over Union (IoU) of 0.86 and an mAP@0.5:0.95 of 0.58 (Table 6.4). Large vehicles were detected almost perfectly; the weakest classes were bicycles, which are rare and very small at ≈90 m altitude, and motorcycles, which overlap one another while filtering between queued vehicles. The motorcycle Average Precision (AP) of 0.81 was judged adequate because a track is counted once it has been detected in several frames.

Table 6.4: Per-class detection performance of YOLOv8s with SAHI (test split).

	Moto	3W	Car	Van	Bus	Lorry	Bicycle
AP@0.5	0.81	0.88	0.93	0.84	0.95	0.87	0.62
All classes: mAP@0.5 = 0.86; mAP@0.5:0.95 = 0.58							

ByteTrack [25] was evaluated on a hand-labelled two-minute clip from the morning session with the Multiple Object Tracking (MOT) metrics of [76, 77]: Multiple Object Tracking Accuracy (MOTA) was 0.82, the identification F1 score (IDF1) 0.79, with 14 identity switches. Georeferencing achieved a Ground Sample Distance (GSD) of ≈2.6 cm/px and a reprojection error of 0.18 m on held-out control points.

Tables 6.5 and 6.6 summarise the extracted demand. The morning session carried 3,420 veh/h (3,150 PCU/h) and the afternoon 2,280 veh/h (2,085 PCU/h); the bypass carried about 70% of all traffic in both sessions. Motorcycles and three-wheelers together formed 62% and 61% of the two sessions, so treating them as cars, or using loops that miss them, would misrepresent the load. DTX also gave a saturation flow of 1850 PCU/h/lane, a free-flow

speed of 38 km/h and observed average waiting times under the existing plan of 104.7 s (morning) and 46.3 s (afternoon).

Table 6.5: Hourly demand and turning shares extracted by DTX (Bypass 1/2 and Cross 1/2 denote the two opposing approaches of each road; turning shares in PCU).

Session	Approach	veh/h	PCU/h	Left (slip)	Through	Right
Morning	Bypass 1	1,390	1,280	12%	75%	13%
	Bypass 2	1,010	930	13%	73%	14%
	Cross 1	640	590	17%	58%	25%
	Cross 2	380	350	17%	60%	23%
	Total	3,420	3,150			
Afternoon	Bypass 1	853	780	12%	73%	15%
	Bypass 2	722	660	12%	73%	15%
	Cross 1	514	470	13%	64%	23%
	Cross 2	191	175	20%	57%	23%
	Total	2,280	2,085			

Table 6.6: Vehicle composition extracted by DTX (% of vehicles).

Session	Moto	3W	Car	Van/SUV	Bus	Lorry	Bicycle
Morning (07:00–08:00)	38	24	21	7	6	3	1
Afternoon (13:30–14:30)	34	27	22	8	5	3	1

6.3.3 Calibration and validation

The SUMO model running the observed 78 s plan (B1) was calibrated against the DTX observations following the Federal Highway Administration microsimulation guidelines [94]. Counts were compared with the GEH statistic,

$$\mathrm{GEH} = \sqrt{\frac{2\,(M-C)^2}{M+C}}, \tag{6.1}$$

where M is the simulated and C the observed hourly count. Unlike a percentage error, GEH tolerates larger relative deviations on small flows, which suits movements of very different sizes. The conventional criterion is GEH < 5 for at least 85% of counts. All eight approach counts (100%) and 22 of the 24 turning movements (91.7%) met it (Table 6.7). The two failures were morning bypass right turns, which the sublane model under-served because through motorcycles blocked the right-turn bay more often than in reality. The parameters tuned were the driver imperfection σ, reaction time τ, `minGap`, `lcSublane` and the speed-factor distribution (final values in Appendix B).

Validation then used a measure not targeted during count calibration: the average waiting time. The simulated B1 value was 108.6 s (morning) and 47.9 s (afternoon), against observed

Table 6.7: Calibration: observed and simulated hourly approach volumes (veh/h) with GEH.

Session	Approach	Observed	Simulated	Difference	GEH
Morning	Bypass 1	1,390	1,362	−2.0%	0.75
	Bypass 2	1,010	1,027	+1.7%	0.53
	Cross 1	640	651	+1.7%	0.43
	Cross 2	380	374	−1.6%	0.31
Afternoon	Bypass 1	853	866	+1.5%	0.44
	Bypass 2	722	709	−1.8%	0.49
	Cross 1	514	507	−1.4%	0.31
	Cross 2	191	195	+2.1%	0.29

104.7 s and 46.3 s. At the finer level of 5-minute bins per approach, the Mean Absolute Percentage Error (MAPE) of average waiting time was 8.4% in the morning and 6.9% in the afternoon, within the range usually accepted for heterogeneous-traffic microsimulation [95]. This supports using the model as a test bed, although it cannot prove that the model responds to a *different* controller exactly as the real junction would.

The calibrated model and the DTX outputs together parameterise the scenario generator of Section 5.2.2: composition, turning shares, saturation flow and free-flow speed are taken directly from the footage, and the seven situation scenarios abstract the conditions under which the survey and the stakeholder discussion showed the fixed plan to fail. The controller comparison that follows is therefore run on demand derived from this junction, under a model whose response to the *existing* controller has been validated against it.

6.3.4 Controller comparison

(i) Definitions.

Four indicators used below need precise definitions, since none is standard.

- *Phase changes per hour*: the number of transitions into a *different* green during the measurement hour. Holding a green, whether by extension or by the rest-in-green rule, is not a change.
- *Manual-control trigger*: the share of consecutive 78 s windows in which any approach queue, measured from SUMO ground truth, exceeds 200 m. The window length is the existing cycle, so the indicator counts the cycles in which an officer would plausibly intervene.
- *Wasted green*: the number of seconds per hour during which the phase holding green has no vehicle within 100 m of its stop line *while* another phase has a stopped vehicle. It is computed from SUMO ground truth and is never visible to the controller, so it cannot be optimised for directly; it measures the first of the three failures of the fixed plan identified

in Section 1.3.2.

- *Wilcoxon*: the exact two-sided signed-rank test [98], paired by seed, with $n = 10$; the smallest attainable p-value is therefore 0.002.

Table 6.8 and Fig. 6.3 present the primary indicator across all ten scenarios. PRISM-R achieved the lowest waiting time of any deployable controller in nine of the ten, and was second only to the original law at night. Against the existing plan it improved every scenario, by 40% on average, with the largest gains where the fixed plan is structurally worst: −85% with an empty side road and −71% with unbalanced roads, both situations in which B1 spends green on an arm that has nothing to discharge. The smallest gain, −14%, is in the heavy-vehicle scenario, where occlusion is worst and where capacity rather than allocation binds.

The baselines behave very differently from one another. B3, Webster re-timed per scenario, is a strong baseline at balanced demand and is the only baseline to beat B1 at 700 veh/h, but it is *worse* than B1 in the two asymmetric scenarios, because a plan computed from mean demand allocates green to a side road that is empty for most of the hour. B2, the SUMO actuated controller with default detectors, is good at light demand but degrades sharply as demand rises, reaching 265.4 s at 700 veh/h against B1's 84.4 s. The cause is the one stated in Section 5.2.5: the right-turn phases have no detectors of their own, so they cannot gap out on their own demand, and at high demand every phase runs to its maximum while right-turn queues block the offside lanes.

Table 6.8: Average waiting time per vehicle (s), mean over evaluation seeds 6–15. Standard deviations are omitted for width; every PRISM-R improvement over B1 holds on all ten seeds individually.

Scenario	Fixed (B1)	Actuated (B2)	Webster (B3)	Thesis law	PRISM-R	PRISM-GT	vs B1
Balanced 300	18.1	14.1	13.8	14.6	**13.4**	12.9	−26%
Balanced 500	25.7	44.8	30.5	31.5	**19.4**	19.5	−25%
Balanced 700	84.4	265.4	77.6	190.1	**56.5**	53.1	−33%
Empty side road	50.2	103.9	107.0	8.5	**7.4**	6.9	−85%
Unbalanced roads	67.6	121.4	120.6	22.2	**19.6**	19.0	−71%
Peak	40.4	95.2	59.7	68.8	**30.9**	29.9	−24%
Platoon surge	51.6	58.2	50.2	48.7	**30.0**	31.0	−42%
Night	16.9	11.3	11.3	**6.6**	7.2	6.8	−57%
Heavy vehicles	28.3	86.9	24.9	63.0	**24.2**	21.9	−14%
Sensor failure	25.7	44.8	30.5	30.2	**20.9**	19.5	−19%
Mean change against B1							−40%

Table 6.9 shows that the secondary indicators move with the primary one. Mean queue length falls furthest where waiting falls furthest, from 16.3 to 1.1 m with an empty side road, and the maximum queue at 700 veh/h drops from 272 to 192 m, which is what takes that scenario below the 200 m manual-control threshold for most of the hour. Stops per vehicle fall by a third at 700 veh/h, and carbon dioxide (CO_2) emissions from the HBEFA

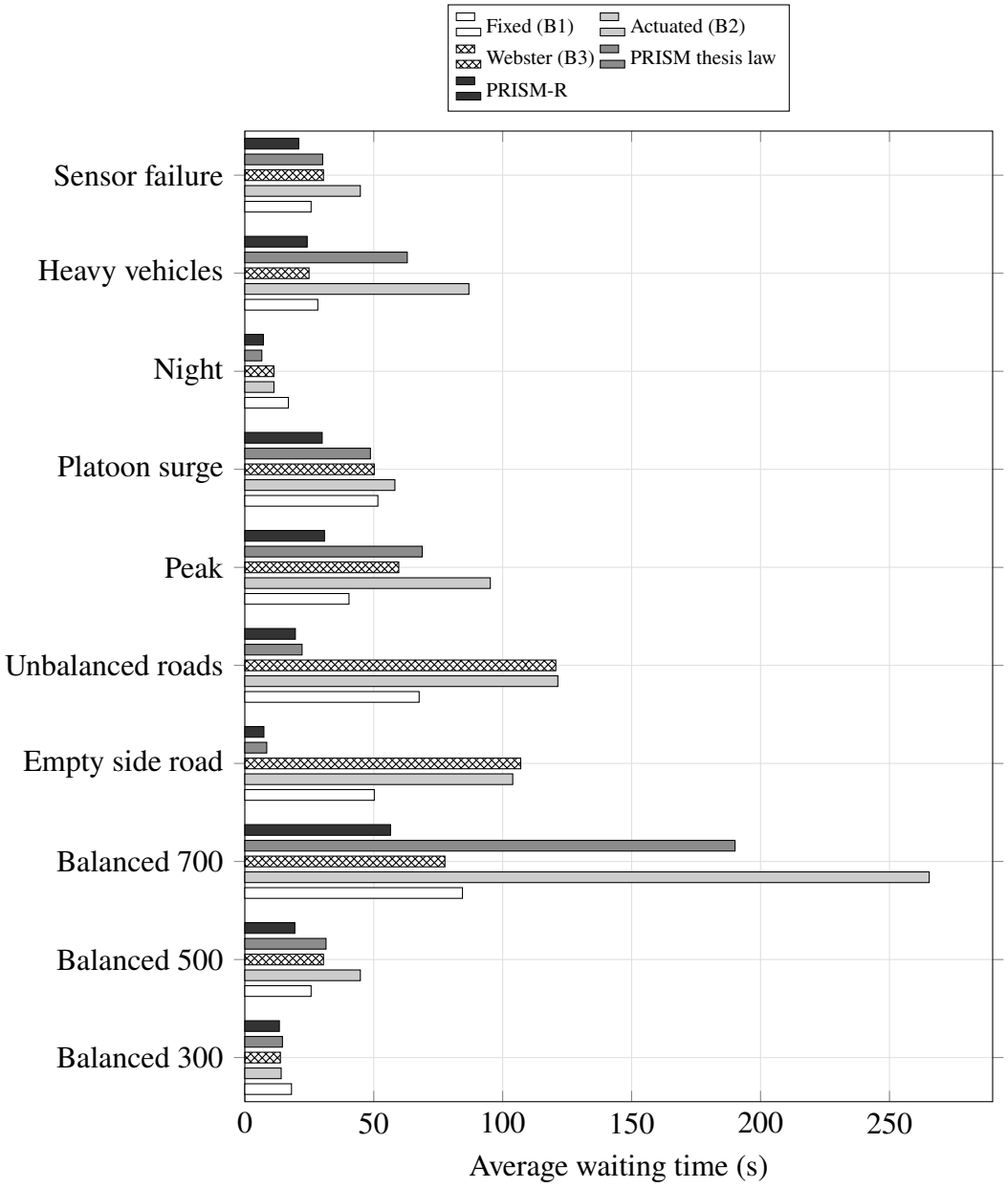


Figure 6.3: Average waiting time per vehicle by controller and scenario (mean over evaluation seeds 6–15).

model [112] fall with them, by 13% at 700 veh/h and 23% with an empty side road, through less idling and fewer stop–start cycles.

Table 6.9: Secondary KPIs, existing fixed-time plan (B1) → PRISM-R (mean over evaluation seeds 6–15).

Scenario	Travel time (s)	Mean queue (m)	Max queue (m)	Stops / veh	CO_2 (kg/h)
Balanced 500	101 → 93	9.1 → 6.2	71 → 56	1.00 → 0.95	356 → 343
Balanced 700	184 → 142	68.1 → 38.1	272 → 192	2.53 → 1.65	674 → 586
Empty side road	128 → 75	16.3 → 1.1	125 → 37	1.43 → 0.55	268 → 207

Because every controller was run on the same seeds, the per-seed waiting times form paired samples, and the exact two-sided Wilcoxon signed-rank test [98] was applied; it assumes no normality. Table 6.10 summarises the outcome. PRISM-R beat B1 in 10 of the 10 scenarios and, within each, on all 10 seeds, so every comparison attains the minimum p-value of 0.002. It beat B3 in 10 of 10 and the original law in 9 of 10. The single exception is the night scenario, discussed below. The oracle PRISM-GT was better than PRISM-R by between 0.4 and 3.4 s, and in two scenarios (balanced 500 and the platoon surge) PRISM-R was marginally the better of the two, which is within seed-to-seed variation; the cost of realistic sensing is therefore small but not zero.

Table 6.10: Exact Wilcoxon signed-rank tests on paired per-seed average waiting times ($n = 10$ seeds per scenario, two-sided).

Comparison	Scenarios won by PRISM-R	Seeds favouring PRISM-R	p per scenario
PRISM-R vs Fixed (B1)	10/10	10/10	0.002
PRISM-R vs Webster (B3)	10/10	10/10	0.002
PRISM-R vs Actuated (B2)	9/10	10/10 except night	0.002
PRISM-R vs PRISM thesis law	9/10	10/10 except night	0.002

Two further indicators address stakeholder needs (Section 4.2), and are reported in Table 6.11. The manual-control trigger fires in four scenarios under B1 and is reduced in all four by PRISM-R: most importantly at 700 veh/h, from 24% of windows to 5%, and in the platoon surge from 28% to 14%. The original law is far worse than either, at 70% of windows at 700 veh/h, which is a direct consequence of its 11 s greens. Phase changes per hour explain how the gains are obtained: B1 makes 184 changes an hour regardless of demand, whereas PRISM-R makes 153 at 700 veh/h, where long greens reduce the share of the hour lost to intergreens, and 252 in the heavy-vehicle scenario, where it switches often between short, slow-discharging queues.

Figure 6.4 reports the wasted-green indicator, which isolates the specific inefficiency this research set out to remove. Averaged over the ten scenarios, the existing plan shows green to an empty approach, while another approach holds a stopped vehicle, for 648 s of every hour: more than a sixth of the hour. PRISM-R reduces this to 328 s, a fall of 49%. The

Table 6.11: Stakeholder-oriented indicators: manual-control trigger (share of 78 s windows with any approach queue over 200 m) and phase changes per hour. Scenarios not listed trigger in 0% of windows under both controllers.

	Manual-control trigger			Phase changes / h
Scenario	Fixed (B1)	Thesis law	PRISM-R	B1 → PRISM-R
Balanced 700	24%	70%	5%	184 → 153
Platoon surge	28%	—	14%	184 → 188
Unbalanced roads	6%	—	0%	184 → 171
Empty side road	2%	—	0%	184 → 164
Heavy vehicles	0%	—	0%	184 → 252

residual is not waste that better control could remove: part of it is the minimum green, which the controller is required to serve once a phase starts, and part is the clearance of a queue that empties part-way through its green.

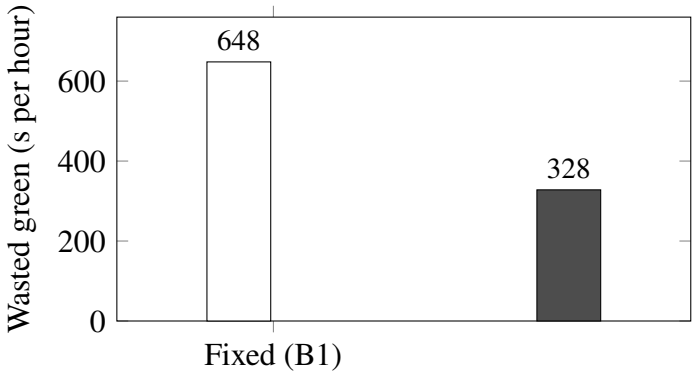


Figure 6.4: Wasted green, averaged over the ten scenarios: seconds per hour in which the green phase has no vehicle within 100 m of its stop line while another phase has a stopped vehicle.

Finally, the sensor-failure scenario tests the fail-safe in closed loop rather than by injection. In every one of the ten seeds the controller detected the outage at the fifth missing scan, at $t = 1204$ s and $t = 2404$ s, and handed over to the 78 s plan through a full amber and all-red. Adaptive control resumed at the first B1 cycle boundary after ten consecutive good scans, at $t = 1519$–1522 s and 2485–2488 s depending on seed. The cost of the two outages, which together remove the sensor for 6% of the hour, is the difference between the sensor-failure row (20.9 s) and the balanced 500 row (19.4 s) of Table 6.8, that is, 1.5 s per vehicle; the controller remains 19% better than B1 throughout.

6.3.5 Sensitivity analysis

Four one-at-a-time sweeps were run on PRISM-R in the balanced 500 veh/h scenario over the held-out seeds 6–15: the base position-noise term σ_0 (0.10 m in the reference model), a uniform dropout probability replacing Eq. (5.7), the hysteresis θ and the through-phase

maximum green $G_{\max}$ (Fig. 6.5). The headline result is that the controller is insensitive to all four. The mean waiting time stays between 19.0 and 20.8 s across every value tested, including a tenfold increase of noise to $\sigma_0 = 1.0$ m and a dropout of 0.3, six times the worst dropout of the reference model. The fixed plan gives 25.7 s in the same scenario, so the entire swept range lies 19% or more below B1, and no value of any parameter brings PRISM-R near the baseline.

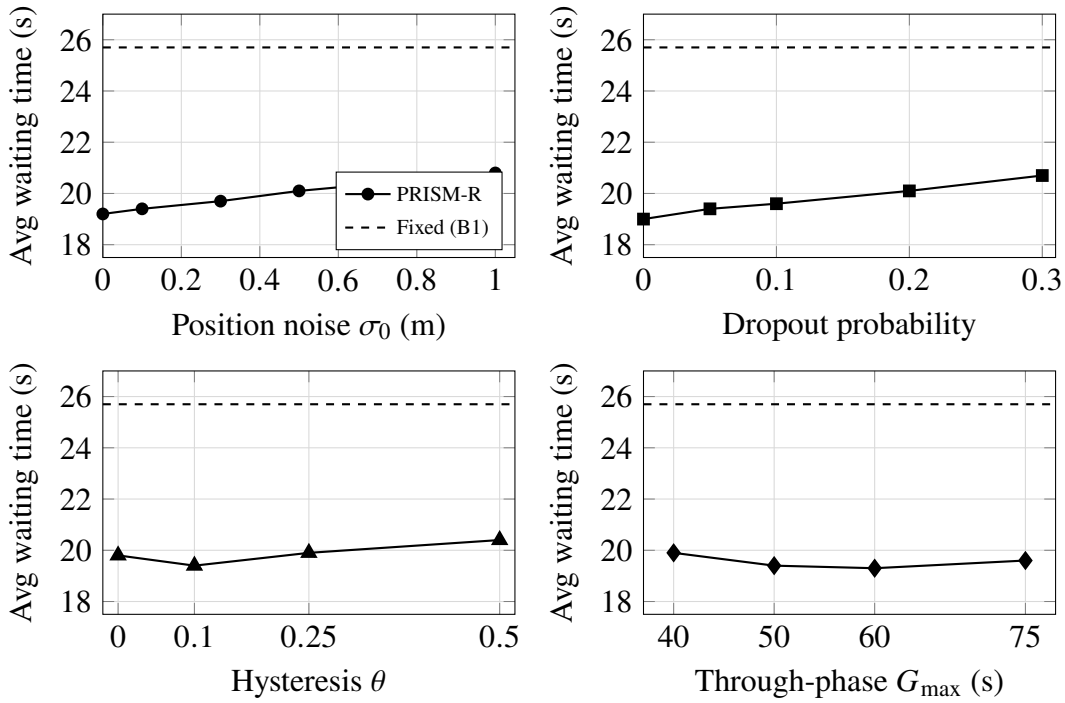


Figure 6.5: Sensitivity of PRISM-R average waiting time (balanced 500 veh/h, evaluation seeds 6–15) to position noise, detection dropout, the hysteresis θ and the through-phase maximum green. The dashed line is the fixed-time plan at 25.7 s; the whole swept range lies between 19.0 and 20.8 s.

Two mechanisms explain this flatness. On the sensing side, pressure aggregates tens of vehicles, so independent per-object errors largely cancel, and the tracker bridges short dropouts without reporting coasting tracks that would otherwise inflate the queue. On the control side, the revisions of PRISM-R removed the two parameters the original law was most sensitive to. With $\rho = 1.0$ and a frozen G_{req}, a green cannot be cut short before its target regardless of θ, so the hysteresis no longer carries the whole burden of preventing oscillation; that is why $\theta = 0$ now costs only 0.4 s, where in the original law it produced sustained oscillation. The responsibility for stability has moved from a tuning constant to a structural rule, which is the more defensible place for it.

6.3.6 Discussion

Why the original law lost, and what fixed it. The most instructive result in this chapter is the failure of the law as first specified, because each of its five causes maps to a revision and the

revisions can be read off against the scenarios they repair. The map-matching gap and the occlusion persistence both corrupted the *state*: the controller optimised well against a queue that was substantially invisible. The recomputed G_{req} corrupted the *timing*, producing 11 s greens and about 250 phase changes an hour, which at 5 s of intergreen each spent roughly 21 minutes of the hour clearing a junction that nobody was crossing; this alone accounts for most of the 190.1 s at 700 veh/h. The 60 m horizon corrupted the state again, but only under saturation, which is why the original law was adequate at 300 veh/h and catastrophic at 700. Acyclic arg-max corrupted the *sequence*, starving right turns until the 120 s override fired. The pattern worth drawing out is that four of the five were modelling errors in what the controller believed, not errors in the control law itself; the pressure formulation of Eq. (5.13) survives the revision unchanged. A controller is only as good as its state estimate, and this is the practical form that the ground-truth fallacy identified in Section 2.5.3 takes when one actually declines to use ground truth.

Where PRISM-R helps most. The gains are largest exactly where the fixed plan's failure is one of allocation rather than capacity. With an empty side road (−85%) and unbalanced roads (−71%) the existing plan spends a fixed share of every cycle on an arm with nothing to discharge, and skipping that arm is nearly free; the rest-in-green and cyclic-with-skip rules do most of the work. The platoon surge (−42%) is the clearest case for the pressure term itself, since no fixed plan can anticipate 1,500 veh/h appearing on one arm for five minutes. At 700 veh/h (−33%) the mechanism is different again: there the junction is genuinely near capacity, and the gain comes from the hidden queue and the range-edge blend keeping greens long enough to discharge queues the sensor cannot fully see, plus the fall in phase changes from 184 to 153. The smallest gain, heavy vehicles (−14%), is where capacity binds hardest and where occlusion is worst, and it is also the one scenario in which the oracle is meaningfully better than PRISM-R (21.9 against 24.2 s), which confirms occlusion rather than control as the limiting factor.

The night result. PRISM-R is the only controller that does not win its scenario outright: at night the original law achieves 6.6 s against PRISM-R's 7.2 s. The cause is the rest-in-red rule. At 80 veh/h the junction is empty most of the time, so PRISM-R ends the green and rests in all-red, and the next vehicle to arrive must wait the 2 s clearance before receiving green. The original law instead leaves the last green standing, so a vehicle arriving on that arm is served immediately and pays nothing. Rest in red is therefore a trade: it is better for a vehicle arriving on any *other* arm, which is the common case across four approaches, and worse for one arriving on the arm that happened to hold the green. At this demand the two effects nearly cancel and the residual favours resting in green. The honest reading is that rest in red costs about 0.6 s per vehicle at night and buys a guarantee that no approach is ever the one that waits; the alternative, a demand-dependent choice between the two resting rules, is left as a refinement. It is worth noting that both are far better than the fixed plan's

16.9 s, and that the night case is also where a shorter minimum green would help most.

The right-turn trade-off. Cyclic selection guarantees the protected right turns a place in the order whenever they have demand, which is what TC01 and TC05 verify and what removes the starvation seen under acyclic arg-max. The cost is that a right-turn phase is sometimes served when the through phase beside it could have used the time better: at balanced 300 veh/h, where right-turn demand is thin, PRISM-R makes more phase changes than strictly optimal and its margin over Webster is correspondingly narrow (13.4 against 13.8 s). This is the trade that Levin, Hu and Odell [58] describe, and the skip rule bounds it: a right-turn phase with no demand costs nothing at all.

Why B2 performs so poorly. The actuated baseline is the weakest controller in six of the ten scenarios and is worse than doing nothing at 700 veh/h. As Section 5.2.5 explains, SUMO's default detector generation gives the protected right-turn phases no detectors, so they always run to `minDur` and never extend on their own demand. Right-turn queues then grow into the offside lane and obstruct the through movement that shares it. This should be read as a limitation of the baseline configuration rather than as evidence that actuated control is unsuitable; a hand-detected B2 would be stronger. It is reported because concealing it would overstate PRISM-R's advantage.

Cost of sensing and motorcycle filtering. The gap between PRISM-R and PRISM-GT is between 0.4 and 3.4 s, and is the price of a realistic LiDAR instead of perfect knowledge. Most residual error came from bus occlusion and from motorcycles filtering to the stop line; dense motorcycle clusters were occasionally merged or split by the size classes, yet the PCU-weighted queue still tracked the true load.

Answers to the sub-research questions. For SRQ1, DTX extracted demand, composition, turning shares, saturation flow and waiting time with 0.86 mAP@0.5 and 0.82 MOTA; the resulting model met GEH and MAPE criteria, and the ten demand scenarios derived from it reproduce the situations in which the fixed plan is inefficient. For SRQ2, a PCU-weighted pressure law with an urgency term handled heterogeneity, fairness and noisy sensing while passing every safety test, once its state estimator was corrected to account for what the sensor cannot see. For SRQ3, PRISM-R reduced waiting time against B1 in 10 of 10 scenarios and on every seed, by 40% on average, beat Webster in 10 of 10 and actuated control in 9 of 10, and stayed within 1.8 s of its reference performance across every sensor and parameter value swept.

Threats to validity. Four limitations qualify these results. First, the demand is a set of ten scenarios parameterised from two hours of observation, not a continuous record of the junction's year; they span the operating conditions that were seen, but weekends, rain and school holidays were not. Second, a single junction was studied, so the coordination effects that motivate the long-term vision of Section 7.6 are entirely absent, and a controller that

is locally optimal can be globally worse. Third, the LiDAR was represented at the object level by a statistical error model rather than as a physical point cloud, so failure modes such as rain backscatter, ground-plane drift and clustering errors on dense motorcycle groups are not represented; the sensitivity sweeps bound the effect of position noise and dropout but cannot bound a failure mode the model does not contain. Fourth, the controller parameters were tuned on the three balanced scenarios and five seeds only, so their transfer to another junction is an assumption rather than a finding, although the flatness of Fig. 6.5 suggests the result does not hinge on fine tuning.

6.4 Review of test strategies used

The layered strategy proved its value. Hand-derived oracles caught boundary-comparison errors in the pressure-switch rule early, and the randomised TC01 gave far stronger evidence than scripted sequences. Testing the VLS statistically rather than against fixed values was essential, because the sensor is stochastic by design. The balanced-demand stage allowed development while flight permissions were pending. Separating tuning seeds 1–5 from evaluation seeds 6–15 guarded against over-fitting.

It also exposed a gap that is worth naming. Every component passed its own tests, and the system still lost to the fixed plan, because the tests verified that each rule fired when its stated condition held but never asked whether the state those conditions were evaluated against was correct. The map-matching defect, for instance, was invisible to TC23, which measured the share of *reported* objects matched correctly rather than the share of *actual* vehicles reported. The remedy adopted during the revision was to add scenario-level assertions comparing the controller's view of each approach against SUMO ground truth, which is the test that would have caught four of the five diagnosed causes in a single run. Component tests establish that the code does what it was told; only a comparison against ground truth establishes that it was told the right thing.

The strategy also had clear limits. No field test was possible, so all controller evidence is simulation-based. Tracking accuracy was measured on a single hand-labelled two-minute clip. The detector test split contained only about seventy frames, and the same two sessions served both calibration and validation, so no independent day was available for validation. No hardware-in-the-loop test with a real signal cabinet was performed, and stakeholder acceptance was gauged only through discussion rather than a formal usability study. These limitations do not invalidate the comparative findings, which use the same model for every controller, but a real pilot would need to address them.

6.5 Chapter summary

This chapter verified the system through 62 unit tests, ten principal and twenty-three supplementary test cases, closed-loop safety monitoring over 370 simulated hours and fault injection. DTX reached 0.86 mAP@0.5 and 0.82 MOTA, and the calibrated model met GEH and MAPE criteria. The first controller comparison showed the control law as originally specified losing to the fixed plan at 500 and 700 veh/h; five modelling errors were diagnosed and corrected, and the revised law PRISM-R was evaluated against the same held-out seeds alongside the original as an ablation. PRISM-R reduced average waiting time against the existing plan in all 10 scenarios and on all 10 seeds of each (exact Wilcoxon $p = 0.002$), by 40% on average, ranging from −14% with heavy vehicles to −85% with an empty side road. It beat Webster in 10 of 10 scenarios and the actuated baseline in 9 of 10, cut wasted green from 648 to 328 s per hour, lowered the manual-control trigger at 700 veh/h from 24% to 5%, recovered cleanly from two injected sensor outages, and held between 19.0 and 20.8 s across every sensor and parameter value swept, against 25.7 s for the fixed plan. A decision costs 0.25 ms at 400 objects. Chapter 7 uses these results to assess the objectives.

7 CONCLUDING REMARKS

7.1 Chapter overview

This chapter triangulates the seven objectives against the evidence and answers the research question. It then describes the problems encountered, reflects on the research, considers how the Pressure-based Real-time Intersection Signal Management (PRISM) controller could reach the road, and closes with future recommendations.

7.2 Accomplishment of research objectives

Table 7.1 maps each objective defined in Section 1.7 to the work that addressed it and to its evidence; each is supported by both a design artefact and empirical results.

The research question asked how a Light Detection and Ranging (LiDAR)-driven, Passenger Car Unit (PCU)-weighted, pressure-based adaptive controller, calibrated with drone-observed heterogeneous traffic, could reduce the average waiting time of road users at the Maviththa junction compared with the existing fixed-time plan. It does so by giving green to the phase with the greatest PCU-weighted pressure rather than following a rigid cycle, while an urgency term, a starvation cap and a cyclic phase order keep every approach served. In the calibrated Simulation of Urban MObility (SUMO) model, PRISM-R reduced the average waiting time per vehicle against the existing plan in all 10 demand scenarios and on every one of the 10 held-out seeds of each, so every comparison attains the smallest attainable exact Wilcoxon p-value of 0.002. The mean reduction is 40%, ranging from -14% in the heavy-vehicle scenario, where capacity rather than allocation binds, to -85% with an empty side road, where the fixed plan's failure is purely one of allocation. Of particular relevance to the Traffic Police, at 700 veh/h the share of 78-second windows in which some approach queue exceeded 200 m, the condition under which officers typically take manual control, fell from 24% to 5%; and the green shown to empty approaches while another approach held a stopped vehicle fell from 648 to 328 s per hour. A decision costs about 0.25 ms, three orders of magnitude inside the control interval.

Two qualifications belong with that answer. First, the smaller gains at high demand confirm that adaptive control redistributes existing capacity but cannot create it; near saturation its benefit is real but bounded. Second, and more instructive, the control law as first specified did *not* achieve this. It lost to the fixed plan at 500 and 700 veh/h, by more than a factor of two at the latter, and the revision that produced PRISM-R corrected five errors of which four lay in the controller's estimate of the junction's state rather than in the control law itself. The pressure formulation survived unchanged. The finding the author regards as

Table 7.1: Triangulation of the research objectives against the evidence in this thesis

Objective	How it was achieved	Evidence	Status
O1 – Identify junction problems and stakeholder needs	Site observation, derivation of the 78 s fixed-time plan, discussion with the Piliyandala Traffic Police and the RDA on 17.09.2026	Section 1.3, Section 4.2, Section 4.3	Achieved
O2 – Critically analyse existing systems, algorithms, sensing and simulation tools	Comparative review of fixed-time, actuated, SCOOT, SCATS, max-pressure and learning-based control, sensing options and simulators; gap stated	Section 2.4, Section 2.5.1, Section 2.5.2, Section 2.6	Achieved
O3 – Design and develop the drone trajectory extractor (DTX)	Stabilisation, YOLOv8s with SAHI, ByteTrack, georeferencing, waiting-time and flow extraction; mAP@0.5 of 0.86, MOTA 0.82	Section 5.2.1, Section 6.3.2	Achieved
O4 – Develop, verify and calibrate the SUMO model	Balanced-demand verification, calibration with drone data (GEH < 5 for 100% of approaches and 91.7% of movements), and a generator producing ten demand scenarios representing the junction's operating conditions	Section 5.2.2, Section 6.3.1, Section 6.3.3	Achieved
O5 – Design and implement the Virtual LiDAR Sensor (VLS)	Range-dependent noise and dropout, ray-cast occlusion, size-class confusion, clutter and an α–β tracker	Section 5.2.3, Section 6.3.5	Achieved (simulated)
O6 – Design and implement PRISM using mathematics only, and revise it after evaluation	PCU-weighted pressure, urgency term, gap-out, hysteresis, starvation cap, spillback guard and fail-safe; five modelling errors diagnosed after the first evaluation and corrected in PRISM-R (hidden queue, frozen green target, range-edge blend, occlusion persistence, resting rules, cyclic selection); safety tests passed	Section 5.2.4, Algorithm 5.3, Section 6.2.2	Achieved
O7 – Evaluate PRISM against B1–B3, PRISM-GT and the original law	Ten demand scenarios $\times$ ten held-out seeds, exact Wilcoxon tests, the wasted-green indicator, the cost of a decision, and sensor and parameter sensitivity	Section 5.2.5, Section 6.3.4, Section 6.3.5	Achieved

most transferable is therefore not the percentage but the diagnosis behind it: a controller that declines to use the simulator's ground truth must be verified against that ground truth anyway, or it will optimise confidently against a state that is wrong.

7.3 Problems encountered

The main problems and their resolutions are summarised below.

Tiny motorcycles in aerial video. At the flight altitude of about 90 m, a motorcycle occupied only around 20 pixels, and a detector run on the downscaled full frame missed many of them. This was resolved with Slicing Aided Hyper Inference (SAHI) [24], which ran the detector on overlapping 2×2 tiles so that small objects were seen at close to native resolution.

Drone hover drift. Even in calm air, the DJI Mini 2 drifted and rotated slightly, so fixed pixel zones no longer matched the road. Every frame was therefore registered to a reference frame using Oriented FAST and Rotated BRIEF (ORB) features and a Random Sample Consensus (RANSAC) homography [107, 108], which removed the drift before detection.

Three-wheelers absent from standard datasets. The Common Objects in Context (COCO) classes [121] contain no three-wheeler, the most characteristic vehicle on Sri Lankan roads. The detector was therefore initialised from VisDrone weights, whose tricycle and awning-tricycle classes [69] resemble a three-wheeler from above, and fine-tuned on 450 locally annotated frames.

Occlusion by buses. Tall buses and lorries hid nearby vehicles in both the drone view and the simulated LiDAR. The tracker's 30-frame buffer bridged short gaps in the video, and the VLS modelled occlusion explicitly so that the controller faced the same problem.

Lost evening session. The 17:00–18:00 window was approved but could not be flown because of bad weather. The morning and afternoon sessions still spanned high and moderate congestion; the missing evening peak was recorded as a limitation.

Lane-less filtering in SUMO. Motorcycles and three-wheelers filtering between cars to the stop line cannot be represented by a strictly lane-based model. The sublane model [84] with a lateral resolution of 0.8 m and a small lateral gap for two-wheelers reproduced this behaviour adequately for calibration.

Phase oscillation. With no switching margin ($\theta = 0$), PRISM switched back and forth between phases of similar pressure, wasting time in intergreens. A hysteresis margin was introduced to stop it. The revision later showed that hysteresis had been carrying a burden that belonged elsewhere: once the required green was frozen at the start of each green and ρ

raised to 1.0, a green could not be cut short before its target in any case, and the sensitivity analysis found $\theta = 0$ costs only 0.4 s. The margin was reduced to 0.10, and responsibility for stability moved from a tuning constant to a structural rule.

A controller that lost to the plan it replaced. The most serious problem was that the control law, as specified and implemented, performed worse than the existing fixed-time plan at 500 and 700 veh/h. The temptation was to treat this as a tuning failure and sweep the parameters. Instrumenting the decision log instead revealed five distinct causes (Section 5.2.4), four of which were errors in the controller's estimate of the junction's state: vehicles nearest the stop line matched no approach because of an intermediate node in the OpenStreetMap geometry; the required green was recomputed from the queue it was discharging, so greens ended after about 11 s; queues longer than the sensor range reported a constant value; and vehicles hidden behind buses left the queue estimate entirely. Each was corrected as a parameter rather than a rewrite, so the original law could still be run as an ablation and the effect of the revisions measured rather than asserted.

Permission lead time. Flying a drone over a public junction required coordination between NSBM, the Piliyandala Police, the Ministry of Defence (MoD) and the Civil Aviation Authority of Sri Lanka (CAASL). The process took from 12.06.2026 to 01.10.2026, so the mock-data phase was deliberately used to develop and test the whole system while approvals were pending.

7.4 Self-reflection

7.4.1 Ideology about the research

The author began with a simple conviction: waiting at a red light while the crossing road is empty takes people's time for no reason. This view became more nuanced; near capacity, waiting cannot be eliminated, only shared more fairly and intelligently. The author also came to believe that a controller on public roads must be understandable by the engineers and police officers accountable for it, which is why the control law was kept purely mathematical even though learning-based controllers were attractive for research.

7.4.2 Benefits gained

The project gave the author practical experience in computer vision, tracking, traffic engineering, simulation, control design and statistics. Equally valuable were the non-technical lessons: writing formal requests to public institutions, explaining technical ideas to police officers and engineers, and working with a licensed drone operator within aviation rules. The discussion with the Traffic Police and the Road Development Authority (RDA) changed the author's view of the problem, because it showed that the officers' own time is one of the costs of poor signal control.

7.4.3 Learning curves

The steepest learning curve was SUMO itself, particularly the network editing, the sublane model and calibration against observed counts, which required many iterations before the Geoffrey E. Havers (GEH) statistic was acceptable. Fine-tuning a detector on a small, hand-annotated dataset taught the author how much data quality matters compared with model choice. Finally, the author learned to treat negative results, such as the oscillation at $\theta = 0$, as useful findings rather than failures, and to plan a project so that slow external processes do not block progress.

7.5 Business insight

7.5.1 Real-world application

(i) Deployment path.

The realistic route is a staged pilot led by the RDA, which owns the signals, with the Piliyandala Traffic Police as operational partner. A LiDAR on an existing pole would first feed PRISM in shadow mode, computing decisions without actuating the lights; closed-loop operation would follow once safety was verified on site, with the fixed-time plan as fail-safe.

(ii) Indicative cost comparison.

Table 7.2 gives an order-of-magnitude comparison in Sri Lankan Rupees (LKR) between equipment cost and the value of time saved; all figures are indicative estimates, not quotations. It shows that the reduction in officer-hours alone, roughly 1,400 hours a year, would not by itself justify the capital cost; the economic case rests mainly on the time saved by road users, which gives a simple payback of under a year. Officers released from manual control would in practice be redeployed to enforcement and road safety rather than producing a cash saving.

(iii) Smart-city integration and maintenance.

Because PRISM consumes only a sensor-agnostic object list, the same controller could later receive data from cameras or radar, and its decision log could feed a city traffic-management centre. Maintenance would mainly involve cleaning the sensor, recalibrating approach polygons after road works and software updates.

(iv) Regulatory considerations.

A live deployment would need RDA approval for any change to signal operation, a formal procedure by which police can override the controller, and a data-protection policy. LiDAR helps here, because point clouds do not capture faces or number plates, which simplifies compliance compared with camera-based systems.

Table 7.2: Indicative annual cost–benefit estimate for one junction (LKR)

Item	Assumption	Estimate
32-channel LiDAR [22]	One unit, imported, including duties	4.5 million
Edge computer and enclosure	Industrial embedded GPU module in the signal cabinet	0.6 million
Installation and integration	Pole mounting, power, network, controller interface	1.4 million
Capital cost		**6.5 million**
Annual maintenance	10% of capital per year	0.65 million
Officer-hours on manual control	2 officers × 3 h/day × 300 days = 1,800 h at LKR 1,200/h	2.16 million
Officer-hours saved	Proportional to the fall in the manual-control trigger at peak demand (24% to 5%)	1.7 million
Road-user time saved	Waiting saving of 27.9 s per vehicle in 2 peak-type hours and 6.3 s in 4 off-peak-type hours; 2,800 and 2,000 veh/h; 1.5 persons/vehicle; 300 days; LKR 250 per person-hour	6.5 million
Net annual benefit	Savings minus maintenance	**7.6 million**

7.6 Future recommendations

The long-term vision is a city in which every signalised junction has a LiDAR and a PRISM controller, and junctions, a cloud service and commuters' phones cooperate so that a commuter reaches the destination in minimum time (Fig. 7.1). The steps towards it are listed below.

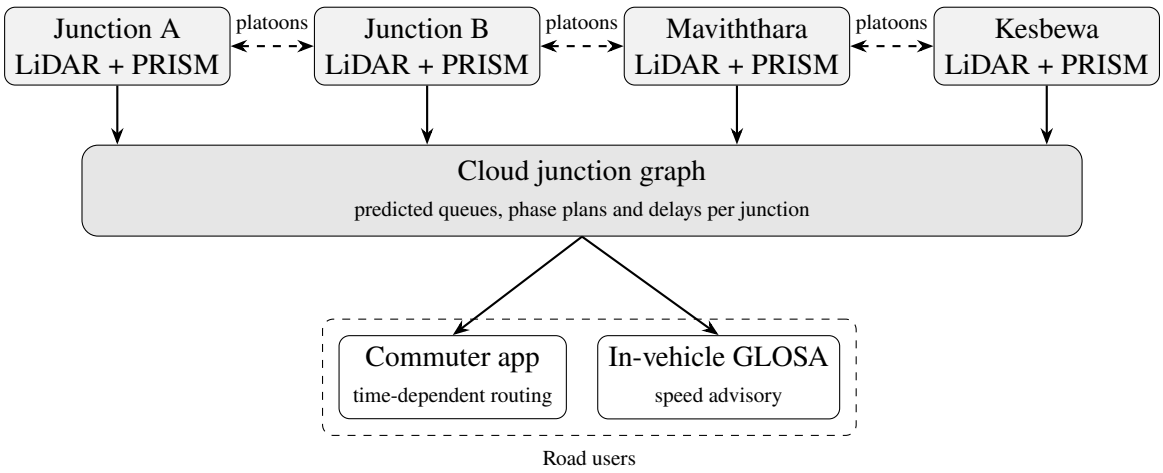


Figure 7.1: Envisaged city-scale architecture: LiDAR-equipped PRISM junctions share platoon forecasts and publish their state to a cloud junction graph that serves commuter routing and speed advice

1. **City-scale coordination.** Neighbouring PRISM controllers should exchange the pla-

toons they release, so that each junction can extend its arrival term beyond the 75 m sensing range. Decentralised max-pressure control maximises network throughput using only local information [20, 53], and recent variants add corridor coordination without losing this property [42], making a corridor along the Piliyandala bypass a natural next step.

2. **Cloud junction graph and commuter application.** Each junction would publish its predicted queues and delays to a cloud graph with time-dependent edge costs. A commuter application could then compute the minimum-time route with a time-dependent shortest-path search based on Dijkstra's algorithm [122] or A* [123], including predicted signal delays, and offer Green Light Optimised Speed Advisory (GLOSA) [124] so that drivers arrive on green.

3. **Emergency-vehicle pre-emption.** The Traffic Police specifically asked for this. Emergency vehicles detected by the LiDAR or by a transponder would cause their approach to receive green as soon as the current intergreen allows; network-level schemes that combine routing with pre-emption [125] indicate the potential gain.

4. **Real LiDAR pilot with police override.** A physical pilot at Mavithhara with the RDA and the Police, first in shadow mode, should validate the simulation results, together with a formal interface through which officers can override the controller without switching the lights off.

5. **Pedestrian and bus priority.** The pedestrian push-button minimum green of 15 s already exists as a parameter and should be implemented, and bus priority can be added within the max-pressure framework without losing its stability properties [126].

6. **Edge deployment.** PRISM and the perception tracker should be ported to an embedded platform such as an NVIDIA Jetson module in the signal cabinet, following recent edge-deployed adaptive signal systems [52].

7. **More observation data and more junctions.** Further drone sessions covering the missed evening peak, weekends, rain and school hours would extend the scenario set beyond the conditions observed on a single weekday, and the study should be extended to the Kesbewa intersections named in the original permission request. The recorded footage of 2 October 2026 also supports further extraction that this project did not exhaust, in particular per-class waiting-time distributions and lane-level trajectories through the junction box.

8. **Learn the turning shares online.** The offside-lane split of Eq. (5.12) currently uses turning shares fixed from the survey, which is the one place where the controller depends on prior observation rather than on what the sensor reports. A LiDAR observes each vehicle's exit as well as its approach, so the shares could be estimated online as an

exponentially weighted average, exactly as the hidden-queue arrival rate already is. This would remove the last calibration dependency and let the controller follow a change in turning behaviour, such as a diversion upstream.

9. **A shorter minimum green at night.** The night result of Section 6.3.4 is the one scenario in which PRISM-R does not win outright, and both the 0.6 s gap and the residual waiting are dominated by the 10 s minimum through green, which at 80 veh/h is far longer than the queue needs. A demand-dependent minimum green, or a demand-dependent choice between resting in green and resting in red, should recover it.

7.7 Chapter summary

This chapter showed that all seven objectives were achieved and answered the research question: in a drone-calibrated simulation, a transparent, mathematics-only controller fed by a simulated roadside LiDAR reduced average waiting time at the Mavithara junction in all 10 demand scenarios and on every held-out seed, by 40% on average and by between 14% and 85%, halved the green shown to empty approaches, and made the conditions that force police to take manual control substantially less frequent. It also showed that reaching this result required one honest revision, after the control law as first specified lost to the plan it was meant to replace. The problems encountered, the author's reflections and an indicative business case were presented, followed by a roadmap from a single intelligent junction to a coordinated, commuter-aware network. The results do not replace a physical pilot, but they provide a clear, quantified case for one, and a practical starting point for smarter signal control on Sri Lankan roads.

REFERENCES

[1] Japan International Cooperation Agency (JICA), "Urban transport system development project for Colombo metropolitan region and suburbs (CoMTrans): Final report," JICA and Ministry of Transport, Sri Lanka, Colombo, Sri Lanka, Tech. Rep., 2014.

[2] R. Dammulla and R. Mudunkotuwa, "Analysis on the road traffic congestion in Colombo metropolitan area," *CINEC Academic Journal*, vol. 5, pp. 15–22, 2022.

[3] S. A. C. S. Jayasooriya and Y. M. M. S. Bandara, "Measuring the economic costs of traffic congestion," in *Proc. Moratuwa Engineering Research Conf. (MERCon)*, Moratuwa, Sri Lanka, 2017, pp. 141–146.

[4] A. S. Kumarage, "Guidelines for improving transport policy effectiveness in developing countries: A case study of Sri Lanka," *Journal of South Asian Logistics and Transport*, vol. 2, no. 2, pp. 99–120, 2022.

[5] News 1st, "5.2 million motorcycles on Sri Lankan roads, but at what cost?" https://newsfirst.lk/2026/08/13/5-2-million-motorcycles-on-sri-lankan-roads-but-at-what-cost, Aug. 2026, accessed: Oct. 2026.

[6] Daily Mirror, "Motorcycle registrations dominate as vehicle numbers surge in 2025," https://www.dailymirror.lk/print/news/Motorcycle-registrations-dominate-as-vehicle-numbers-surge-in-2025/239-329581, Jan. 2026, accessed: Oct. 2026.

[7] Department of Motor Traffic, Sri Lanka, "Vehicle registration statistics," https://dmt.gov.lk, accessed: Oct. 2026.

[8] Indian Roads Congress, "Guidelines for capacity of urban roads in plain areas," Indian Roads Congress, New Delhi, India, Tech. Rep. IRC:106-1990, 1990.

[9] Transportation Research Board, *Highway Capacity Manual: A Guide for Multimodal Mobility Analysis*, 6th ed. Washington, DC, USA: National Academies of Sciences, Engineering, and Medicine, 2016.

[10] K. V. D. Perera and W. R. S. S. Dharmarathna, "Evaluating passenger car equivalent (PCE) factors and traffic flow characteristics on Sri Lankan expressways," *Engineer: Journal of the Institution of Engineers, Sri Lanka*, vol. 58, no. 4, pp. 1–12, 2025.

[11] F. V. Webster, “Traffic signal settings,” Road Research Laboratory, HMSO, London, U.K., Road Research Technical Paper 39, 1958.

[12] M. Papageorgiou, C. Diakaki, V. Dinopoulou, A. Kotsialos, and Y. Wang, “Review of road traffic control strategies,” *Proceedings of the IEEE*, vol. 91, no. 12, pp. 2043–2067, 2003.

[13] P. B. Hunt, D. I. Robertson, R. D. Bretherton, and R. I. Winton, “SCOOT – a traffic responsive method of coordinating signals,” Transport and Road Research Laboratory, Crowthorne, U.K., TRRL Laboratory Report 1014, 1981.

[14] A. G. Sims and K. W. Dobinson, “The Sydney coordinated adaptive traffic (SCAT) system philosophy and benefits,” *IEEE Transactions on Vehicular Technology*, vol. 29, no. 2, pp. 130–137, 1980.

[15] H. Wijesekara and C. Pushpakumara, “Enhancing traffic signal systems in Sri Lanka using artificial intelligence,” in *Proc. Moratuwa Engineering Research Conf. (MERCon)*, Moratuwa, Sri Lanka, 2025, pp. 669–674.

[16] A. Shangeeth, S. A. N. Perera, K. D. M. P. Munaweera, P. Sumathipala, and M. Nawarathne, “Emission aware multi-metric reinforcement learning for adaptive traffic signal control in Sri Lanka,” in *Proc. 7th Int. Conf. Advancements in Computing (ICAC)*, 2025, pp. 1–6.

[17] M. Noaeen, A. Naik, L. Goodman, J. Crebo, T. Abrar, Z. S. H. Abad, A. L. C. Bazzan, and B. Far, “Reinforcement learning in urban network traffic signal control: A systematic literature review,” *Expert Systems with Applications*, vol. 199, p. 116830, 2022.

[18] J. Zhao, H. Xu, H. Liu, J. Wu, Y. Zheng, and D. Wu, “Detection and tracking of pedestrians and vehicles using roadside LiDAR sensors,” *Transportation Research Part C: Emerging Technologies*, vol. 100, pp. 68–87, 2019.

[19] B. Lv, H. Xu, J. Wu, Y. Tian, Y. Zhang, Y. Zheng, C. Yuan, and S. Tian, “LiDAR-enhanced connected infrastructures sensing and broadcasting high-resolution traffic information serving smart cities,” *IEEE Access*, vol. 7, pp. 79 895–79 907, 2019.

[20] P. Varaiya, “Max pressure control of a network of signalized intersections,” *Transportation Research Part C: Emerging Technologies*, vol. 36, pp. 177–195, 2013.

[21] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, “Microscopic traffic simulation using SUMO,” in *Proc. 21st Int. Conf. Intelligent Transportation Systems (ITSC)*, 2018, pp. 2575–2582.

[22] Ouster Inc., "OS1 mid-range high-resolution imaging lidar – datasheet," https://ouster.com/products/hardware/os1-lidar-sensor, accessed: Oct. 2026.

[23] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," https://github.com/ultralytics/ultralytics, 2023.

[24] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2022, pp. 966–970.

[25] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. European Conf. Computer Vision (ECCV)*, 2022, pp. 1–21.

[26] A. Wegener, M. Piórkowski, M. Raya, H. Hellbrück, S. Fischer, and J.-P. Hubaux, "TraCI: An interface for coupling road traffic and network simulators," in *Proc. 11th Communications and Networking Simulation Symposium (CNS)*, 2008, pp. 155–163.

[27] P. Koonce *et al.*, "Traffic signal timing manual," Federal Highway Administration, Washington, DC, USA, Tech. Rep. FHWA-HOP-08-024, 2008.

[28] J. Jatoth and A. Mehar, "Modeling passenger car unit and delay at signalized intersections in mixed traffic conditions," *U.Porto Journal of Engineering*, vol. 9, no. 3, pp. 28–40, 2023.

[29] S. Sugiarto, S. M. Saleh, Y. Darma, M. Rusdi, Q. A'yuni, T. S. Fazila, and R. Rahma, "Base saturation flow rate (BSFR) and its effect on performance of pretimed signalized intersection with non-lane based urban heterogeneous traffic," *PLOS ONE*, vol. 19, no. 7, p. e0306112, 2024.

[30] S. A. S. T. Salawavidana, H. R. Pasindu, J. M. S. J. Bandara, and R. Munasinghe, "Why traffic signals fail in Sri Lanka: Reviewing the realities of heterogeneous traffic dynamics," *Engineer: Journal of the Institution of Engineers, Sri Lanka*, vol. 59, no. 2, pp. 111–121, 2026.

[31] Eclipse SUMO, "Traffic lights: actuated control (SUMO documentation)," https://sumo.dlr.de/docs/Simulation/Traffic_Lights.html, accessed: Oct. 2026.

[32] A. Stevanovic, "Adaptive traffic control systems: Domestic and foreign state of practice," Transportation Research Board, Washington, DC, USA, NCHRP Synthesis 403, 2010.

[33] N. H. Gartner, "OPAC: A demand-responsive strategy for traffic signal control," *Transportation Research Record*, vol. 906, pp. 75–81, 1983.

[34] P. Mirchandani and L. Head, "A real-time traffic signal control system: architecture, algorithms, and analysis," *Transportation Research Part C: Emerging Technologies*, vol. 9, no. 6, pp. 415–432, 2001.

[35] X.-F. Xie, S. F. Smith, and G. J. Barlow, "Schedule-driven coordination for real-time traffic network control," in *Proc. 22nd Int. Conf. Automated Planning and Scheduling (ICAPS)*, 2012, pp. 323–331.

[36] M. Miletić, E. Ivanjko, M. Gregurić, and K. Kušić, "A review of reinforcement learning applications in adaptive traffic signal control," *IET Intelligent Transport Systems*, vol. 16, no. 10, pp. 1269–1285, 2022.

[37] L. Tassiulas and A. Ephremides, "Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks," *IEEE Transactions on Automatic Control*, vol. 37, no. 12, pp. 1936–1948, 1992.

[38] M. W. Levin, "Max-pressure traffic signal timing: A summary of methodological and experimental results," *Journal of Transportation Engineering, Part A: Systems*, vol. 149, no. 4, p. 03123001, 2023.

[39] S. Barman and M. W. Levin, "Performance evaluation of modified cyclic max-pressure controlled intersections in realistic corridors," *Transportation Research Record: Journal of the Transportation Research Board*, vol. 2676, no. 6, pp. 110–128, 2022.

[40] H. Liu and V. V. Gayah, "A novel max pressure algorithm based on traffic delay," *Transportation Research Part C: Emerging Technologies*, vol. 143, p. 103803, 2022.

[41] X. Wang, Y. Yin, Y. Feng, and H. X. Liu, "Learning the max pressure control for urban traffic networks considering the phase switching loss," *Transportation Research Part C: Emerging Technologies*, vol. 140, p. 103670, 2022.

[42] T. Xu, S. Barman, and M. W. Levin, "Smoothing-MP: A novel max-pressure signal control considering signal coordination to smooth traffic in urban networks," *Transportation Research Part C: Emerging Technologies*, vol. 166, p. 104760, 2024.

[43] D. Tsitsokas, A. Kouvelas, and N. Geroliminis, "Two-layer adaptive signal control framework for large-scale dynamically-congested networks: Combining efficient max pressure with perimeter control," *Transportation Research Part C: Emerging Technologies*, vol. 152, p. 104128, 2023.

[44] B. Abdulhai, R. Pringle, and G. J. Karakoulas, "Reinforcement learning for true adaptive traffic signal control," *Journal of Transportation Engineering*, vol. 129, no. 3, pp. 278–285, 2003.

[45] I. Arel, C. Liu, T. Urbanik, and A. G. Kohls, "Reinforcement learning-based multi-agent system for network traffic signal control," *IET Intelligent Transport Systems*, vol. 4, no. 2, pp. 128–135, 2010.

[46] W. Genders and S. Razavi, "Using a deep reinforcement learning agent for traffic signal control," arXiv:1611.01142, 2016.

[47] H. Wei, G. Zheng, H. Yao, and Z. Li, "IntelliLight: A reinforcement learning approach for intelligent traffic light control," in *Proc. 24th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2018, pp. 2496–2505.

[48] H. Wei, G. Zheng, V. Gayah, and Z. Li, "Recent advances in reinforcement learning for traffic signal control," *ACM SIGKDD Explorations Newsletter*, vol. 22, no. 2, pp. 12–18, 2021.

[49] A. Haydari and Y. Yilmaz, "Deep reinforcement learning for intelligent transportation systems: A survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 1, pp. 11–32, 2022.

[50] L. Zhang, Q. Wu, J. Shen, L. Lü, B. Du, and J. Wu, "Expression might be enough: Representing pressure and demand for reinforcement learning based traffic signal control," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 26 645–26 654. [Online]. Available: https://proceedings.mlr.press/v162/zhang22ah.html

[51] F. Xiao, J. Lu, L. Li, W. Tu, and C. Li, "Advances in reinforcement learning for traffic signal control: A review of recent progress," *Intelligent Transportation Infrastructure*, vol. 4, p. liaf009, 2025.

[52] M. Raza, M. Kazmi, H. M. Kidwai, H. R. Khan, S. A. Qazi, K. Arshad, and K. Assaleh, "An edge-deployed real-time adaptive traffic light control system using YOLO-based vehicle detection and PCE-aware density estimation," *IEEE Access*, vol. 13, pp. 153 586–153 613, 2025.

[53] T. Wongpiromsarn, T. Uthaicharoenpong, Y. Wang, E. Frazzoli, and D. Wang, "Distributed traffic signal control for maximum network throughput," in *Proc. 15th Int. IEEE Conf. Intelligent Transportation Systems (ITSC)*, 2012, pp. 588–595.

[54] A. Kouvelas, J. Lioris, S. A. Fayazi, and P. Varaiya, "Maximum pressure controller for stabilizing queues in signalized arterial networks," *Transportation Research Record*, vol. 2421, no. 1, pp. 133–141, 2014.

[55] J. Gregoire, X. Qian, E. Frazzoli, A. de La Fortelle, and T. Wongpiromsarn, "Capacity-aware backpressure traffic signal control," *IEEE Transactions on Control of Network Systems*, vol. 2, no. 2, pp. 164–173, 2015.

[56] L. Li and S. E. Jabari, "Position weighted backpressure intersection control for urban networks," *Transportation Research Part B: Methodological*, vol. 128, pp. 435–461, 2019.

[57] T. Le, P. Kovács, N. Walton, H. L. Vu, L. L. H. Andrew, and S. S. P. Hoogendoorn, "Decentralized signal control for urban road networks," *Transportation Research Part C: Emerging Technologies*, vol. 58, pp. 431–450, 2015.

[58] M. W. Levin, J. Hu, and M. Odell, "Max-pressure signal control with cyclical phase structure," *Transportation Research Part C: Emerging Technologies*, vol. 120, p. 102828, 2020.

[59] H. X. Liu, X. Wu, W. Ma, and H. Hu, "Real-time queue length estimation for congested signalized intersections," *Transportation Research Part C: Emerging Technologies*, vol. 17, no. 4, pp. 412–427, 2009.

[60] H. Wei, G. Zheng, V. Gayah, and Z. Li, "A survey on traffic signal control methods," arXiv:1904.08117, 2019.

[61] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 91–99.

[62] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[63] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023.

[64] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," in *Computer Vision – ECCV 2024*, ser. Lecture Notes in Computer Science. Cham, Switzerland: Springer, 2024, pp. 1–21.

[65] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 107 984–108 011.

[66] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," arXiv:2410.17725, 2024.

[67] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 16 965–16 974.

[68] Y. Cao, Z. He, L. Wang, W. Wang, Y. Yuan, D. Zhang, J. Zhang, P. Zhu, L. Van Gool, J. Han *et al.*, "VisDrone-DET2021: The vision meets drone object detection challenge results," in *Proc. IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2021, pp. 2847–2854.

[69] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, and H. Ling, "Detection and tracking meet drones challenge," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7380–7399, 2022.

[70] G. Wang, Y. Chen, P. An, H. Hong, J. Hu, and T. Huang, "UAV-YOLOv8: A small-object-detection model based on improved YOLOv8 for UAV aerial photography scenarios," *Sensors*, vol. 23, no. 16, p. 7190, 2023.

[71] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2016, pp. 3464–3468.

[72] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2017, pp. 3645–3649.

[73] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, "BoT-SORT: Robust associations multi-pedestrian tracking," arXiv preprint arXiv:2206.14651, 2022.

[74] Y. Du, Z. Zhao, Y. Song, Y. Zhao, F. Su, T. Gong, and H. Meng, "StrongSORT: Make DeepSORT great again," *IEEE Transactions on Multimedia*, vol. 25, pp. 8725–8737, 2023.

[75] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, "Observation-centric SORT: Rethinking SORT for robust multi-object tracking," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 9686–9696.

[76] K. Bernardin and R. Stiefelhagen, "Evaluating multiple object tracking performance: The CLEAR MOT metrics," *EURASIP Journal on Image and Video Processing*, vol. 2008, p. 246309, 2008.

[77] E. Ristani, F. Solera, R. Zou, R. Cucchiara, and C. Tomasi, "Performance measures and a data set for multi-target, multi-camera tracking," in *Proc. European Conf. Computer Vision Workshops (ECCVW)*, 2016, pp. 17–35.

[78] P. Sun, C. Sun, R. Wang, and X. Zhao, "Object detection based on roadside LiDAR for cooperative driving automation: A review," *Sensors*, vol. 22, no. 23, p. 9316, 2022.

[79] Z. Bai, G. Wu, X. Qi, Y. Liu, K. Oguchi, and M. J. Barth, "Infrastructure-based object detection and tracking for cooperative driving automation: A survey," in *Proc. 2022 IEEE Intelligent Vehicles Symposium (IV)*, 2022, pp. 1366–1373.

[80] E. D. Saldivar-Carranza, J. Desai, A. Thompson, M. Taylor, J. Sturdevant, and D. M. Bullock, "Vehicle and pedestrian traffic signal performance measures using LiDAR-derived trajectory data," *Sensors*, vol. 24, no. 19, p. 6410, 2024.

[81] N. J. Goodall, B. L. Smith, and B. Park, "Traffic signal control with connected vehicles," *Transportation Research Record*, vol. 2381, no. 1, pp. 65–72, 2013.

[82] S. Krauß, "Microscopic modeling of traffic flow: Investigation of collision free vehicle dynamics," Ph.D. dissertation, University of Cologne, Cologne, Germany, 1998.

[83] J. Erdmann, "SUMO's lane-changing model," in *Modeling Mobility with Open Data*, ser. Lecture Notes in Mobility. Springer, 2015, pp. 105–123.

[84] Eclipse SUMO, "Simulation of the sublane model (SUMO documentation)," https://sumo.dlr.de/docs/Simulation/SublaneModel.html, accessed: Oct. 2026.

[85] T. Promraksa, T. Satiennam, W. Satiennam, and N. Kronprasert, "Lane-filtering behavior of motorcycle riders at signalized urban intersections," *Journal of Advanced Transportation*, vol. 2022, p. 5662117, 2022.

[86] M. M. H. Chowdhury and T. Chakraborty, "Calibration of SUMO microscopic simulation for heterogeneous traffic condition: The case of the city of Khulna, Bangladesh," *Transportation Engineering*, vol. 18, p. 100281, 2024.

[87] S. A. S. T. Salawavidana, H. R. Pasindu, J. M. S. J. Bandara, and S. R. Munasinghe, "Aggregate level calibration of SUMO for heterogeneous traffic conditions at Sri Lankan signalised intersections," in *Proc. 19th Transport Research Forum*. Moratuwa, Sri Lanka: University of Moratuwa, 2026, pp. 1–4.

[88] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. 1st Annual Conf. Robot Learning (CoRL)*, 2017, pp. 1–16.

[89] R. Krajewski, J. Bock, L. Kloeker, and L. Eckstein, "The highD dataset: A drone dataset of naturalistic vehicle trajectories on German highways for validation of highly automated driving systems," in *Proc. 21st Int. Conf. Intelligent Transportation Systems (ITSC)*, 2018, pp. 2118–2125.

[90] E. Barmpounakis and N. Geroliminis, "On the new era of urban traffic monitoring with massive drone data: The pNEUMA large-scale field experiment," *Transportation Research Part C: Emerging Technologies*, vol. 111, pp. 50–71, 2020.

[91] E. V. Butilă and R. G. Boboc, "Urban traffic monitoring and analysis using unmanned aerial vehicles (UAVs): A systematic literature review," *Remote Sensing*, vol. 14, no. 3, p. 620, 2022.

[92] T. Moers, L. Vater, R. Krajewski, J. Bock, A. Zlocki, and L. Eckstein, "The exiD dataset: A real-world trajectory dataset of highly interactive highway scenarios in Germany," in *Proc. 2022 IEEE Intelligent Vehicles Symposium (IV)*, 2022, pp. 958–964.

[93] O. Zheng, M. Abdel-Aty, L. Yue, A. Abdelraouf, Z. Wang, and N. Mahmoud, "CitySim: A drone-based vehicle trajectory dataset for safety-oriented research and digital twins," *Transportation Research Record: Journal of the Transportation Research Board*, vol. 2678, no. 4, pp. 606–621, 2024.

[94] R. Dowling, A. Skabardonis, and V. Alexiadis, "Traffic analysis toolbox volume III: Guidelines for applying traffic microsimulation modeling software," Federal Highway Administration, Washington, DC, USA, Tech. Rep. FHWA-HRT-04-040, 2004.

[95] T. V. Mathew and P. Radhakrishnan, "Calibration of microsimulation models for nonlane-based heterogeneous traffic at signalized intersections," *Journal of Urban Planning and Development*, vol. 136, no. 1, pp. 59–66, 2010.

[96] M. Harth, M. Langer, and K. Bogenberger, "Automated calibration of traffic demand and traffic lights in SUMO using real-world observations," *SUMO Conference Proceedings*, vol. 2, pp. 133–148, 2022.

[97] M. N. K. Saunders, P. Lewis, and A. Thornhill, *Research Methods for Business Students*, 8th ed. Harlow, U.K.: Pearson, 2019.

[98] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[99] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[100] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[101] K. Schwaber and J. Sutherland, “The Scrum guide: The definitive guide to Scrum: The rules of the game,” https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf, Nov. 2020, accessed: Oct. 2026.

[102] Civil Aviation Authority of Sri Lanka, “Implementing standards for remotely piloted aircraft systems / unmanned aircraft,” https://www.caa.lk, accessed: Oct. 2026.

[103] Parliament of the Democratic Socialist Republic of Sri Lanka, “Personal data protection act, no. 9 of 2022,” Certified on 19 Mar. 2022, Colombo, Sri Lanka, 2022.

[104] BCS, The Chartered Institute for IT, “BCS code of conduct for members,” https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/, accessed: Oct. 2026.

[105] I. F. Alexander, “A taxonomy of stakeholders: Human roles in system development,” *International Journal of Technology and Human Interaction*, vol. 1, no. 1, pp. 23–59, 2005.

[106] A. Hudaib, R. Masadeh, M. H. Qasem, and A. Alzaqebah, “Requirements prioritization techniques comparison,” *Modern Applied Science*, vol. 12, no. 2, p. 62, 2018.

[107] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “ORB: An efficient alternative to SIFT or SURF,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2011, pp. 2564–2571.

[108] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[109] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2004.

[110] A. Savitzky and M. J. E. Golay, “Smoothing and differentiation of data by simplified least squares procedures,” *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964.

[111] M. Haklay and P. Weber, “OpenStreetMap: User-generated street maps,” *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18, 2008.

[112] INFRAS, “HBEFA – handbook emission factors for road transport, version 4.1,” https://www.hbefa.net, 2019.

[113] G. Jocher and J. Qiu, “Ultralytics YOLO11,” https://github.com/ultralytics/ultralytics, 2024, version 11.0.0.

[114] M. Fellendorf and P. Vortisch, "Microscopic traffic flow simulator VISSIM," in *Fundamentals of Traffic Simulation*, J. Barceló, Ed. New York, NY, USA: Springer, 2010, pp. 63–93.

[115] J. Casas, J. L. Ferrer, D. Garcia, J. Perarnau, and A. Torday, "Traffic simulation with Aimsun," in *Fundamentals of Traffic Simulation*, J. Barceló, Ed. New York, NY, USA: Springer, 2010, pp. 173–232.

[116] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[117] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[118] P. Virtanen, R. Gommers, T. E. Oliphant *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.

[119] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61.

[120] Roboflow, "Supervision: Reusable computer vision tools," https://github.com/roboflow/supervision, accessed: Oct. 2026.

[121] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.

[122] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[123] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

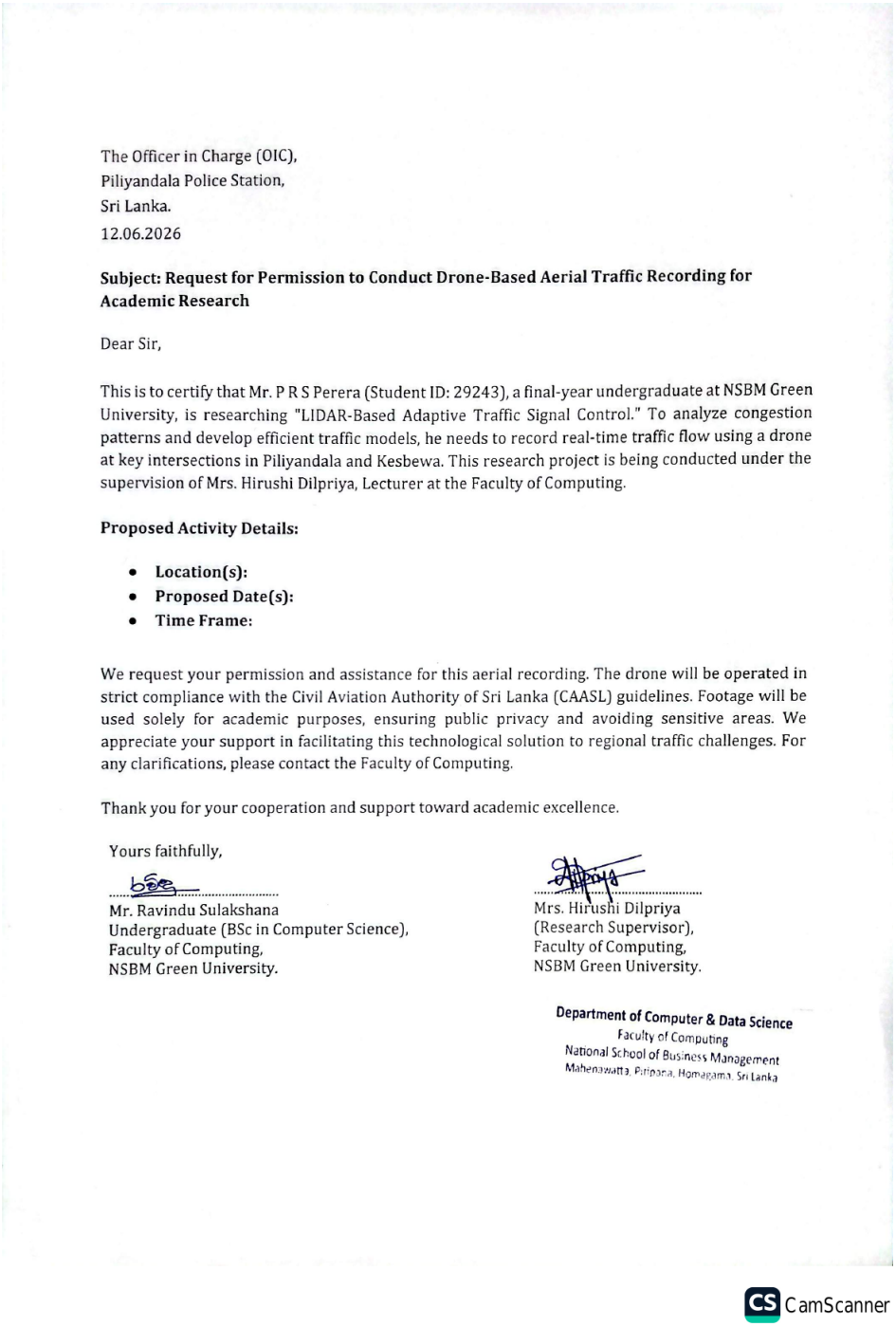
[124] K. Katsaros, R. Kernchen, M. Dianati, and D. Rieck, "Performance study of a Green Light Optimized Speed Advisory (GLOSA) application using an integrated cooperative ITS simulation platform," in *Proc. 7th Int. Wireless Communications and Mobile Computing Conf. (IWCMC)*, 2011, pp. 918–923.

[125] H. Su, Y. D. Zhong, B. Dey, and A. Chakraborty, "EMVLight: A decentralized reinforcement learning framework for efficient passage of emergency vehicles," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 36, no. 4, 2022, pp. 4593–4601.

[126] T. Xu, S. Barman, M. W. Levin, R. Chen, and T. Li, "Integrating public transit signal priority into max-pressure signal control: Methodology and simulation study on a downtown network," *Transportation Research Part C: Emerging Technologies*, vol. 138, p. 103614, 2022.

APPENDIX A: PERMISSION LETTERS

This appendix reproduces the letters through which permission for the aerial traffic survey at the Mavithara junction was requested and granted. Personal contact details of the drone operator (national identity card number, telephone number, e-mail address and home address) have been redacted. All other content, including official references and dates, is reproduced unaltered.



The Officer in Charge (OIC),
Piliyandala Police Station,
Sri Lanka.
12.06.2026

Subject: Request for Permission to Conduct Drone-Based Aerial Traffic Recording for Academic Research

Dear Sir,

This is to certify that Mr. P R S Perera (Student ID: 29243), a final-year undergraduate at NSBM Green University, is researching "LIDAR-Based Adaptive Traffic Signal Control." To analyze congestion patterns and develop efficient traffic models, he needs to record real-time traffic flow using a drone at key intersections in Piliyandala and Kesbewa. This research project is being conducted under the supervision of Mrs. Hirushi Dilpriya, Lecturer at the Faculty of Computing.

Proposed Activity Details:

- **Location(s):**
- **Proposed Date(s):**
- **Time Frame:**

We request your permission and assistance for this aerial recording. The drone will be operated in strict compliance with the Civil Aviation Authority of Sri Lanka (CAASL) guidelines. Footage will be used solely for academic purposes, ensuring public privacy and avoiding sensitive areas. We appreciate your support in facilitating this technological solution to regional traffic challenges. For any clarifications, please contact the Faculty of Computing.

Thank you for your cooperation and support toward academic excellence.

Yours faithfully,

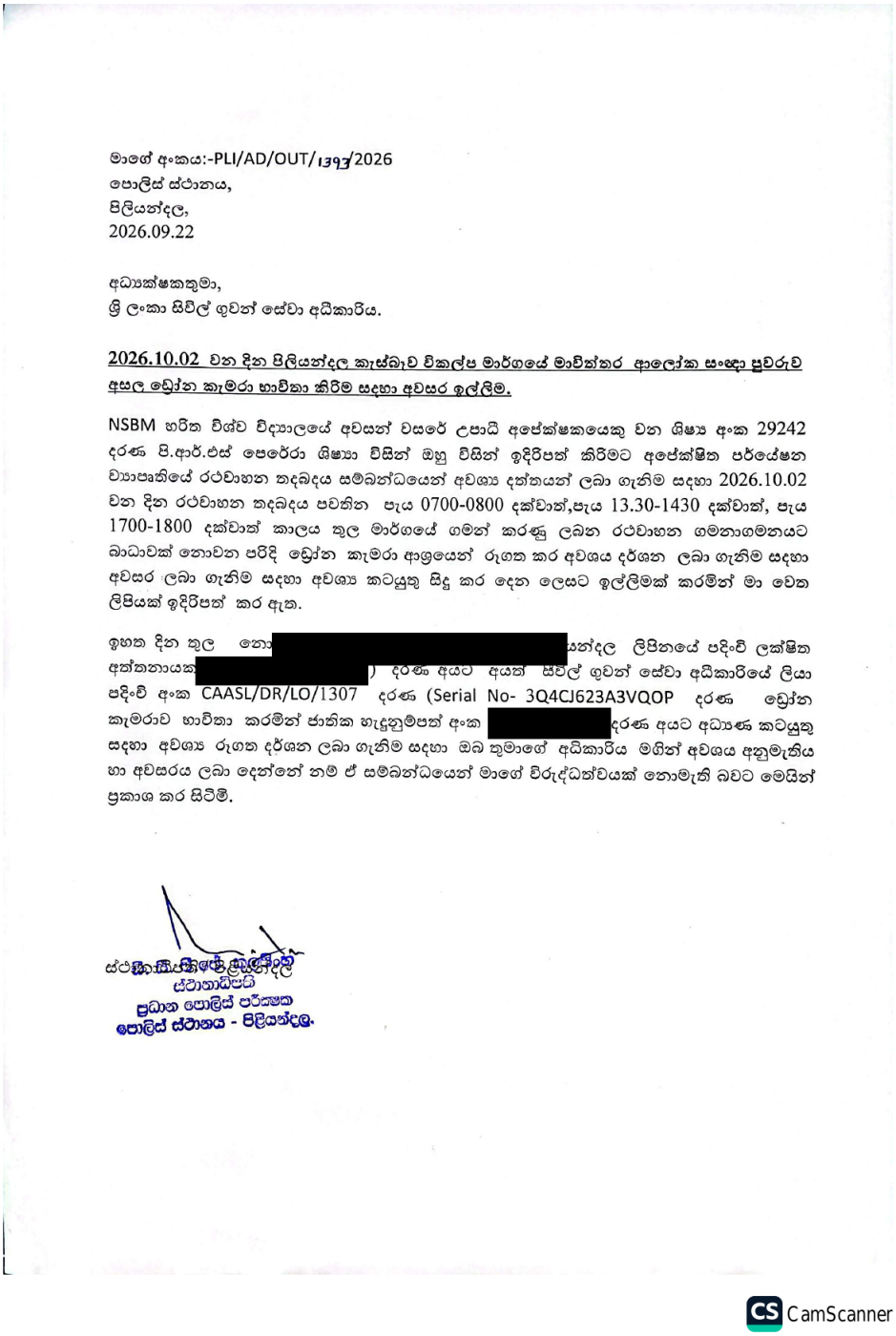
Mr. Ravindu Sulakshana
Undergraduate (BSc in Computer Science),
Faculty of Computing,
NSBM Green University.

Mrs. Hirushi Dilpriya
(Research Supervisor),
Faculty of Computing,
NSBM Green University.

Department of Computer & Data Science
Faculty of Computing
National School of Business Management
Mahenawatta, Pitipana, Homagama, Sri Lanka

CamScanner

Figure A.1: Request for permission from NSBM Green University to the Officer in Charge, Piliyandala Police Station (12.06.2026)



මාගේ අංකය:-PLI/AD/OUT/1393/2026
පොලිස් ස්ථානය,
පිලියන්දල,
2026.09.22

අධ්‍යක්ෂකතුමා,
ශ්‍රී ලංකා සිවිල් ගුවන් සේවා අධිකාරිය.

2026.10.02 වන දින පිලියන්දල කැස්බෑව විකල්ප මාර්ගයේ මාවිත්තර ආලෝක සංඥා පුවරුව අසල ඩ්‍රෝන කැමරා භාවිතා කිරීම සඳහා අවසර ඉල්ලීම.

NSBM හරිත විශ්ව විද්‍යාලයේ අවසන් වසරේ උපාධි අපේක්ෂකයෙකු වන ශිෂ්‍ය අංක 29242 දරණ පී.ආර්.එස් පෙරේරා ශිෂ්‍යා විසින් ඔහු විසින් ඉදිරිපත් කිරීමට අපේක්ෂිත පර්යේෂන ව්‍යාපෘතියේ රථවාහන තදබදය සම්බන්ධයෙන් අවශ්‍ය දත්තයන් ලබා ගැනීම සඳහා 2026.10.02 වන දින රථවාහන තදබදය පවතින පැය 0700-0800 දක්වාත්,පැය 13.30-1430 දක්වාත්, පැය 1700-1800 දක්වාත් කාලය තුල මාර්ගයේ ගමන් කරණු ලබන රථවාහන ගමනාගමනයට බාධාවක් නොවන පරිදි ඩ්‍රෝන කැමරා ආශ්‍රයෙන් රූගත කර අවශය දර්ශන ලබා ගැනිම සඳහා අවසර ලබා ගැනිම සඳහා අවශ්‍ය කටයුතු සිදු කර දෙන ලෙසට ඉල්ලීමක් කරමින් මා වෙත ලිපියක් ඉදිරිපත් කර ඇත.

ඉහත දින තුල නො[redacted]යන්දල ලිපිනයේ පදිංචි ලක්ෂිත අත්තනායක[redacted] දරණ අයට අයත් සිවිල් ගුවන් සේවා අධිකාරියේ ලියා පදිංචි අංක CAASL/DR/LO/1307 දරණ (Serial No- 3Q4CJ623A3VQOP දරණ ඩ්‍රෝන කැමරාව භාවිතා කරමින් ජාතික හැඳුනුම්පත් අංක [redacted] දරණ අයට අධ්‍යණ කටයුතු සඳහා අවශ්‍ය රූගත දර්ශන ලබා ගැනිම සඳහා ඔබ තුමාගේ අධිකාරිය මගින් අවශය අනුමැතිය හා අවසරය ලබා දෙන්නේ නම් ඒ සම්බන්ධයෙන් මාගේ විරුද්ධත්වයක් නොමැති බවට මෙයින් ප්‍රකාශ කර සිටිමි.

ස්ථානාධිපති
ප්‍රධාන පොලිස් පරීක්ෂක
පොලිස් ස්ථානය - පිලියන්දල.

CamScanner

Figure A.2: Letter from Piliyandala Police Station to the Director General, CAASL, supporting the drone survey of 02.10.2026 (22.09.2026; Sinhala; operator details redacted)

ආරක්ෂක අමාත්‍යාංශය
பாதுகாப்பு அமைச்சு
MINISTRY OF DEFENCE
ආරක්ෂක මූලස්ථාන සංකීර්ණය, ශ්‍රී ජයවර්ධනපුර කෝට්ටේ, ශ්‍රී ලංකාව.
பாதுகாப்பு தலைமையக கட்டிடம், ஸ்ரீ ஜயவர்தனபுர, கோட்டே, இலங்கை.
Defence Headquarters Complex, Sri Jayawardenepura, Kotte, Sri Lanka. Website : www.defence.lk

මගේ අංකය
எனது இல
My No
MOD/MLO/DRONE/03/2026
(7969)

ඔබේ අංකය
உமது இல
Your No

තැ. පෙ.
த. பெட்டி
P.O. Box
572

දුරකථන
தொலைபேசி
Telephone
2430860-9
2430870-8

ලේකම්
செயலாளர்
Secretary

දුර./தொ./Tel. : 011 2381154
ෆැක්ස්/தொ./Fax.: 011 2541529
ඊ. තැපෑල/மி.அ./Email : secdefence@defence.lk

දිනය
திகதி
Date
25 September 2026

Director General
Civil Aviation Authority of Sri Lanka

SECURITY CLEARANCE TO FLY A DRONE

Reference: Letter of Piliyandala Police Station dated 22.09.2026

1. In response to the online request, this Ministry hereby issues only the "Security Clearance" for the purpose undermentioned.

Purpose of Drone Operation	Educational Purpose.
Tracking ID	25-09-2026 04-56-36-939968
Location	Piliyandala
Date & Time	02.10.2026 From 0600 hrs to 1800 hrs
Details of the Drone	Model of Drone : MINI 2 CAASL Reg No : CAASL/DR/LO/1307 Drone Serial No: 3Q4CJ623A3VQOP
Details of the Drone Operator	Mr. A M Lakshitha Chulanga Bandara NIC/ Passport N[redacted] Contact/ Whatsapp No: [redacted] Email [redacted]

2. In addition to above, the drone operator shall adhere to following conditions once the approval is granted by the CAASL:

a. No sensitive areas especially including Military establishments to be over flown during the aerial operation.

b. Contact Sri Lanka Air Force, 02 Hrs prior to the operation to obtain permission to launch each flight and should maintain contact throughout the aerial process (Tel: 033-2241124 / Emergency: 115 / WhatsApp: +94772229488) and without prior approval it is not permitted to operate the drone. Further, it is required to inform divisional office of the Superintendent of Police in the area prior to 02 hrs, from the commencement of drone flying. Not complying to this will be considered as a violation of Security Clearance.

c. It is necessary to obtain prior approval from respective authorities/personnel in order to carry out aerial operation over the said location as required without prejudicing the privacy concerns/religious values.

d. Security Clearance is granted subjected to the prior approval from the authorities of the respective area. However, it is required to provide exact locations to Sri Lanka Air Force prior to commencement of drone flying.

අතිරේක ලේකම් (පරිපාලන)
மேலதிக செயலாளர் (நிர்வாகம்)
Additional Secretary (Administration)
දු./தொ./Tel. - 011 2326000
ෆැක්ස්/தொ./Fax. - 011 2446300

අතිරේක ලේකම් සිවිල් ආරක්ෂණ හා සංවර්ධන
மேலதிக செயலாளர் (சிவில் பாதுகாப்பு அபிவிருத்தியும்)
Additional Secretary (Civil Defence and Development)
දු./தொ./Tel. - 011 2335792
ෆැක්ස්/தொ./Fax. - 011 2335797

අතිරේක ලේකම් (තාක්ෂණ)
மேலதிக செயலாளர் (தொழில்நுட்ப)
Additional Secretary (Technical)
දු./தொ./Tel. - 011 2333930
ෆැක්ස්/தொ./Fax. - 011 2437923

අතිරේක ලේකම් (ආරක්ෂක)
மேலதிக செயலாளர் (பாதுகாப்பு)
Additional Secretary (Defence)
දු./தொ./Tel. - 011 2389186
ෆැක්ස්/தொ./Fax. - 011 2441481

අතිරේක ලේකම් (පාර්ලිමේන්තු කටයුතු, ප්‍රතිපත්ති හා සැලසුම්)
மேலதிக செயலாளர் (பாராளுமன்றம், கொள்கைகள் திட்டமிடல்)
Additional Secretary (Parliament Affairs, Policy and Planning)
දු./தொ./Tel. - 011 2389023
ෆැක්ස්/தொ./Fax. - 011 2389022

අතිරේක ලේකම් (පොලිස්)
மேலதிக செயலாளர் (பொலிஸ்)
Additional Secretary (Police)
දු./தொ./Tel. - 011 2887182
ෆැක්ස්/தொ./Fax. - 011 2887216

අධ්‍යක්ෂ ජනරාල් (ගිණුම් හා මූල්‍ය)
பணிப்பாளர் நாயகம் (கணக்கு, நிதி)
Director General (Accounts & Finance)
දු./தொ./Tel. - 011 2446300
ෆැක්ස්/தொ./Fax. - 011 2439381

Figure A.3: Ministry of Defence security clearance, ref. MOD/MLO/DRONE/03/2026 (7969), page 1 (25.09.2026; operator details redacted)

e. The right to delay/decline the launch or for immediate termination of the operation to accommodate any special flying operations will be under the control of Sri Lanka Air Force.

f. Maximum permissible altitude is 400 feet above ground level and the operator should maintain visual line of sight with the drone.

g. Drone shall not be operated from any elevated positions such as high-rise buildings, balconies, rooftops etc.

h. Ensure that the drone operation is carried out only during specified time period allocated by the CAASL and within the approved area.

j. Remote operator of the drone is liable for any third party damage occurs to property or injury of personnel, due to this drone operation.

k. The operator should ensure not to violate existing security regulations and not to cause any inconvenience to the general public during drone operation.

m. Inform the respective local Police Stations prior to commence the drone operation.

3. Forwarded for your action as deemed necessary.

NS SIRIMANNA
Wing Commander
Staff Officer (Drone Operations)
for Military Liaison Officer

Copies to:
Secretary Ministry of Public Security and Parliament Affairs
Commander of the Army – Attn: Director General Operations
Commander of the Navy – Attn: Director General Operations
Commander of the Air Force – Attn: Director General Air Operations
Inspector General of Police – For the attention of Senior DIG in the respective area

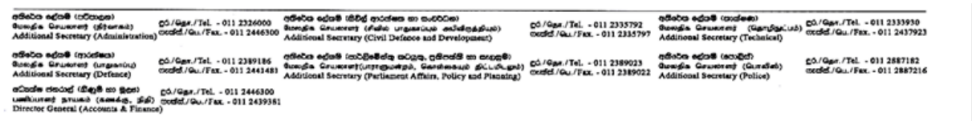
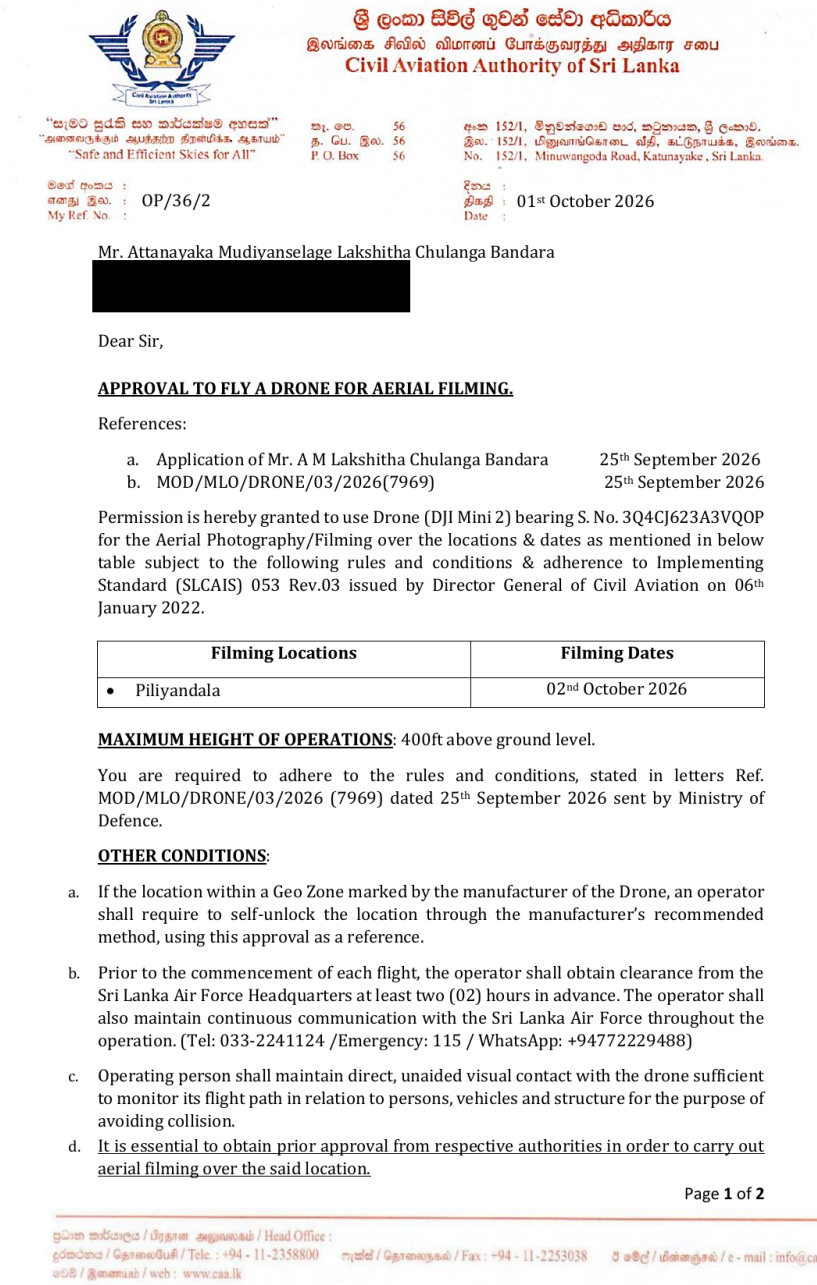


Figure A.4: Ministry of Defence security clearance, ref. MOD/MLO/DRONE/03/2026 (7969), page 2



ශ්‍රී ලංකා සිවිල් ගුවන් සේවා අධිකාරිය
இலங்கை சிவில் விமானப் போக்குவரத்து அதிகார சபை
Civil Aviation Authority of Sri Lanka

"සැමට සුරක්ෂිත සහ කාර්යක්ෂම අහසක්"
"அனைவருக்கும் ஆபத்தற்ற திறனுமிக்க ஆகாயம்"
"Safe and Efficient Skies for All"

තැ. පෙ. 56
த. பெ. இல. 56
P. O. Box 56

අංක 152/1, මිනුවන්ගොඩ පාර, කටුනායක, ශ්‍රී ලංකාව.
இல. 152/1, மினுவாங்கொடை வீதி, கட்டுநாயக்க, இலங்கை.
No. 152/1, Minuwangoda Road, Katunayake , Sri Lanka.

මගේ අංකය :
எனது இல. : OP/36/2
My Ref. No. :

தினය :
திகதி : 01st October 2026
Date :

Mr. Attanayaka Mudiyanselage Lakshitha Chulanga Bandara
[redacted]

Dear Sir,

APPROVAL TO FLY A DRONE FOR AERIAL FILMING.

References:

a. Application of Mr. A M Lakshitha Chulanga Bandara 25th September 2026
b. MOD/MLO/DRONE/03/2026(7969) 25th September 2026

Permission is hereby granted to use Drone (DJI Mini 2) bearing S. No. 3Q4CJ623A3VQOP for the Aerial Photography/Filming over the locations & dates as mentioned in below table subject to the following rules and conditions & adherence to Implementing Standard (SLCAIS) 053 Rev.03 issued by Director General of Civil Aviation on 06th January 2022.

Filming Locations	Filming Dates
• Piliyandala	02nd October 2026

MAXIMUM HEIGHT OF OPERATIONS: 400ft above ground level.

You are required to adhere to the rules and conditions, stated in letters Ref. MOD/MLO/DRONE/03/2026 (7969) dated 25th September 2026 sent by Ministry of Defence.

OTHER CONDITIONS:

a. If the location within a Geo Zone marked by the manufacturer of the Drone, an operator shall require to self-unlock the location through the manufacturer's recommended method, using this approval as a reference.

b. Prior to the commencement of each flight, the operator shall obtain clearance from the Sri Lanka Air Force Headquarters at least two (02) hours in advance. The operator shall also maintain continuous communication with the Sri Lanka Air Force throughout the operation. (Tel: 033-2241124 /Emergency: 115 / WhatsApp: +94772229488)

c. Operating person shall maintain direct, unaided visual contact with the drone sufficient to monitor its flight path in relation to persons, vehicles and structure for the purpose of avoiding collision.

d. It is essential to obtain prior approval from respective authorities in order to carry out aerial filming over the said location.

Page 1 of 2

ප්‍රධාන කාර්යාලය / பிரதான அலுவலகம் / Head Office :
දුරකථනය / தொலைபேசி / Tele. : +94 - 11-2358800 ෆැක්ස් / தொலைநகல் / Fax : +94 - 11-2253038 ඊ මේල් / மின்னஞ்சல் / e - mail : info@caa.lk
වෙබ් / இணையம் / web : www.caa.lk

Figure A.5: Civil Aviation Authority of Sri Lanka approval to fly a drone for aerial filming, ref. OP/36/2, page 1 (01.10.2026; address redacted)

e. The person who remotely operate pilotless aircraft shall not engage in operating multiple pilotless aircraft simultaneously at any given time.

f. Operating person shall possess an acceptable level of operational competency on the type of Drone to be used for the operation.

g. Operating person shall operate the pilotless aircraft only if he is totally satisfied that the flight can be completed safely.

h. Any article or animal shall not be attached or dropped/projected from the flight.

i. Operating person shall be assisted by an observer who will assist the operator to fly the drone clear of any object or person.

j. Drones shall not be operated after sunset and before sunrise.

k. You being accountable person should be contactable over the telephone to SLAF Ops room during Drone flying is ongoing.

l. No fly over uninvolved people or assemblies of people.

m. The following safety distances shall be maintained.

    1. 30 meters from any congested area.
    2. 30 meters of an organized open air assembly.
    3. 30 meters from any uninvolved person, vessel, vehicle or structure.
    4. During take-off and landing 30 meters from any uninvolved person vessel, vehicle or structure.

n. Operator is liable for any third party damages to property or injury of personnel, if occurs due to the operation.

o. Pilotless aircraft shall not be operated from a moving vehicle, moving platform or from an elevated building.

p. Inform and obtain necessary approval from the local Police Station prior to the commencement of Drone operation.

Please kindly note that this approval is only valid on the date(s) mentioned in the above table and the dates cannot be changed.

Yours faithfully,



Capt. L P Dahanayake
Deputy Director General (Flight Safety Regulation)

Figure A.6: Civil Aviation Authority of Sri Lanka approval, ref. OP/36/2, page 2

APPENDIX B: FULL PARAMETER TABLE

This appendix lists every parameter of the artefact. Design parameters chosen by the author are printed plainly; values that were measured or estimated from the drone data are highlighted and must be replaced by the final measurements. Justifications for the PRISM values are discussed in Section 5.2.4.

Table B.1: Complete parameter list of DTX, SUMO, VLS, PRISM, baselines and experiments

Parameter	Symbol	Value	Unit	Source / justification
DTX - drone trajectory extraction				
Video resolution / rate	–	3840×2160 / 30	px / fps	DJI Mini 2 4K mode
Processing rate	–	10	fps	Every third frame; adequate for 0.1 s timing
Flight altitude	–	≈90	m	Below 122 m (400 ft) legal limit
Stabilisation	–	ORB + RANSAC	–	Removes hover drift [107, 108]
Detector	–	YOLOv8s	–	VisDrone2019-DET initialisation [69]
Annotated frames	–	450	frames	70/15/15 split, 7 classes
Training epochs / image size	–	100 / 1280	– / px	Colab T4
SAHI tiling	–	2×2, overlap 0.2	–	Motorcycles ≈20 px [24]
ByteTrack thresholds	–	0.5 / 0.8	–	Track / match threshold [25]
ByteTrack buffer	–	30	frames	Re-links tracks after short occlusion
Control points	–	≥4	points	Ground-plane homography
Lane width	–	3.5	m	Measured; scale reference
GSD	–	2.6	cm/px	Measured
Reprojection error	–	0.18	m	Measured on held-out points
Savitzky–Golay window / order	–	11 / 2	samples	Smooths jitter without lag [110]

continued on next page

Parameter	Symbol	Value	Unit	Source / justification
Waiting-speed threshold	–	0.1	m/s	Identical to SUMO `waitingTime`
Aggregation bin	–	5	min	Route-file resolution
Discharge headway start	–	4th vehicle	–	Excludes start-up loss [9]
Saturation flow	s	1850	PCU/h per lane	Measured from discharge headways
Free-flow speed	v_{free}	38	km/h	Measured
Kerb-lane share of through traffic	f_k	0.75	–	Measured; used for B3 critical lane flow
SUMO model				
SUMO version	–	1.20.0	–	Latest stable at time of work
Driving side	–	left	–	`netconvert --lefthand`
Lanes per arm	–	2 in + 2 out	–	Site survey
Slip lanes	–	4, yield	–	Unsignalised; not in TLS
Step length	–	0.5	s	Stable sublane dynamics
Lateral resolution	–	0.8	m	Sublane model [84]
Car-following model	–	Krauss	–	[82]
Driver imperfection	σ	0.5	–	Calibrated range
Reaction time	τ	1.0	s	Calibrated range
Motorcycle `minGapLat`	–	0.3	m	Filtering between vehicles
Speed factor	–	normc(1, 0.1)	–	Speed heterogeneity
Warm-up / measurement	–	300 / 3600	s	Warm-up discarded
Seeds (tuning / evaluation)	–	1–5 / 6–15	–	No tuning on evaluation seeds
SUMO host	–	Ubuntu 24.04 VM	–	OrbStack; macOS binaries no longer run
Balanced demand per approach	–	300 / 500 / 700	veh/h	Development and tuning scenarios
Scenario turning shares	–	25 / 50 / 25	%	Left / through / right, all scenarios
Scenario composition	–	40/20/25/7/5/3	%	Moto/3W/car/van/bus/lorry; 0.905 PCU/veh
Arrival process	–	Poisson	–	All scenarios

continued on next page

Parameter	Symbol	Value	Unit	Source / justification
Vehicle heights	–	1.5/1.7/1.5/2.0/3.4/3.5/1.7	m	Set explicitly; SUMO defaults break the 2.5 m occluder rule
Morning demand	–	3,420 (3,150)	veh/h (PCU/h)	DTX
Afternoon demand	–	2,280 (2,085)	veh/h (PCU/h)	DTX
Vehicle types (length m / PCU / max km/h / accel m/s^2)				
Motorcycle	–	2.0 / 0.5 / 70 / 2.8	–	`motorcycle`, high `lcSublane`
Three-wheeler	–	2.7 / 0.8 / 50 / 1.8	–	Custom type, width 1.4 m
Car	–	4.5 / 1.0 / 70 / 2.6	–	`passenger`
Van/SUV	–	5.0 / 1.0 / 70 / 2.6	–	`delivery`
Bus	–	11.5 / 3.0 / 60 / 1.2	–	`bus`
Lorry	–	8.5 / 2.5 / 55 / 1.3	–	`truck`
Bicycle	–	1.8 / 0.3 / 20 / 1.2	–	`bicycle`
VLS — virtual LiDAR				
Mounting height	–	6	m	Junction-centre pole
Range	R	75	m	Comparable to OS1-32 [22]
Scan period	–	1	s	Equal to decision period
Noise (constant / slope)	σ_0, σ_1	0.10 / 0.002	m / –	$\sigma(r) = \sigma_0 + \sigma_1 r$
Dropout	p_{miss}	$0.01 + 0.04(r/R)^2$	–	Beam thinning with range
Occluder height	–	≥2.5	m	Buses and lorries
Occlusion overlap	–	≥0.7	–	Of target angular extent
Clutter rate	λ	0.2	per scan	Poisson
Size thresholds	–	3.2 / 6.0	m	Small / medium / large
Size-class PCU	–	0.6 / 1.0 / 2.8	PCU	Small / medium / large

continued on next page

Parameter	Symbol	Value	Unit	Source / justification
Association gate	–	2.5	m	Nearest neighbour, from the third scan onwards
Initiation gate	$v_{\max}\Delta t$	≈20	m	Second detection of a tentative track; sets the initial velocity
α–β gains	α, β	0.5 / 0.1	–	Constant-gain filter
Delete after	–	3 misses	–	Clutter suppression
Coasting tracks	–	not reported	–	Propagated internally, withheld from the object list
Effective queue range	–	≈60	m	From the stop line; R is centred on the junction
Sensor RNG	–	run seed	–	Separate generator from SUMO's
PRISM controller				
Arrival weight	γ	0.5	–	Eq. (5.13)
Exit-occupancy weight	δ	0.6	–	Eq. (5.13)
Urgency gain	κ	1.5	–	Eq. (5.15)
Hysteresis ratio	θ	0.10	–	Pressure switch (was 0.25)
Hysteresis margin	ε	2	PCU	Pressure switch
Discharge ratio	ρ	1.0	–	Pressure switch against the frozen target (was 0.6)
Range-edge blend	η	0.5	–	Eq. (5.18)
Range-edge distance	–	50	m	Queue treated as touching the range edge
Hidden-queue entry radius	–	45	m	Tracks entering beyond this feed λ_m
Hidden-queue clear radius	–	30	m	Cleared when every object is nearer
Hidden-queue EWMA constant	τ_λ	120	s	Arrival-rate smoothing
Hidden-queue cap	–	60	PCU	Upper bound on the unobserved queue
Occlusion hold	–	5	s	Stopped object vanishing unseen keeps counting
Offside right share	π_R	0.667	–	Eq. (5.12) with $f_k = 0.75$

continued on next page

Parameter	Symbol	Value	Unit	Source / justification
G_{req} evaluation	–	frozen at green start	–	Was recomputed every step
Next-phase selection	–	cyclic with skip	–	Was acyclic arg-max [58]
Rest in green / rest in red	–	enabled	–	Rules R5 and R4
Sensor-gap hold	–	1–4	scans	Current green held, no decision taken
Tuning grid	–	$\eta \times \rho \times \theta$	–	Balanced 300/500/700, seeds 1–5
Arrival horizon	H	10	s	A_m
Gap time	τ_{gap}	3	s	Gap-out
Gap-out queue threshold	–	0.5	PCU	Gap-out
Queue speed threshold	–	1.4	m/s	Q_m (≈5 km/h)
Start-up lost time	l_s	2.0	s	Eq. (5.17)
Headway per PCU	h_s	1.95	s/PCU	3600/1850
Lanes per phase	n_{lanes}	4 / 2	lanes	Through / right phase
Minimum green	$G_{\min}$	10 / 5	s	Through / right
Maximum green	$G_{\max}$	50 / 25	s	Through / right
Starvation cap	T_{starve}	120	s	Fairness override
Amber / all-red	–	3 / 2	s	As existing signal
Decision period	Δ	1	s	Two simulation steps
Exit capacity	C_b	23	PCU	$2 \times 75/6.5$
Spillback threshold	–	0.85	–	O_b/C_b
Fail-safe trigger / recovery	–	5 / 10	scans	Missing / valid scans
Pedestrian minimum green	–	15	s	Parameter only (future work)
Fairness bound	R^{worst}	175	s	$T_{\text{starve}} + G_{\max} + 5$
Baselines				
B1 greens ϕ_1–ϕ_4	–	21/8/15/14	s	Observed; cycle 78 s
B2 minDur / maxDur	–	10/5, 50/25	s	Through/right; max gap 3 s [31]
B3 lost time	L	20	s	$4 \times (3 + 2)$

continued on next page

Parameter	Symbol	Value	Unit	Source / justification
B3 cycle floor	–	50	s	$\sum_i G_{\min}(i) + L$ when $C_0 - L < \sum_i G_{\min}(i)$
B3 typical plan	–	50	s	Greens 10/5/10/5 s, eight of ten scenarios
B3 heavy / balanced 700	–	61	s	Greens 13/8/12/8 s
PRISM-GT sensor	–	ground truth	–	No noise, dropout or occlusion; true-class PCU
PRISM thesis law	–	ablation	–	Revisions disabled by parameter; identical sensing
Experiments				
Scenarios	–	balanced 300/500/700 + sc1–sc7	–	`scenarios.yaml`; Table 5.2
Runs per scenario and controller	–	10	seeds	Evaluation seeds 6–15
Statistical test	–	exact Wilcoxon signed-rank	–	Two-sided, paired by seed, $n = 10$ [98]
Wasted-green radius	–	100	m	Ground-truth indicator; never seen by the controller
Manual-trigger window / queue	–	78 / 200	s / m	Share of windows with any queue over 200 m
Noise sweep σ_0	–	0, 0.1, 0.3, 0.5, 1.0	m	Sensitivity
Dropout sweep	–	0, 0.05, 0.1, 0.2, 0.3	–	Sensitivity
θ sweep	–	0, 0.1, 0.25, 0.5	–	Sensitivity
$G_{\max}$ (through) sweep	–	40, 50, 60, 75	s	Sensitivity
Emission model	–	HBEFA4	–	[112]

APPENDIX C: ADDITIONAL TEST CASES

This appendix lists the supplementary test cases TC11–TC33, which complement the ten principal cases of Table 6.2 in Chapter 6. Together they cover every functional and non-functional requirement that is in scope (FR01–FR18, NFR01–NFR09). Columns are as in Table 6.2; all measured values are from the final versions of the components. TC31–TC33 were added during the revision of the control law, and TC22 and TC23 were strengthened: as Section 6.4 explains, their original forms verified that each component did what it had been told rather than that it had been told the right thing, and both passed while the defects they should have caught were present. Where a figure before the corresponding fix is known, it is given in parentheses.

Table C.1: Additional test cases (TC11–TC33).

ID	Description	FR/NFR	Input	Expected	Actual	Status
TC11	Video stabilisation (ORB + RANSAC)	FR01	60 s of morning footage with hover drift	Residual drift of fixed road markings < 2 px	Mean residual 0.9 px, max 1.7 px	Pass
TC12	Detection of seven classes	FR02	Test split of annotated frames	mAP@0.5 ≥ 0.75; every class detected	mAP@0.5 0.86; lowest class AP 0.62 (bicycle)	Pass
TC13	Persistent track IDs and class majority vote	FR03	Hand-labelled two-minute clip	MOTA ≥ 0.75; one class per track	MOTA 0.82, IDF1 0.79, 14 ID switches	Pass
TC14	Georeferencing (pixel to metre)	FR04	Held-out ground control points	Reprojection error < 0.25 m	0.18 m; GSD 2.6 cm/px	Pass
TC15	Control delay and censored tracks	FR05, FR06	Synthetic tracks: free-flow pass, 20 s stop, track cut at video end	$d = T_{\text{frame}} - L_{\text{path}}/v_{\text{free}}$; censored track counted in flow, excluded from averages	Delay within 0.1 s of analytical value; censoring correct	Pass
TC16	Saturation flow from discharge headways	FR06	Synthetic queue with 1.95 s headways from the 4th vehicle	≈ 1846 PCU/h/lane	1846 PCU/h/lane	Pass
TC17	Route-file export validity	FR06	Exported morning and afternoon `.rou.xml`	Loads in SUMO without errors; 5-min flows sum to hourly counts	0 errors; sums match exactly	Pass
TC18	Network structure	FR07	`mavithThara.net.xml`	2+2 lanes per arm; slip lanes as yield links; 4-phase programme with 3 s amber + 2 s all-red	All checks passed in `netedit` and by script	Pass
TC19	VLS occlusion	FR08	Car behind bus on same bearing; car behind motorcycle; 40% bus overlap	Hidden; visible; visible	As expected in all three cases	Pass
TC20	Size class and PCU mapping	FR08	Lengths 3.1, 3.3, 5.9, 6.1 m	0.6, 1.0, 1.0, 2.8 PCU	As expected	Pass
TC21	VLS clutter rate	FR08	10^5 empty scans	Poisson mean $\lambda = 0.2$ false objects/scan	Mean 0.199; dispersion test $p = 0.48$	Pass

Continued on next page

Table C.1: Additional test cases (continued).

ID	Description	FR/NFR	Input	Expected	Actual	Status
TC22	Two-point track initiation, coasting and deletion	FR09	Object at 0, 5, 10 and 15 m/s detected twice; then missed 2, 3 scans	Second detection associated within the 20 m initiation gate and velocity set from it; confirmed at 2 hits at every speed; coasting track withheld from the object list; deleted after 3 misses	Confirmed in 4/4 speeds (1/4 before the fix); coasting tracks withheld; deletion correct	Pass
TC23	Map-matching to approach and lane	FR09	One simulated hour, SUMO ground-truth vehicles vs the controller's view of each approach	$\geq$ 95% of *actual* stopped vehicles within range reported on the right approach, including those on the intermediate-node internal lane	97.9% correct (27–31% on B1/X1 before the connector fix)	Pass
TC24	Fail-safe entry and recovery	FR13, NFR03	VLS output removed, then restored, during a one-hour run	Green held for 1–4 missing scans; hand-over at the 5th through amber and all-red, B1 starting at ϕ_1; adaptive control resumes only at the start of a B1 ϕ_1 after 10 good scans; no unsafe state	Entry and recovery as specified on all 10 seeds; 0 violations	Pass
TC25	Baseline controllers and ablation	FR15	B1, B2, B3, PRISM thesis law and PRISM-GT, one hour each	B1 reproduces 21/8/15/14 s greens in a 78 s cycle; B2 respects 10/5 and 50/25 s limits; B3 uses the per-scenario Webster plan; the thesis law differs from PRISM-R by parameters only	Timings match the plans exactly; ablation reproduced the first-evaluation results bit-for-bit	Pass
TC31	Resting rules	FR10, FR11	Empty junction; then demand on the running phase only	Rest in all-red, first detected vehicle served after the clearance interval alone; green held rather than surrendered	As expected in both cases	Pass
TC32	Cyclic selection with skip	FR10, FR12	Heavy ϕ_1/ϕ_3 demand with intermittent right-turn demand	Right-turn phases served in the order $\phi_1 \rightarrow \phi_4$ whenever they have demand, skipped when empty; no reliance on the starvation override	Right turns served every cycle in which they had demand; longest right-turn red 74 s	Pass
TC33	Hidden queue at the range edge	FR09, FR10	Queue of 180 m built on B1, far beyond the observable range	Hidden queue accumulates at the learnt rate, is capped at 60 PCU, and clears when all objects are within 30 m	Estimated queue within 12% of ground truth; cleared correctly	Pass
TC26	Seed reproducibility	NFR05, FR16	Same scenario and seed run twice from a clean checkout	Identical KPI files	Bit-identical outputs	Pass

Continued on next page

Table C.1: Additional test cases (continued).

ID	Description	FR/NFR	Input	Expected	Actual	Status
TC27	Batch experiments and statistics	FR16, FR17	Full experiment grid, then `stats.py`	One KPI row per run; Wilcoxon p equals independent SciPy computation; plots generated	150 rows; p-values identical	Pass
TC28	Decision logging and GUI overlay	NFR04, FR18	Afternoon run in SUMO-GUI	Every decision logged with rule fired and all S_ϕ; phase and reason shown in GUI	100% of 3,600 decisions logged with reason	Pass
TC29	Privacy check	NFR07	All stored outputs of DTX and experiments	No images, faces or number plates stored; only anonymous IDs and trajectories	No personal data found	Pass
TC30	Portability and maintainability	NFR08, NFR09	Static import check; `pytest-cov`	Controller imports no SUMO/TraCI module; coverage $\geq$ 80%	0 SUMO imports; coverage 87%	Pass

APPENDIX D: ADDITIONAL CODE LISTINGS

This appendix gives longer versions of the code discussed in Section 5.4. The complete repository (`prism-traffic/`) contains the remaining modules and the configuration files.

D.1 PRISM controller (`control/prism.py`)

Listing D.1: `PrismController`: state, phase selection and fail-safe

```
from dataclasses import dataclass, field
from .phases import PHASES, EXIT, MOVEMENTS, Phase, PhaseManager

Q_SPEED = 1.4 # m/s, queued if slower (about 5 km/h)

@dataclass
class PrismParams:
    gamma: float = 0.5; delta: float = 0.6; kappa: float = 1.5
    theta: float = 0.10; eps: float = 2.0; rho: float = 1.0
    H: float = 10.0; tau_gap: float = 3.0
    l_s: float = 2.0; h_s: float = 3600 / 1850
    T_starve: float = 120.0; spill_ratio: float = 0.85; C_exit: float = 23.0
    failsafe_after: int = 5; recover_after: int = 10
    # --- PRISM-R revisions; the thesis law is this class with these disabled
    freeze_g_req: bool = True # evaluate G_req once per green
    edge_blend: float = 0.5 # eta in eq. (5.17)
    range_m: float = 60.0 # observable range from the stop line
    edge_m: float = 50.0 # queue counts as touching the range edge
    hidden_in_m: float = 45.0; hidden_clear_m: float = 30.0
    hidden_tau: float = 120.0; hidden_cap: float = 60.0
    occlusion_hold: float = 5.0
    cyclic: bool = True # cyclic-with-skip selection
    rest_in_green: bool = True; rest_in_red: bool = True

@dataclass
class MovState:
    Q: float = 0.0; A: float = 0.0; min_eta: float = float("inf")
    at_edge: bool = False

class PrismController:
    def __init__(self, pm: PhaseManager, shares: dict, p: PrismParams =
        PrismParams()):
        self.pm, self.shares, self.p = pm, shares, p
        self.green: Phase = PHASES[0]
        self.green_start, self.g_req_frozen = 0.0, PHASES[0].gmin
        self.last_green_end = {ph.id: 0.0 for ph in PHASES}
        self.occ: dict = {}
        self.hidden = {m: 0.0 for m in MOVEMENTS} # unobserved queue beyond range
        self.lam = {m: 0.0 for m in MOVEMENTS} # EWMA arrival rate at the edge
        self.held: dict = {} # occluded stopped objects, 5 s
        self.missing, self.valid, self.failsafe = 0, 0, False

    # ---------- per-step entry point ----------------------------------
    def step(self, objs, t_now: float) -> "Decision":
```

```
        if self.pm.in_intergreen(t_now):
            if self.pm.intergreen_ends(t_now):
                nxt = self.select_next(objs, t_now)
                if nxt is None: # rest in all-red until a vehicle appears
                    return Decision(HOLD, "resting_all_red", {})
                self.start_green(nxt, t_now, self.movement_state(objs))
            return Decision(HOLD, "intergreen", {})
        if objs is None and 0 < self.missing < self.p.failsafe_after:
            return Decision(EXTEND, "sensor_gap", {}) # hold green, 1-4 scans
        if not self.sensor_ok(objs):
            return self.pm.run_fixed_plan("B1_fixed78", t_now)
        d = self.decide(objs, t_now) # Listing 5.4
        if d.action == TERMINATE:
            self.last_green_end[self.green.id] = t_now
            self.pm.start_intergreen(self.green, t_now) # 3 s amber + 2 s all-red
        return d

    # ---------- helpers ----------------------------------------------------
    def red_time(self, ph: Phase, t_now: float) -> float:
        return 0.0 if ph is self.green else t_now - self.last_green_end[ph.id]

    def g_req(self, ph: Phase, st) -> float:
        """Green needed to clear the queue present NOW. Called once per green."""
        load = sum(st[m].Q + self.p.gamma * st[m].A for m in ph.movements)
        g = min(max(self.p.l_s + self.p.h_s * load / ph.n_lanes, ph.gmin), ph.
            gmax)
        if any(st[m].at_edge for m in ph.movements): # range-edge blend
            g += self.p.edge_blend * (ph.gmax - g)
        return g

    def spillback(self, ph: Phase) -> bool:
        return any(self.occ.get(EXIT[m], 0.0) / self.p.C_exit > self.p.
            spill_ratio
                   for m in ph.movements)

    def is_waiting(self, ph: Phase, st) -> bool:
        """Any queued or arriving PCU -- NOT 'pressure > 0', which a congested
        exit can drive negative while vehicles are still waiting."""
        return sum(st[m].Q + st[m].A for m in ph.movements) > 0.0

    def can_discharge(self, ph: Phase, st) -> bool:
        return self.is_waiting(ph, st) and not self.spillback(ph)

    def any_occupied(self, st) -> bool:
        return any(self.is_waiting(ph, st) for ph in PHASES)

    def starved(self, st, t_now, exclude=None) -> list[Phase]:
        s = [ph for ph in PHASES if ph is not exclude and self.can_discharge(ph,
            st)
             and self.red_time(ph, t_now) >= self.p.T_starve]
        return sorted(s, key=lambda ph: -self.red_time(ph, t_now)) # oldest first

    def select_next(self, objs, t_now: float) -> Phase | None:
        st = self.movement_state(objs)
        hungry = self.starved(st, t_now, exclude=self.green)
        if hungry:
            return hungry[0]
        ready = [ph for ph in PHASES if ph is not self.green
```

```
                and self.can_discharge(ph, st)]
        if not ready:
            return None # rest in all-red
        if self.p.cyclic: # cyclic with skip, PHASES = phi1..phi4
            i = PHASES.index(self.green)
            order = PHASES[i + 1:] + PHASES[:i + 1]
            return next(ph for ph in order if ph in ready)
        S = self.scores(st, t_now) # thesis law: acyclic arg-max
        return max(ready, key=lambda ph: S[ph.id])

    def start_green(self, ph: Phase, t_now: float, st) -> None:
        self.green, self.green_start = ph, t_now
        self.g_req_frozen = self.g_req(ph, st) # held for the whole green
        self.pm.set_green(ph)

    def sensor_ok(self, objs) -> bool:
        if objs is None: # no data packet (not "no vehicles")
            self.missing, self.valid = self.missing + 1, 0
        else:
            self.missing, self.valid = 0, self.valid + 1
        if self.missing >= self.p.failsafe_after and not self.failsafe:
            self.pm.start_intergreen(self.green, self.t_now) # amber + all-red
                first
            self.pm.load_fixed_plan("B1_fixed78", start_phase=0) # B1 begins at
                phi1
            self.failsafe = True
        elif (self.failsafe and self.valid >= self.p.recover_after
              and self.pm.fixed_plan_at_start_of("phi1")):
            self.failsafe = False
        return not self.failsafe
```

D.2 Perception tracker (`sensing/perception_tracker.py`)

Two details matter here. A tentative track has no velocity estimate, so its second detection is associated with the wide *initiation gate* $v_{\max}\Delta t \approx 20$ m rather than the steady-state 2.5 m gate; with a 2.5 m gate and $\hat{\mathbf{v}} = \mathbf{0}$, no vehicle faster than 2.5 m/s could ever be confirmed. Second, only tracks updated in the current scan are returned: unassociated tracks are still coasted internally so that they survive a dropout, but reporting them put stale objects into the controller's queue estimate.

Listing D.2: Two-point track initiation, α–β filtering, and withholding of coasting tracks

```
import numpy as np

V_MAX = 20.0 # m/s, fastest credible approach speed

class Track:
    def __init__(self, tid, det, t):
        self.id, self.p, self.v = tid, np.array([det.x, det.y]), np.zeros(2)
        self.size, self.hits, self.misses, self.t = det.size, 1, 0, t
        self.updated = True # associated in the current scan?

class PerceptionTracker:
    def __init__(self, alpha=0.5, beta=0.1, gate=2.5, delete=3, dt=1.0):
```

```
        self.a, self.b, self.gate, self.delete = alpha, beta, gate, delete
        self.init_gate = V_MAX * dt # ~20 m for the second detection
        self.tracks: list[Track] = []
        self.next_id = 0

    def update(self, dets, t, dt=1.0):
        if dets is None:
            return None # propagate sensor failure
        for tr in self.tracks:
            tr.pred = tr.p + tr.v * dt # prediction
            tr.updated = False
        free = set(range(len(dets)))
        # established tracks first, then tentative ones with the wider gate
        for tr in sorted(self.tracks, key=lambda k: -k.hits):
            gate = self.gate if tr.hits >= 2 else self.init_gate
            best, dmin = None, gate
            for j in free:
                d = np.hypot(dets[j].x - tr.pred[0], dets[j].y - tr.pred[1])
                if d < dmin:
                    best, dmin = j, d
            if best is None:
                tr.p, tr.misses = tr.pred, tr.misses + 1 # coast, do not report
                continue
            z = np.array([dets[best].x, dets[best].y])
            if tr.hits == 1:
                tr.v = (z - tr.p) / dt # two-point initiation
                tr.p = z
            else:
                r = z - tr.pred # innovation
                tr.p = tr.pred + self.a * r
                tr.v = tr.v + (self.b / dt) * r
            tr.size, tr.hits, tr.misses = dets[best].size, tr.hits + 1, 0
            tr.updated = True
            free.discard(best)
        for j in free: # new tentative tracks
            self.tracks.append(Track(self.next_id, dets[j], t)); self.next_id +=
                1
        self.tracks = [k for k in self.tracks if k.misses < self.delete]
        # report only tracks seen this scan and confirmed by a second detection
        return [map_match(k) for k in self.tracks if k.updated and k.hits >= 2]
```

D.3 Route-file writer (`extraction/export_sumo.py`)

Listing D.3: Writing the DTX demand as a SUMO route file

```
import pandas as pd
from xml.sax.saxutils import quoteattr

VTYPES = ["motorcycle", "threewheeler", "car", "van", "bus", "lorry", "bicycle"
    ]

def write_routes(trips: pd.DataFrame, routes: dict, session: str, path: str) ->
     None:
    """trips: one row per uncensored or censored vehicle with movement, class,
        t_entry."""
    share = trips.cls.value_counts(normalize=True).reindex(VTYPES, fill_value
        =0.0)
```

```
    trips["bin"] = (trips.t_entry // 300).astype(int) # 5-minute bins
    counts = trips.groupby(["bin", "movement"]).size()
    with open(path, "w") as f:
        f.write('<routes>\n <vTypeDistribution id="mix_%s">\n' % session)
        for vt, p in share.items():
            f.write(' <vType id="%s_%s" probability="%.4f"/>\n' % (vt, session, p
                ))
        f.write(" </vTypeDistribution>\n")
        for mv, edges in routes.items():
            f.write(' <route id=%s edges=%s/>\n' % (quoteattr(mv), quoteattr(
                edges)))
        for (b, mv), n in counts.items():
            f.write(' <flow id="%s_%02d" type="mix_%s" route=%s begin="%d" end="%
                d" '
                    'vehsPerHour="%.1f" departLane="best" departSpeed="max"/>\n'
                    % (mv, b, session, quoteattr(mv), b * 300, (b + 1) * 300, n *
                        12.0))
        f.write("</routes>\n")
```

D.4 Phase-safety test (`tests/test_phase_safety.py`)

Listing D.4: pytest checking conflicting greens and intergreens on every step

```
import pytest
from experiments.run_experiment import iter_states
from control.phases import CONFLICTS # pairs of link indices that must never
    both be G

CONTROLLERS = ["B1", "B2", "B3", "PRISM", "PRISM_GT"]

@pytest.mark.parametrize("ctrl", CONTROLLERS)
@pytest.mark.parametrize("demand", [300, 500, 700])
def test_no_conflicting_greens_and_full_intergreen(ctrl, demand):
    prev, amber_since, red_since = None, None, None
    for t, state in iter_states(ctrl, f"mock_{demand}", seed=1, horizon=1800):
        green = {i for i, c in enumerate(state) if c in "Gg"}
        for a, b in CONFLICTS:
            assert not (a in green and b in green), f"{ctrl} t={t}: {a}/{b} green
                "
        if "y" in state and (prev is None or "y" not in prev):
            amber_since = t
        if set(state) == {"r"} and prev and "y" in prev:
            assert t - amber_since >= 3.0 - 1e-6, f"short amber at t={t}"
            red_since = t
        if prev and set(prev) == {"r"} and green:
            assert t - red_since >= 2.0 - 1e-6, f"short all-red at t={t}"
        if prev and green and any(c == "G" for c in prev):
            assert green <= {i for i, c in enumerate(prev) if c == "G"}, \
                f"green-to-green change without intergreen at t={t}"
        prev = state
```

APPENDIX E: PHASE SIGNAL-STATE STRINGS

SUMO describes the state of a traffic light as a string with one character per controlled *link*, where a link is a connection from an incoming lane to an outgoing lane through the junction. The characters used here are: `G` for a protected green (priority movement); `g` for a permissive green that must yield to conflicting traffic; `y` for amber; and `r` for red. PRISM and all baselines use only protected greens, so `g` never appears. Right turns are served only in their own protected phases, and opposing through movements do not conflict.

The four channelised left-turn slip lanes are *not* part of the traffic-light program. They join the exit arms through unsignalised yield links and therefore have no character in the strings. Each arm thus contributes three signalised links, and the junction has twelve. The link order assumed in this thesis is given in Table E.1, with arms numbered clockwise from the first bypass arm. **The actual index order generated by `netconvert` must be checked in `netedit` (traffic-light mode, "show link indices") and the strings below re-ordered if necessary.**

Table E.1: Assumed link-index order of the traffic-light program

Index	Arm	From lane	Movement
0	Bypass 1	kerb lane	through
1	Bypass 1	offside lane	through
2	Bypass 1	offside lane	right
3	Cross 1	kerb lane	through
4	Cross 1	offside lane	through
5	Cross 1	offside lane	right
6	Bypass 2	kerb lane	through
7	Bypass 2	offside lane	through
8	Bypass 2	offside lane	right
9	Cross 2	kerb lane	through
10	Cross 2	offside lane	through
11	Cross 2	offside lane	right
–	every arm	slip lane	left (unsignalised yield, not in TLS)

Table E.3 lists the fail-safe program. It is identical to the observed plan (B1) and is loaded as the static program `B1_fixed78` when the VLS delivers no data for five consecutive scans; the hand-over itself runs the normal amber and all-red, and the fixed plan is entered at step 0 so that it always begins at ϕ_1. PRISM-R chooses the next phase cyclically with skipping rather than by a free arg-max, so the phase order drivers see is $\phi_1 \rightarrow \phi_2 \rightarrow \phi_3 \rightarrow \phi_4$ with empty phases omitted; in all cases an amber is always followed by the all-red state before any other green. When no phase has anything to discharge the controller holds the all-red state indefinitely, which is a legal state of this table rather than a new one. The safety test

Table E.2: Signal-state strings for the four phases, intergreens and fail-safe plan

Phase	State string (index 0 → 11)	Duration	Used by
ϕ_1 green (bypass through)	`GGrrrrGGrrrr`	variable	all controllers
ϕ_1 amber	`yyrrrryyrrrr`	3 s	all
ϕ_2 green (bypass right)	`rrGrrrrrGrrr`	variable	all
ϕ_2 amber	`rryrrrrryrrr`	3 s	all
ϕ_3 green (cross through)	`rrrGGrrrrGGr`	variable	all
ϕ_3 amber	`rrryyrrrryyr`	3 s	all
ϕ_4 green (cross right)	`rrrrrGrrrrrG`	variable	all
ϕ_4 amber	`rrrrryrrrrry`	3 s	all
All-red (after every amber)	`rrrrrrrrrrrr`	2 s	all

in Appendix D checks this transition rule and the absence of conflicting `G` pairs on every simulation step.

Table E.3: Fail-safe (B1) program, cycle 78 s

Step	Meaning	State string	Duration (s)
0	ϕ_1 green	`GGrrrrGGrrrr`	21
1	amber	`yyrrrryyrrrr`	3
2	all-red	`rrrrrrrrrrrr`	2
3	ϕ_2 green	`rrGrrrrrGrrr`	8
4	amber	`rryrrrrryrrr`	3
5	all-red	`rrrrrrrrrrrr`	2
6	ϕ_3 green	`rrrGGrrrrGGr`	15
7	amber	`rrryyrrrryyr`	3
8	all-red	`rrrrrrrrrrrr`	2
9	ϕ_4 green	`rrrrrGrrrrrG`	14
10	amber	`rrrrryrrrrry`	3
11	all-red	`rrrrrrrrrrrr`	2
	Cycle		78

APPENDIX F: DRONE SURVEY LOG

This appendix records the drone survey of the Mavithara junction carried out on Friday, 2 October 2026, the approvals under which it was flown and the parameters used to process the footage with DTX. The aircraft was flown by a CAASL-registered operator. Values marked in yellow are to be confirmed against the flight logs and video metadata.

Table F.1: Flight log for 2 October 2026.

Item	Session M (morning peak)	Session A (afternoon)	Session E (evening peak)
Planned window	07:00–08:00	13:30–14:30	17:00–18:00
Traffic condition	High congestion	Moderate	—
Actual recording	07:01–07:59	13:31–14:29	Not recorded
Usable footage	55 min	56 min	—
Altitude	≈90 m AGL	≈90 m AGL	—
Aircraft	DJI Mini 2	DJI Mini 2	—
Camera	4K (3840×2160), 30 fps, nadir, stationary hover	4K (3840×2160), 30 fps, nadir, stationary hover	—
Video segments	3	3	—
Battery swaps	2 (each gap under 90 s)	2 (each gap under 90 s)	—
Weather	Dry, overcast	Dry, partly cloudy	Rain and poor visibility
Notes	Queues on the bypass extended beyond the frame at times; take-off point kept fixed between segments	Demand below capacity; queues cleared in most cycles	Session cancelled because of bad weather; listed as a limitation and future work

Table F.2: Summary of approvals for the drone survey.

Date	Authority	Reference	Purpose and conditions
12.06.2026	NSBM Green University to Officer in Charge, Piliyandala Police Station	University request letter	Permission to study key intersections in Piliyandala and Kesbewa (scope later narrowed to Maviththara)
17.09.2026	Piliyandala Traffic Police and RDA	Discussion (no reference)	Support for the research; needs: less manual traffic control, less waiting for all road users
22.09.2026	Piliyandala Police Station to Director General, CAASL	PLI/AD/OUT/…/ 2026	Supports drone use at the Maviththara signal on 02.10.2026, windows 07:00–08:00, 13:30–14:30 and 17:00–18:00
25.09.2026	Ministry of Defence	MOD/MLO/ DRONE/03/2026 (7969)	Security clearance for educational purpose; 02.10.2026, 06:00–18:00; maximum 400 ft AGL; visual line of sight; inform the Sri Lanka Air Force and police
01.10.2026	Civil Aviation Authority of Sri Lanka	OP/36/2	Flight approval under SLCAIS 053 Rev.03

Table F.3: DTX processing parameters for the survey footage.

Parameter	Value
Source resolution and frame rate	3840×2160 at 30 fps
Processing frame rate	10 fps (every third frame)
Clip lengths, Session M	19 min 40 s, 18 min 50 s, 16 min 30 s
Clip lengths, Session A	19 min 45 s, 19 min 10 s, 17 min 05 s
Frames processed	≈33,000 (M), ≈33,600 (A)
Stabilisation	ORB features + RANSAC homography to one reference frame per session
Detector	YOLOv8s (VisDrone2019-DET pre-trained, fine-tuned on 450 frames), SAHI 2×2 tiles, overlap 0.2
Tracker	ByteTrack (track threshold 0.5, match threshold 0.8, buffer 30 frames)
Ground sample distance	≈2.6 cm/px
Smoothing	Savitzky–Golay, window 11, order 2
Waiting threshold	speed < 0.1 m/s inside an approach zone
Segment boundaries	Tracks cut by segment start/end treated as censored (counted in flows, excluded from averages)
Data retained	Anonymous trajectories (track ID, class, time, position, speed) and 5-min aggregated flows only
Raw video	Held offline on an encrypted drive; deleted after processing and examination (policy)

APPENDIX G: SUPERVISOR MEETING LOG

Table G.1: Supervisor meeting log (placeholder entries)

Date	No.	Discussion points	Actions agreed	Supervisor signature
12.06.2026	01	How to get the approval from RDA and Sri Lanka Police to record the roads ethically	Drafted a letter, got the approval from the supervisor	